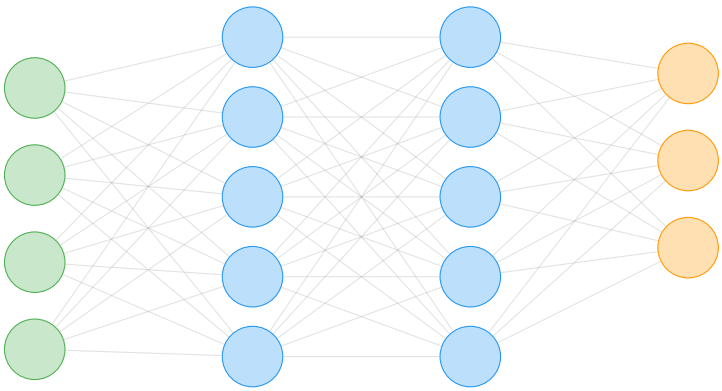
神经网络

面向人工智能专业学生的综合指南



G. Qu 等

云南大学
信息学院

中国昆明

三月 2026

许可证

This work is licensed under a Creative Commons “Attribution-NonCommercial-ShareAlike 4.0 International” license.

关于本书

本书为人工智能领域的学生、自学者和从业者提供了一本结构清晰、内容全面的神经网络入门教材。本书从数学基础和激活函数入手，逐步展开架构、优化、训练、数据处理和现代大规模模型的核心思想，特别注重概念清晰度和实际实现。

主要特色：

- 从数学基础到 Transformer 和现代训练技术的完整 13 章学习路径
- 全书包含严谨的数学推导、直观的解释和精心设计的图表
- 以 PyTorch 生态系统为中心的实际实现示例
- 涵盖 CNN、RNN、跳跃连接和注意力等核心架构和机制
- 包含优化、训练流程、数据集处理和计算效率的专门章节
- 精心策划的参考文献库，连接经典论文与现代发展

作者： G. Qu 等
机构：云南大学, 信息学院
年份： 2026

摘要

神经网络彻底改变了人工智能领域，使计算机视觉、自然语言处理、语音识别等众多领域实现了重大突破。本教材全面介绍了神经网络，从基本的数学概念到最新的网络架构和训练技术。

本书共分十三章，内容涵盖：

1. 数学基础：线性代数（SVD、特征分解、条件数）、微积分（泰勒展开、海森矩阵、凸性）、概率论（最大似然估计、最大后验估计、信息论）和优化理论。
2. 激活函数：对 Sigmoid、tanh、ReLU 及其变体（Leaky ReLU、PReLU、ELU）、Swish/SiLU、Mish、GELU、StarReLU 进行了全面分析，并给出了通用近似定理的证明和选择标准指导。
3. 损失函数：详细探讨了回归损失（MSE、Huber、MAE）、分类损失（交叉熵、Focal Loss、标签平滑）以及对比损失和 Dice 损失等任务特定目标函数。
4. 神经网络架构：深度神经网络（DNN）、卷积神经网络（CNN）和循环神经网络（RNN/LSTM/GRU），包含数学推导和实际示例。
5. 核心操作：卷积（标准卷积、膨胀卷积、深度可分离卷积、转置卷积）、池化和归一化操作（Batch Norm、Layer Norm、Group Norm、Instance Norm），具有数学严谨性。
6. 高级机制：跳跃连接（残差连接、密集连接）、通道注意力（SE 模块、CBAM）和空间注意力机制，包括梯度流的理论分析。
7. 优化：梯度下降变体（SGD、动量、Nesterov）、自适应优化器（AdaGrad、RMSProp、Adam、AdamW、AMSGrad）以及学习率调度器（余弦退火、周期性调度、热重启）。
8. 经典架构：按时间顺序介绍从 LeNet-5（1998）和 AlexNet（2012）到 VGGNet、GoogLeNet/Inception、ResNet、DenseNet、MobileNet、SENet、EfficientNet、Vision Transformer（ViT，2020）、U-Net 和 ConvNeXt。
9. 计算分析：全连接层、卷积层、注意力层和归一化层的参数量统计和 FLOPs 计算，以及模型压缩和缩放定律。
10. 训练流程与框架：实际训练流程、正则化技术（Dropout、权重衰减、早停）、超参数调优、混合精度训练，以及 PyTorch 实现与 CUDA 概念。

11. 数据集处理：数据收集、标注、增强策略（Mixup、CutMix、AutoAugment、RandAugment）、划分策略，以及图像、文本和音频的数据加载管道。
12. **Transformer** 架构：自注意力机制、多头注意力、位置编码（正弦编码、RoPE、ALiBi）、编码器-解码器架构、BERT、GPT，以及现代大语言模型（包括 LLaMA 和缩放定律）的概述。
13. 现代训练技术：预训练和微调范式、参数高效微调（LoRA、适配器）、对比学习（SimCLR、InfoNCE）、掩码自编码（MAE），以及灾难性遗忘和多任务学习的进阶主题。

每章都包含详细的数学推导、直观的解释、PyTorch 代码示例、自我评估练习以及经典和最新研究论文的参考文献。完成本教材后，读者将能够为各种实际应用设计、实现、训练和分析神经网络。

关键词：神经网络、深度学习、机器学习、CNN、RNN、LSTM、Transformer、注意力机制、自注意力、优化、反向传播、PyTorch、计算机视觉、自然语言处理、大语言模型、迁移学习、微调

致谢

我们向所有为这本教材的编写做出贡献的人表示衷心的感谢。

首先，我们感谢那些为现代神经网络奠定基础的先驱研究者，包括 Yann LeCun、Geoffrey Hinton、Yoshua Bengio，以及许多其他为深度学习领域做出贡献的学者。

感谢 云南大学 的同事们在教材开发过程中提供的宝贵反馈和建议。

特别感谢开源社区，特别是 PyTorch、NumPy 和其他工具的开发者们，正是他们的努力使深度学习对全球的研究者和从业者变得触手可及。

最后，感谢我们的家人和朋友在整个项目过程中的耐心和支持。

作者

三月 2026

献给所有渴望理解
学习机器数学原理
的好奇心灵，
以及将推动
人工智能边界不断前行的
未来研究者。

- 作者 -

Contents

目录 **20**

插图目录 **21**

表格目录 **24**

代码清单目录 **25**

1 数学基础 **1**

1.1 引言 1

1.2 线性代数 1

1.2.1 向量与矩阵 1

1.2.2 矩阵运算 2

1.2.3 范数 2

1.2.4 特征值与奇异值 3

1.2.4.1 为什么 SVD 很重要：几何解释 3

1.2.4.2 低秩近似：Eckart–Young 定理 4

1.2.4.3 特征分解与谱定理 4

1.2.4.4 条件数与数值稳定性 5

1.2.4.5 正定矩阵与 Cholesky 分解 5

1.2.4.6 行列式、迹与 Jacobian 行列式 6

1.2.5 张量 7

1.3 微积分 8

1.3.1 导数与梯度 8

1.3.2 高阶导数 9

1.3.2.1 泰勒展开：理论桥梁 9

1.3.2.2 凸性与损失景观 10

1.3.2.3 Lipschitz 连续与梯度裁剪 10

1.3.3 Jacobian 矩阵 11

1.4 概率论 11

1.4.1 基本概率 12

1.4.2 常见分布 12

1.4.3 贝叶斯概率 13

1.4.4 最大似然估计 13

1.4.5 贝叶斯推断与 MAP 估计 14

1.4.6 共轭先验与指数族 15

1.5 优化 15

1.5.1 优化问题 16

1.5.2 梯度下降 16

1.5.3 优化中的挑战 16

1.6 信息论 17
1.6.1 熵 17
1.6.1.1 证明：通过 Jensen 不等式证明 $D_{KL}(p\|q) \geq 0$ 18
1.6.1.2 互信息 19
1.6.2 交叉熵与 KL 散度 20
1.7 本章小结 21
1.8 扩展阅读 21
1.9 练习 22

2 激活函数 23
2.1 引言 23
2.2 为什么需要非线性？——通用逼近定理 23
2.2.1 定理 23
2.2.2 直观解释 24
2.2.3 注意事项和实际意义 24
2.3 激活函数的历史发展 24
2.4 神经网络神经元 25
2.5 线性激活函数 26
2.5.1 恒等函数 26
2.6 Sigmoid 族 26
2.6.1 Sigmoid（逻辑斯蒂） 26
2.6.1.1 数学分析：为什么 Sigmoid 导致梯度消失 28
2.6.1.2 导数推导 28
2.6.2 双曲正切（Tanh） 29
2.7 修正线性单元（ReLU）族 30
2.7.1 ReLU 30
2.7.1.1 为什么 ReLU 防止梯度消失：数学分析 31
2.7.1.2 ReLU、稀疏性与生物学动机 31
2.7.2 Leaky ReLU 32
2.7.3 参数化 ReLU（PReLU） 33
2.7.4 指数线性单元（ELU） 33
2.7.5 Swish / SiLU 34
2.7.6 Mish 34
2.7.7 StarReLU 36
2.7.8 GELU（高斯误差线性单元） 36
2.7.8.1 概率解释：GELU、Dropout 和高斯 CDF 37
2.7.8.2 GELU 的导数 37
2.8 特殊激活函数 38
2.8.1 Softmax 38
2.9 选择激活函数 39
2.10 实践注意事项 39
2.10.1 权重初始化 39
2.10.2 批归一化与激活函数的结合 40
2.11 PyTorch 实现 40

2.12 本章小结 41
2.13 扩展阅读 42
2.14 习题 42

3 损失函数 43
3.1 引言 43
3.2 损失函数的作用 43
3.3 回归损失函数 43
3.3.1 均方误差（MSE） 44
3.3.1.1 推导：MSE 作为高斯噪声下的最大似然估计 44
3.3.1.2 梯度分析 45
3.3.1.3 与方差和 R^2 的联系 45
3.3.2 平均绝对误差（MAE） 45
3.3.2.1 推导：MAE 作为拉普拉斯噪声下的 MLE 45
3.3.2.2 比较：MSE vs MAE 46
3.3.3 Huber 损失：两全其美 46
3.4 分类损失函数 47
3.4.1 从似然到交叉熵：完整推导链 47
3.4.1.1 第 1 步：生成模型 47
3.4.1.2 第 2 步：最大似然 47
3.4.1.3 第 3 步：负对数似然 = 交叉熵 48
3.4.1.4 第 4 步：与信息理论的联系 48
3.4.1.5 第 5 步：美丽的梯度 48
3.4.2 二元交叉熵 49
3.4.3 分类交叉熵 49
3.5 高级损失函数 50
3.5.1 Focal 损失 50
3.5.1.1 动机：类别不平衡问题 50
3.5.1.2 调制因子 50
3.5.1.3 调制因子分析 50
3.5.1.4 Alpha 平衡 Focal 损失 51
3.5.2 标签平滑 51
3.5.2.1 机制 52
3.5.2.2 为什么标签平滑有效？三个视角 52
3.6 对比学习和自监督损失函数 53
3.6.1 对比损失 53
3.6.2 三元组损失 53
3.6.3 InfoNCE 损失 53
3.6.3.1 与互信息的联系 54
3.6.3.2 温度 τ 的作用 54
3.6.4 Dice 损失 54
3.6.4.1 推导：Dice 系数和 Jaccard 指数 54
3.6.5 KL 散度损失 55
3.7 选择损失函数 55

3.8 本章小结 55
3.9 扩展阅读 56
3.10 习题 57

4 神经网络架构 59
4.1 引言 59
4.2 深度神经网络（DNN） 59
4.2.1 架构 59
4.3 卷积神经网络（CNN） 60
4.3.1 卷积层 60
4.3.2 感受野分析 60
4.3.2.1 单层感受野 61
4.3.2.2 多层感受野 61
4.3.2.3 有效感受野 61
4.3.3 1×1 卷积：跨通道混合 61
4.3.4 转置卷积（反卷积） 62
4.4 循环神经网络（RNN） 62
4.4.1 架构 62
4.4.2 RNN 中的梯度消失：数学证明 63
4.4.2.1 时间反向传播（BPTT） 63
4.4.2.2 谱分析 64
4.5 长短期记忆网络（LSTM） 64
4.5.1 LSTM 门 65
4.5.2 为什么 LSTM 解决梯度消失 65
4.5.2.1 通过细胞状态的梯度流动 65
4.5.2.2 比较：RNN vs LSTM 梯度 66
4.6 门控循环单元（GRU） 67
4.6.1 GRU 方程 67
4.7 选择架构 67
4.8 本章小结 67
4.9 扩展阅读 68
4.10 习题 68

5 神经网络核心操作 69
5.1 引言 69
5.2 卷积操作 69
5.2.1 2D 卷积 69
5.2.2 分组卷积和深度卷积 69
5.2.3 空洞（带孔）卷积 70
5.3 池化操作 71
5.3.1 最大池化 71
5.3.2 平均池化 71
5.3.3 全局池化 72
5.4 归一化技术 72

5.4.1 批归一化 72
5.4.2 层归一化 73
5.4.3 实例归一化 74
5.4.4 组归一化 74
5.5 正则化操作 74
5.5.1 Dropout 75
5.5.1.1 为什么 Dropout 有效：贝叶斯解释 75
5.5.1.2 Dropout 变体 75
5.6 组合操作 76
5.6.1 基本卷积块 76
5.7 本章小结 77
5.8 扩展阅读 77
5.9 习题 78

6 跳跃连接与注意力机制 79
6.1 引言 79
6.2 跳跃连接 79
6.2.1 残差连接（ResNet） 79
6.2.2 为什么残差连接有效：理论分析 80
6.2.2.1 梯度流视角 80
6.2.2.2 恒等映射假设 80
6.2.3 预激活 ResNet 81
6.2.4 瓶颈架构 81
6.2.5 ResNeXt：残差块中的分组卷积 81
6.2.6 随机深度（DropPath） 82
6.2.7 密集连接（DenseNet） 82
6.3 注意力机制 82
6.3.1 自注意力（缩放点积） 82
6.3.1.1 为什么要按 $\sqrt{d_k}$ 缩放？ 83
6.3.1.2 计算复杂度 83
6.3.2 多头注意力 84
6.3.3 交叉注意力 84
6.3.4 位置编码 85
6.3.4.1 正弦位置编码 85
6.3.4.2 现代替代方案 85
6.3.5 压缩激励（SE）注意力 86
6.3.6 CBAM：卷积块注意力模块 86
6.3.7 窗口注意力（Swin Transformer） 87
6.3.8 高效注意力变体 87
6.4 本章小结 88
6.5 扩展阅读 89
6.6 习题 89

7 优化器与学习率调度器 91

7.1 梯度下降优化 91
7.1.1 批量梯度下降 91
7.1.2 随机梯度下降（SGD） 91
7.1.3 小批量梯度下降 91
7.2 带动量的梯度下降 91
7.3 自适应学习率方法 92
7.3.1 AdaGrad 92
7.3.2 RMSProp 92
7.3.3 Adam（自适应矩估计） 93
7.3.4 AdamW（带权重衰减的 Adam） 94
7.3.5 AdaDelta 94
7.3.6 自适应方法的统一视角 94
7.3.7 收敛速率分析 94
7.4 梯度裁剪 95
7.4.1 值裁剪 95
7.4.2 范数裁剪 95
7.5 学习率调度器 96
7.5.1 步进衰减 96
7.5.2 指数衰减 96
7.5.3 余弦退火 96
7.5.4 预热 97
7.5.5 预热 + 余弦衰减 97
7.5.6 ReduceLROnPlateau 97
7.5.7 OneCycleLR 97
7.6 优化器选择 98
7.7 本章小结 98
7.8 扩展阅读 98
7.9 习题 99

8 经典网络架构 101
8.1 引言 101
8.2 神经网络的发展历程 101
8.3 感知机与早期神经网络 101
8.3.1 感知机（1958 年） 101
8.4 LeNet-5（1998 年） 102
8.5 AlexNet（2012 年） 102
8.6 VGGNet（2014 年） 103
8.7 GoogLeNet / Inception（2014 年） 104
8.8 ResNet（2015 年） 105
8.9 U-Net（2015 年） 106
8.10 DenseNet（2017 年） 106
8.11 MobileNet（2017 年） 107
8.12 ShuffleNet（2018 年） 108
8.13 SENet（2018 年） 108

8.14 EfficientNet（2019 年） 109
8.15 Vision Transformer（ViT，2020 年） 109
8.16 总结与比较 110
8.17 章节小结 110
8.18 延伸阅读 112
8.19 练习题 113

9 参数与计算分析 115
9.1 模型效率的重要性 115
9.2 参数计数 115
9.2.1 全连接层 115
9.2.2 卷积层 116
9.2.3 批量归一化 116
9.3 FLOPs（浮点运算） 116
9.3.1 全连接层的 FLOPs 116
9.3.2 卷积层的 FLOPs 117
9.4 实际示例 117
9.4.1 LeNet-5 参数计数 117
9.4.2 AlexNet 参数计数 117
9.5 迁移学习与效率 117
9.6 详细模型分解 118
9.6.1 ResNet-50 参数分析 118
9.6.2 Transformer 模型缩放 119
9.7 激活内存分析 120
9.7.1 激活内存公式 120
9.8 模型压缩与量化 121
9.8.1 量化 121
9.8.2 其他压缩技术 121
9.9 章节小结 121
9.10 延伸阅读 122
9.11 练习题 122

10 训练过程与框架 123
10.1 引言 123
10.2 训练循环 123
10.2.1 形式化描述 123
10.2.2 为什么小批量 SGD 有效 124
10.3 超参数 125
10.3.1 学习率 125
10.3.1.1 学习率的影响 125
10.3.1.2 学习率范围测试 125
10.3.1.3 学习率预热 125
10.3.2 批次大小 126
10.3.2.1 批次大小的影响 126

10.3.2.2 线性缩放规则 . . . 126
10.4 正则化 . . . 127
10.4.1 Dropout . . . 127
10.4.1.1 带 Dropout 的前向传播 . . . 127
10.4.1.2 反转 Dropout . . . 127
10.4.1.3 集成解释 . . . 128
10.4.1.4 Dropout 作为近似贝叶斯推理 . . . 128
10.4.1.5 Dropout 变体 . . . 128
10.4.2 权重衰减 . . . 128
10.4.2.1 L2 正则化作为 MAP 估计 . . . 128
10.4.2.2 权重衰减 vs. L2 正则化：AdamW . . . 129
10.4.3 其他正则化技术 . . . 130
10.4.3.1 梯度裁剪 . . . 130
10.4.3.2 标签平滑 . . . 130
10.4.3.3 随机深度 . . . 131
10.5 早停 . . . 131
10.5.1 算法 . . . 131
10.5.2 早停作为正则化 . . . 131
10.6 反向传播 . . . 132
10.6.1 计算图视角 . . . 132
10.6.1.1 前向传播 . . . 132
10.6.1.2 反向传播 . . . 132
10.6.2 示例：双层网络 . . . 132
10.6.3 自动微分 . . . 133
10.6.4 常见训练陷阱 . . . 134
10.7 模型检查点保存 . . . 134
10.8 混合精度训练 . . . 134
10.8.1 动机 . . . 134
10.8.2 损失缩放问题 . . . 135
10.8.3 BF16：现代替代方案 . . . 135
10.9 PyTorch 框架 . . . 136
10.10 CUDA 与 GPU 训练 . . . 138
10.10.1 GPU 编程模型 . . . 138
10.10.2 分布式训练 . . . 138
10.10.3 梯度累积 . . . 138
10.11 训练诊断 . . . 139
10.11.1 损失曲线 . . . 139
10.11.2 梯度统计 . . . 139
10.12 章节小结 . . . 140
10.13 延伸阅读 . . . 140
10.14 练习题 . . . 141

11 数据集处理 143
11.1 引言 . . . 143

11.2 数据收集 143
11.2.1 数据来源 143
11.2.2 数据质量标准 143
11.3 数据标注 144
11.3.1 标注类型 144
11.3.2 标注工具 144
11.4 数据划分 145
11.4.1 基本划分 145
11.4.2 分层划分 145
11.4.3 交叉验证 146
11.5 数据增强 146
11.5.1 图像增强 146
11.5.2 Mixup：样本的凸组合 147
11.5.3 CutMix：粘贴补丁 148
11.5.4 高级增强策略 149
11.6 文本数据处理 149
11.6.1 分词 149
11.6.2 字节对编码（BPE） 150
11.7 音频数据处理 150
11.7.1 频谱图计算 150
11.8 数据归一化 151
11.8.1 图像归一化 151
11.9 数据加载 151
11.10 处理数据不平衡 153
11.10.1 不平衡比例 153
11.10.2 不平衡数据技术 153
11.11 数据质量与版本管理 154
11.11.1 数据版本管理 154
11.11.2 数据溯源 154
11.12 评估指标 155
11.12.1 分类指标 155
11.12.2 分割指标 155
11.13 章节小结 155
11.14 延伸阅读 156
11.15 练习题 156

12 Transformer 架构 159
12.1 引言 159
12.2 注意力机制 159
12.2.1 背景：RNN 的局限性 159
12.2.2 缩放点积注意力 159
12.2.3 自注意力 160
12.3 多头注意力 160
12.4 Transformer 架构 161

12.4.1 前馈网络 161
12.4.2 层归一化 161
12.4.3 残差连接 161
12.5 位置编码 161
12.6 编码器-解码器架构 162
12.7 Vision Transformer（ViT） 163
12.8 大型语言模型 164
12.8.1 GPT 架构 164
12.8.2 BERT 架构 164
12.8.3 现代 LLM 164
12.9 效率考量 165
12.9.1 注意力复杂度 165
12.9.2 内存优化 165
12.10 章节小结 165
12.11 延伸阅读 166
12.12 练习题 166

13 现代训练技术 167
13.1 引言 167
13.2 预训练与微调 167
13.2.1 预训练目标 167
13.2.2 微调策略 167
13.3 LoRA：低秩适应 168
13.4 迁移学习 169
13.4.1 特征迁移 169
13.4.2 渐进式解冻 169
13.5 多任务学习 170
13.5.1 硬参数共享 170
13.5.2 软参数共享 170
13.5.3 任务平衡 170
13.6 持续学习 170
13.6.1 灾难性遗忘 170
13.6.2 EWC 正则化 171
13.7 自监督学习 171
13.7.1 对比学习 171
13.7.2 掩码图像建模 171
13.8 蒸馏与压缩 171
13.8.1 知识蒸馏 172
13.8.2 自蒸馏 172
13.9 大批量训练 172
13.9.1 线性缩放规则 172
13.9.2 预热与缩放 172
13.10 高效训练技术 173
13.10.1 梯度检查点 173

13.10.2 卸载 173
13.11 调试训练 173
13.11.1 常见问题与解决方案 173
13.11.2 训练配方 173
13.12 章节小结 174
13.13 延伸阅读 175
13.14 练习题 175

Bibliography **177**

A 数学参考 **189**
A.1 线性代数 189
A.1.1 矩阵运算 189
A.1.2 范数 189
A.1.3 特征值分解 190
A.1.4 奇异值分解（SVD） 190
A.2 微积分 191
A.2.1 常用导数 191
A.2.2 链式法则 191
A.2.3 Jacobian 和 Hessian 192
A.2.4 泰勒展开 192
A.2.5 拉格朗日乘数法 192
A.3 概率与统计 193
A.3.1 贝叶斯定理 193
A.3.2 概率分布 193
A.3.3 期望与方差 193
A.3.4 KL 散度与交叉熵 193
A.3.5 信息论 194

B 快速参考表 **195**
B.1 激活函数 195
B.2 损失函数 195
B.3 常用数据集 196

C PyTorch 快速参考 **197**
C.1 常用操作 197
C.2 模型定义 197
C.3 训练工具 198

D 术语表 **201**
D.1 架构术语 201
D.2 训练术语 201
D.3 层与操作术语 202
D.4 性能指标 202
D.5 其他常用缩写 203

List of Figures

1.1 奇异值谱 4
1.2 条件数与损失等高线 6
1.3 二次损失表面上的梯度下降 17
1.4 包含局部和全局最小值的损失景观示意图 17
1.5 Jensen 不等式可视化 19

2.1 通用逼近的直观理解 24
2.2 单个神经元计算 26
2.3 Sigmoid 函数及其导数 27
2.4 Tanh 函数及其导数 29
2.5 ReLU 函数及其导数 30
2.6 ReLU 变体比较 34
2.7 Swish 激活函数 35
2.8 各种激活函数比较 40

3.1 回归损失函数比较 47
3.2 Focal 损失调制 51

4.1 深度神经网络架构 59
4.2 卷积操作 60
4.3 RNN 架构 63
4.4 LSTM 单元结构 65

5.1 带填充的卷积 70
5.2 最大池化 2x2, 步长 2 72

6.1 残差连接 79
6.2 自注意力机制 83

7.1 学习率调度器 96

10.1 学习率范围测试 126
10.2 双层网络的计算图。实线箭头表示前向计算；红色虚线箭头表示反向梯度流。 133
10.3 训练损失曲线：健康训练（蓝色）vs. 过拟合（红色）。实线：训练损失。虚线：验证损失。 139

11.1 标注粒度：从图像级标签到像素级分割掩码。 144
11.2 图像增强示例：水平翻转、旋转、随机裁剪和颜色抖动。 146

12.1 带残差连接和层归一化的 Transformer 层。 161

List of Tables

1.1 凸性性质及其对优化的影响 10
1.2 深度学习中的常见概率分布 12
1.3 常见共轭先验关系 15

2.1 激活函数发展历史时间表 25
2.2 激活函数选择指南 39
2.3 激活函数快速参考 41

3.1 MSE vs MAE：综合比较 46
3.2 Focal 损失调制因子 51
3.3 损失函数选择指南 56
3.4 完整损失函数决策框架 57

4.1 梯度流动：Vanilla RNN vs LSTM 66
4.2 架构选择指南 67

5.1 归一化比较 73
5.2 归一化方法：统一视图 75
5.3 Dropout 变体 75

6.1 位置编码方法对比 85
6.2 注意力复杂度对比 87

7.1 自适应优化器家族 95
7.2 优化器选择指南 98

8.1 EfficientNet 系列（ImageNet Top-1 精度） 109
8.2 按时间排序的架构比较（ImageNet Top-1 精度） 111

9.1 ResNet-50 逐层参数计数 119
9.2 Transformer 模型规模 119
9.3 量化级别 121

10.1 超参数范围和默认值 127
10.2 Dropout 变体及其特性 129
10.3 微分方法比较 134
10.4 诊断训练失败 134
10.5 浮点格式比较 135
10.6 分布式训练策略 138

11.1 分词策略 150

12.1 现代 LLM 架构比较 165

13.1 常见训练问题与解决方案 173

B.1 激活函数参考 195
B.2 损失函数参考 195
B.3 基准数据集 196

Listings

2.1 PyTorch 中的激活函数 40

5.1 PyTorch 中的卷积块 76

6.1 PyTorch 多头自注意力实现 87

7.1 PyTorch 中的优化器 93

10.1 PyTorch 中的完整训练循环 136
10.2 PyTorch 中的梯度累积 139

11.1 PyTorch 中的 Mixup 和 CutMix 148
11.2 PyTorch 中的高效数据加载 151

12.1 PyTorch 多头注意力实现 162

13.1 PyTorch 中的 LoRA 实现 168

C.1 PyTorch 张量常用操作 197
C.2 PyTorch 模型定义模式 197
C.3 常用训练工具 198

Chapter 1

数学基础

1.1 引言

神经网络建立在坚实的数学基础之上，结合了线性代数、微积分、概率论和优化理论。本章提供了理解神经网络如何学习和预测的基本数学先验知识。我们将在深度学习应用的背景下介绍这些主题，提供理论洞察和实践直觉。

神经网络的数学可以看作是非线性激活函数的线性变换组合，通过基于梯度的方法进行优化。理解这个框架需要熟练掌握向量和矩阵运算、多元函数微分以及随机优化。

1.2 线性代数

线性代数提供了在神经网络中表示和操作数据的语言。神经网络基本上通过线性变换（矩阵乘法）与非线性激活函数结合来运作。

1.2.1 向量与矩阵

向量是有序的数字集合，可以表示 n 维空间中的一个点。在神经网络中，输入数据、隐藏表示和参数都表示为向量或更高维的张量。

定义 **1.1** (向量). 向量 $\mathbf{x} \in \mathbb{R}^n$ 是 n 个实数的有序元组：

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_n \end{bmatrix} = (x_1, x_2, \dots, x_n)^\top \tag{1.1}$$

其中 x_i 表示向量的第 i 个分量。

定义 **1.2** (矩阵). 矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$ 是 m 行 n 列的二维数组：

$$\mathbf{A} = \begin{bmatrix} a_{11} & a_{12} & \cdots & a_{1n} \\ a_{21} & a_{22} & \cdots & a_{2n} \\ \vdots & \vdots & \ddots & \vdots \\ a_{m1} & a_{m2} & \cdots & a_{mn} \end{bmatrix} \tag{1.2}$$

其中 a_{ij} 表示第 i 行第 j 列的元素。

示例 **1.1** (神经网络层作为矩阵运算). 单层全连接神经网络执行以下运算：

$$\mathbf{y} = \mathbf{W}\mathbf{x} + \mathbf{b} \tag{1.3}$$

其中：

- $\mathbf{x} \in \mathbb{R}^{d_{in}}$ 是输入向量
- $\mathbf{W} \in \mathbb{R}^{d_{out} \times d_{in}}$ 是权重矩阵
- $\mathbf{b} \in \mathbb{R}^{d_{out}}$ 是偏置向量
- $\mathbf{y} \in \mathbb{R}^{d_{out}}$ 是输出向量

如果 $d_{in} = 784$（*MNIST* 图像）且 $d_{out} = 128$，则 $\mathbf{W}$ 包含 $784 \times 128 = 100,352$ 个参数。

1.2.2 矩阵运算

- 矩阵加法： $\mathbf{C} = \mathbf{A} + \mathbf{B}$，其中 $c_{ij} = a_{ij} + b_{ij}$
- 标量乘法： $c\mathbf{A}$，其中每个元素都乘以 c
- 矩阵乘法： $\mathbf{C} = \mathbf{A}\mathbf{B}$，其中 $c_{ij} = \sum_k a_{ik} b_{kj}$
- 转置： $(\mathbf{A}^\top)_{ij} = a_{ji}$
- 逐元素（哈达玛）乘积： $(\mathbf{A} \odot \mathbf{B})_{ij} = a_{ij} \cdot b_{ij}$

矩阵乘法性质

对于适当维度的矩阵 $\mathbf{A}$、$\mathbf{B}$ 和 $\mathbf{C}$：

$$\mathbf{A}(\mathbf{B}\mathbf{C}) = (\mathbf{A}\mathbf{B})\mathbf{C} \quad \text{（结合律）} \tag{1.4}$$

$$\mathbf{A}(\mathbf{B} + \mathbf{C}) = \mathbf{A}\mathbf{B} + \mathbf{A}\mathbf{C} \quad \text{（分配律）} \tag{1.5}$$

$$(\mathbf{A}\mathbf{B})^\top = \mathbf{B}^\top \mathbf{A}^\top \quad \text{（乘积的转置）} \tag{1.6}$$

注意：一般情况下 $\mathbf{A}\mathbf{B} \neq \mathbf{B}\mathbf{A}$（不可交换）。

1.2.3 范数

范数衡量向量和矩阵的 “大小” 或 “长度”。

定义 **1.3** (向量范数). 对于向量 $\mathbf{x} \in \mathbb{R}^n$：

- L_1 范数（曼哈顿范数）： $\|\mathbf{x}\|_1 = \sum_{i=1}^n |x_i|$

- L_2 范数（欧几里得范数）：$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^n x_i^2}$
- L_p 范数：$\|\mathbf{x}\|_p = \left(\sum_{i=1}^n |x_i|^p\right)^{1/p}$
- L_∞ 范数（最大范数）：$\|\mathbf{x}\|_\infty = \max_i |x_i|$

在神经网络中，L_2 范数常用于正则化（权重衰减），而 L_1 范数促进稀疏性。

示例 **1.2** (权重衰减正则化). 权重衰减向损失函数添加惩罚项：

$$\mathscr{L}_{reg}(\mathbf{W}) = \mathscr{L}(\mathbf{W}) + \lambda\|\mathbf{W}\|_F^2 \tag{1.7}$$

其中 $\|\mathbf{W}\|_F = \sqrt{\sum_{i,j} w_{ij}^2}$ 是 *Frobenius* 范数（展平矩阵的 L_2 范数）。

1.2.4 特征值与奇异值

特征值和奇异值对于理解线性变换、维度约简和网络分析至关重要。

定义 **1.4** (特征值与特征向量). 对于方阵 $\mathbf{A} \in \mathbb{R}^{n\times n}$，如果存在标量 λ 和非零向量 $\mathbf{v}$ 使得：

$$\mathbf{A}\mathbf{v} = \lambda\mathbf{v} \tag{1.8}$$

则称 λ 为特征值，$\mathbf{v}$ 为对应的特征向量。

定义 **1.5** (奇异值分解). 对于任意矩阵 $\mathbf{A} \in \mathbb{R}^{m\times n}$，奇异值分解（*SVD*）为：

$$\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top \tag{1.9}$$

其中：

- $\mathbf{U} \in \mathbb{R}^{m\times m}$ 和 $\mathbf{V} \in \mathbb{R}^{n\times n}$ 是正交矩阵
- $\Sigma \in \mathbb{R}^{m\times n}$ 是包含奇异值 $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ 的对角矩阵

奇异值与特征值的关系为 $\sigma_i = \sqrt{\lambda_i(\mathbf{A}^\top\mathbf{A})}$。

SVD 对主成分分析（PCA）至关重要，PCA 用于理解神经网络表示的维度。

1.2.4.1 为什么 SVD 很重要：几何解释

为了理解 SVD 的基础性，考虑矩阵 $\mathbf{A}$ 的几何作用。方程 $\mathbf{y} = \mathbf{A}\mathbf{x}$ 可以通过 SVD 分解为三个顺序变换：

1. $\mathbf{V}^\top$：将输入向量旋转（或反射）到 “右奇异基”。

2. Σ：沿每个轴按相应奇异值 σ_i 缩放。
3. $\mathbf{U}$：旋转到输出空间的“左奇异基”。

这个几何视图揭示了奇异值 $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$ 精确描述了 $\mathbf{A}$ 沿每个主方向拉伸空间的程度。在神经网络中，一层的权重矩阵 $\mathbf{W}$ 正好对输入激活执行这种变换。

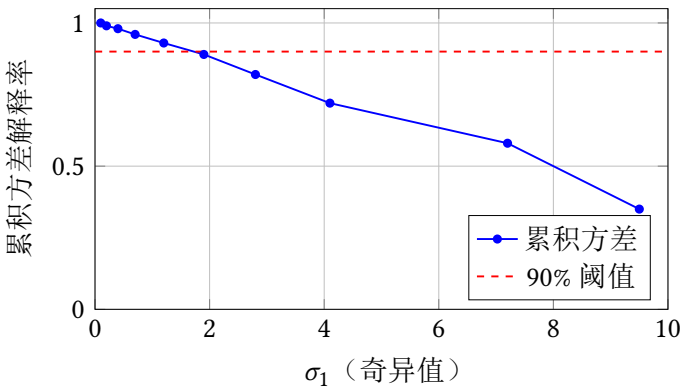


Figure 1.1 – 神经网络权重矩阵的典型奇异值谱。大多数方差由前几个奇异值捕获，这促使人们使用低秩近似。

1.2.4.2 低秩近似：Eckart–Young 定理

SVD 在深度学习中最强大的应用之一是低秩近似。给定秩-r 近似 $\hat{\mathbf{A}}_r = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top$，Eckart–Young 定理保证：

定理 1.1 (Eckart–Young–Mirsky，1936/1949). 对于任意矩阵 $\mathbf{A} \in \mathbb{R}^{m \times n}$，其 *SVD* 为 $\mathbf{A} = \mathbf{U}\Sigma\mathbf{V}^\top$，在谱范数和 *Frobenius* 范数意义下的最佳秩-r 近似由保留前 r 个奇异值给出：

$$\hat{\mathbf{A}}_r = \arg \min_{rank(\mathbf{B}) \leq r} \|\mathbf{A} - \mathbf{B}\|_F = \sum_{i=1}^{r} \sigma_i \mathbf{u}_i \mathbf{v}_i^\top \tag{1.10}$$

近似误差为 $\|\mathbf{A} - \hat{\mathbf{A}}_r\|_F = \sqrt{\sum_{i=r+1}^{\min(m,n)} \sigma_i^2}$。

这个定理支撑了诸如 **LoRA**（低秩适应）等现代技术 [E. J. Hu et al. 2022]，用于高效微调大语言模型，其中权重更新 $\Delta\mathbf{W}$ 被参数化为 $\mathbf{BA}$ 的乘积，秩 $r \ll \min(m,n)$，将可训练参数从 mn 减少到 $(m+n)r$。

1.2.4.3 特征分解与谱定理

对于对称矩阵（作为优化中的 Hessian 矩阵 $\mathbf{H}$ 经常出现），特征分解采用一种特殊且简化的形式：

定理 1.2 (谱定理). 对于任意实对称矩阵 $\mathbf{A} \in \mathbb{R}^{n\times n}$（$\mathbf{A} = \mathbf{A}^\top$），存在正交矩阵 $\mathbf{Q}$ 和对角矩阵 Λ 使得：

$$\mathbf{A} = \mathbf{Q}\Lambda\mathbf{Q}^\top = \sum_{i=1}^{n} \lambda_i \mathbf{q}_i \mathbf{q}_i^\top \tag{1.11}$$

其中 $\lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_n$ 是实特征值，$\mathbf{q}_i$ 是正交特征向量。

损失函数的 Hessian $\mathbf{H} = \nabla^2 \mathscr{L}(\boldsymbol{\theta})$ 始终是对称的（根据混合偏导数相等的 Clairaut 定理）。其特征值决定了损失表面的局部几何：

- 所有 $\lambda_i > 0$：局部最小值（正定）
- 所有 $\lambda_i < 0$：局部最大值（负定）
- 混合符号：鞍点（不定）

1.2.4.4 条件数与数值稳定性

矩阵的条件数衡量 $\mathbf{Ax} = \mathbf{b}$ 的解对 $\mathbf{b}$ 中扰动的敏感程度：

定义 1.6 (条件数). 矩阵 $\mathbf{A}$（相对于 L_2 范数）的条件数为：

$$\kappa(\mathbf{A}) = \|\mathbf{A}\|_2 \cdot \|\mathbf{A}^{-1}\|_2 = \frac{\sigma_{\max}(\mathbf{A})}{\sigma_{\min}(\mathbf{A})} \tag{1.12}$$

对于对称正定矩阵，$\kappa(\mathbf{A}) = \lambda_{\max}/\lambda_{\min}$。

在梯度下降中，Hessian $\kappa(\mathbf{H})$ 的条件数直接限制收敛速度。对于二次目标 $\mathscr{L}(\boldsymbol{\theta}) = \frac{1}{2}\boldsymbol{\theta}^\top \mathbf{H}\boldsymbol{\theta} - \mathbf{b}^\top\boldsymbol{\theta}$，学习率为 α 的梯度下降当且仅当以下条件成立时收敛：

$$0 < \alpha < \frac{2}{\lambda_{\max}} \tag{1.13}$$

最优学习率为 $\alpha^* = \frac{2}{\lambda_{\max}+\lambda_{\min}}$，给出每轮收敛速度：

$$\left(\frac{\kappa(\mathbf{H}) - 1}{\kappa(\mathbf{H}) + 1}\right)^2 \tag{1.14}$$

当 $\kappa(\mathbf{H}) \gg 1$（病态）时，收敛变得极慢——这是像 Adam（第 7 章）这样的自适应优化器的根本动机，它有效地沿每个维度重新缩放梯度。

1.2.4.5 正定矩阵与 Cholesky 分解

定义 1.7 (正定). 如果对称矩阵 $\mathbf{A}$ 对所有非零 $\mathbf{x}$ 满足 $\mathbf{x}^\top\mathbf{Ax} > 0$，则它是正定的。

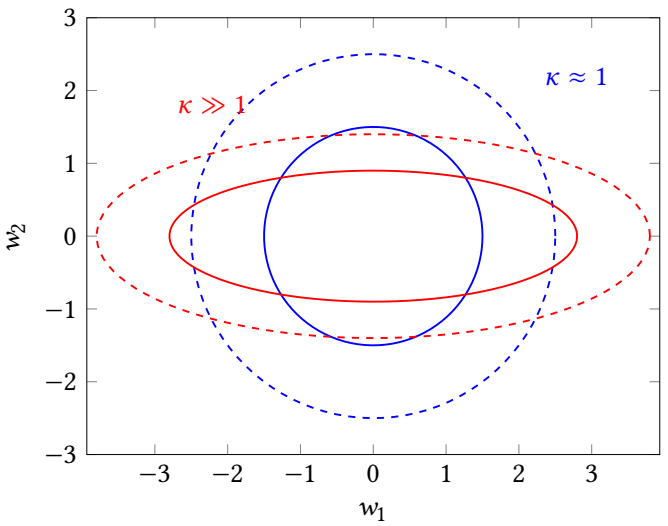


Figure 1.2 – 良态（圆形，$\kappa \approx 1$）和病态（椭圆形，$\kappa \gg 1$）问题的损失等高线。梯度下降在病态地形上呈锯齿状。

对于协方差矩阵（始终是半正定的），Cholesky 分解提供了高效的因式分解：

定理 **1.3** (Cholesky 分解). 对于任意对称正定 $\mathbf{A} \in \mathbb{R}^{n \times n}$：

$$\mathbf{A} = \mathbf{L}\mathbf{L}^\top \tag{1.15}$$

其中 $\mathbf{L}$ 是对角元素为正的下三角矩阵。计算需要 $\frac{n^3}{6}$ *FLOPs*，是 *LU* 分解的一半。

Cholesky 分解对于以下方面至关重要：

- 从多元高斯分布采样：如果 $\mathbf{x} \sim \mathcal{N}(\boldsymbol{\mu}, \boldsymbol{\Sigma})$，则 $\mathbf{x} = \boldsymbol{\mu} + \mathbf{L}\mathbf{z}$，其中 $\mathbf{z} \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$ 且 $\boldsymbol{\Sigma} = \mathbf{L}\mathbf{L}^\top$。
- 卡尔曼滤波器和概率深度学习中的变分推断。
- 优化子程序中求解线性系统 $\mathbf{A}\mathbf{x} = \mathbf{b}$。

1.2.4.6 行列式、迹与 Jacobian 行列式

两个在深度学习中经常出现的矩阵标量总结：

关键矩阵性质

对于具有特征值 $\lambda_1, \ldots, \lambda_n$ 的 $n \times n$ 矩阵 $\mathbf{A}$：

$$\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i = \text{符号体积缩放因子} \tag{1.16}$$

$$\operatorname{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i = \sum_{i=1}^{n} a_{ii} \tag{1.17}$$

$$\det(\mathbf{A}) = \exp(\operatorname{tr}(\log \mathbf{A})) \quad \text{（矩阵对数恒等式）} \tag{1.18}$$

Jacobian 行列式在归一化流和变分自编码器（VAE）中至关重要。如果 $\mathbf{y} = g(\mathbf{x})$ 是可逆变换，则根据变量变换公式：

$$p_{\mathbf{Y}}(\mathbf{y}) = p_{\mathbf{X}}(g^{-1}(\mathbf{y})) \cdot \left| \det \frac{\partial g^{-1}(\mathbf{y})}{\partial \mathbf{y}} \right| = p_{\mathbf{X}}(\mathbf{x}) \cdot \left| \det \frac{\partial g(\mathbf{x})}{\partial \mathbf{x}} \right|^{-1} \tag{1.19}$$

归一化流设计 g 使得行列式计算成本低廉，从而实现精确的似然评估。本书后面将讨论用于生成建模的变分自编码器和归一化流。

1.2.5 张量

神经网络，特别是卷积神经网络，在称为张量的高维数组上运作。

定义 1.8 (张量). 张量是向量和矩阵到更高维度的推广：

- *0D* 张量：标量
- *1D* 张量：向量
- *2D* 张量：矩阵
- *3D* 张量：例如形状为 (H, W, C) 的 *RGB* 图像
- *4D* 张量：形状为 (N, H, W, C) 的图像批次
- *5D* 张量：形状为 (D, H, W, C) 的体积

示例 1.3 (CNN 中的张量形状). 对于 N 张大小为 $H \times W$、*3* 通道（*RGB*）的彩色图像批次：

- 输入张量：$\mathbf{X} \in \mathbb{R}^{N \times H \times W \times 3}$
- 与 C 个滤波器卷积后：$\mathbf{Y} \in \mathbb{R}^{N \times H' \times W' \times C}$

PyTorch 约定：(N, C, H, W) *TensorFlow* 约定：(N, H, W, C)

1.3 微积分

微积分，特别是微分，对于理解神经网络通过基于梯度的优化进行学习至关重要。

1.3.1 导数与梯度

定义 1.9 (导数). 函数 $f : \mathbb{R} \to \mathbb{R}$ 在点 x 处的导数定义为：

$$f'(x) = \frac{df}{dx} = \lim_{h \to 0} \frac{f(x+h) - f(x)}{h} \tag{1.20}$$

定义 1.10 (梯度). 对于标量函数 $f : \mathbb{R}^n \to \mathbb{R}$，梯度是偏导数的向量：

$$\nabla f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f}{\partial x_1} \\ \frac{\partial f}{\partial x_2} \\ \vdots \\ \frac{\partial f}{\partial x_n} \end{bmatrix} \tag{1.21}$$

梯度指向最陡上升方向。

微分规则

- 和规则：$\frac{d}{dx}(f+g) = f' + g'$
- 乘积规则：$\frac{d}{dx}(fg) = f'g + fg'$
- 链式规则：$\frac{d}{dx}f(g(x)) = f'(g(x)) \cdot g'(x)$
- 商规则：$\frac{d}{dx}\left(\frac{f}{g}\right) = \frac{f'g - fg'}{g^2}$

示例 1.4 (神经网络中的链式规则). 对于双层网络 $f(\mathbf{x}) = \mathbf{W}_2(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2$，梯度计算广泛使用链式规则。如果我们想要损失 $\mathscr{L}$ 关于 $\mathbf{W}_1$ 的梯度 $\frac{\partial \mathscr{L}}{\partial \mathbf{W}_1}$，有：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{W}_1} = \frac{\partial \mathscr{L}}{\partial \mathbf{z}_2} \cdot \frac{\partial \mathbf{z}_2}{\partial \mathbf{z}_1} \cdot \frac{\partial \mathbf{z}_1}{\partial \mathbf{W}_1} \tag{1.22}$$

其中 $\mathbf{z}_1 = \mathbf{W}_1\mathbf{x} + \mathbf{b}_1$ 且 $\mathbf{z}_2 = \mathbf{W}_2\mathbf{z}_1 + \mathbf{b}_2$。

1.3.2 高阶导数

定义 **1.11** (Hessian 矩阵). 对于函数 $f : \mathbb{R}^n \to \mathbb{R}$，*Hessian* 矩阵包含所有二阶偏导数：

$$\mathbf{H} = \nabla^2 f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \tag{1.23}$$

如果所有二阶导数连续，则 *Hessian* 是对称的。

Hessian 用于牛顿法和理解损失景观的曲率。

1.3.2.1 泰勒展开：理论桥梁

泰勒展开是连接一阶和二阶优化方法的单一最重要的数学工具。对于围绕点 $\mathbf{a}$ 的二次可微函数 $f : \mathbb{R}^n \to \mathbb{R}$：

定义 **1.12** (多元泰勒展开).

$$f(\mathbf{a} + \boldsymbol{\delta}) = f(\mathbf{a}) + \nabla f(\mathbf{a})^\top \boldsymbol{\delta} + \frac{1}{2} \boldsymbol{\delta}^\top \mathbf{H}(\mathbf{a}) \boldsymbol{\delta} + \mathcal{O}(\|\boldsymbol{\delta}\|^3) \tag{1.24}$$

其中 $\nabla f(\mathbf{a})$ 是梯度，$\mathbf{H}(\mathbf{a}) = \nabla^2 f(\mathbf{a})$ 是在 $\mathbf{a}$ 处求值的 *Hessian*。

这对神经网络的重要性：几乎所有优化理论都可以通过局部泰勒近似的视角来理解。

示例 **1.5** (梯度下降作为一阶泰勒近似). 将 $\boldsymbol{\delta} = -\alpha \nabla f(\mathbf{a})$ 代入一阶展开：

$$f(\mathbf{a} - \alpha \nabla f(\mathbf{a})) \approx f(\mathbf{a}) - \alpha \|\nabla f(\mathbf{a})\|^2 \tag{1.25}$$

只要 $\alpha > 0$ 且线性近似有效——即 α 不是太大——这保证了下降。

示例 **1.6** (牛顿法作为二阶近似). 最小化二阶泰勒近似：

$$f(\mathbf{a} + \boldsymbol{\delta}) \approx f(\mathbf{a}) + \nabla f(\mathbf{a})^\top \boldsymbol{\delta} + \frac{1}{2} \boldsymbol{\delta}^\top \mathbf{H} \boldsymbol{\delta} \tag{1.26}$$

令关于 $\boldsymbol{\delta}$ 的梯度为零：

$$\nabla f(\mathbf{a}) + \mathbf{H} \boldsymbol{\delta}^* = 0 \implies \boldsymbol{\delta}^* = -\mathbf{H}^{-1} \nabla f(\mathbf{a}) \tag{1.27}$$

这就是牛顿更新。它在最小值附近二次收敛，但每步需要 $\mathcal{O}(n^3)$ 进行矩阵求逆——对于具有数百万参数深度网络来说成本过高。现代拟牛顿法（*L-BFGS*）和近似二阶法（*K-FAC*、*Shampoo*）试图以可管理的成本捕获曲率信息。

1.3.2.2 凸性与损失景观

凸性决定梯度下降是否能找到全局最小值：

定义 1.13 (凸函数)**.** 如果函数 f 对所有 $\mathbf{x}, \mathbf{y}$ 和 $\lambda \in [0, 1]$ 满足：

$$f(\lambda\mathbf{x} + (1-\lambda)\mathbf{y}) \leq \lambda f(\mathbf{x}) + (1-\lambda) f(\mathbf{y}) \tag{1.28}$$

则 f 是凸函数。

定义 1.14 (强凸函数)**.** 如果二次可微的 f 是 μ 强凸的：

$$\nabla^2 f(\mathbf{x}) \succeq \mu\mathbf{I} \quad （\textit{Hessian} \text{ 的所有特征值} \geq \mu > 0） \tag{1.29}$$

坏消息：神经网络损失景观是非凸的——Hessian 到处都是既有正特征值又有负特征值。然而，一个引人注目的经验发现是在过参数化网络中，大多数局部最小值具有相似的损失值 [Choromanska et al. 2015]，主要挑战是鞍点而不是差的局部最小值。

Table 1.1 – 凸性性质及其对优化的影响

性质	条件	含义
凸	$\nabla^2 f \succeq 0$	所有局部最小值都是全局的
严格凸	$\nabla^2 f \succ 0$	唯一全局最小值
μ 强凸	$\nabla^2 f \succeq \mu\mathbf{I}$	线性收敛速度
L 平滑	$\lVert\nabla^2 f\rVert \leq L$	$\alpha < 2/L$ 保证下降
PL 条件	–	即使非凸也能线性收敛

Polyak–Łojasiewicz（PL）条件是强凸性的放松，适用于一些非凸问题：

$$\frac{1}{2}\lVert\nabla f(\mathbf{x})\rVert^2 \geq \mu(f(\mathbf{x}) - f^*) \tag{1.30}$$

其中 f^* 是最优值。许多神经网络训练目标被推测近似满足此条件，这解释了尽管非凸性梯度下降为何经常找到好的解。

1.3.2.3 Lipschitz 连续与梯度裁剪

定义 1.15 (L-Lipschitz 连续)**.** 如果函数 f 是 *L-Lipschitz* 连续的：

$$|f(\mathbf{x}) - f(\mathbf{y})| \leq L\lVert\mathbf{x} - \mathbf{y}\rVert \quad \forall \mathbf{x}, \mathbf{y} \tag{1.31}$$

对于可微 f，等价于 $\|\nabla f(\mathbf{x})\| \leq L$。

在神经网络中，每层的 Lipschitz 常数限制输入的微小变化能多大程度上影响输出。对于线性层 $\mathbf{y} = \mathbf{W}\mathbf{x}$，Lipschitz 常数等于 $\|\mathbf{W}\|_2 = \sigma_{\max}(\mathbf{W})$。

RNN 和 **transformer** 中的梯度裁剪限制梯度范数：

$$\mathbf{g} \leftarrow \begin{cases} \mathbf{g} & \text{如果} \|\mathbf{g}\| \leq \tau \\ \frac{\tau}{\|\mathbf{g}\|}\mathbf{g} & \text{否则} \end{cases} \tag{1.32}$$

这防止梯度爆炸，这对循环架构和大语言模型尤其成问题。

示例 **1.7** (梯度下降 vs. 牛顿法).

- 梯度下降：$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \nabla \mathscr{L}(\mathbf{w}_t)$
- 牛顿法：$\mathbf{w}_{t+1} = \mathbf{w}_t - \alpha \mathbf{H}^{-1} \nabla \mathscr{L}(\mathbf{w}_t)$

牛顿法利用二阶信息（曲率）实现潜在更快的收敛，但对于大型网络计算 *Hessian* 成本高昂。

1.3.3 Jacobian 矩阵

定义 **1.16** (Jacobian). 对于向量值函数 $\mathbf{f} : \mathbb{R}^n \to \mathbb{R}^m$，*Jacobian* 是偏导数的 $m \times n$ 矩阵：

$$\mathbf{J} = \frac{\partial \mathbf{f}}{\partial \mathbf{x}} = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \frac{\partial f_1}{\partial x_2} & \cdots \\ \frac{\partial f_2}{\partial x_1} & \frac{\partial f_2}{\partial x_2} & \cdots \\ \vdots & \vdots & \ddots \end{bmatrix} \tag{1.33}$$

每行 i 是 f_i 的梯度。

Jacobian 将梯度推广到向量值函数，这对通过复杂计算图的反向传播至关重要。

示例 **1.8** (反向传播中的 Jacobian). 在反向传播中，我们计算标量输出（如损失）相对于向量输入的梯度。如果 $\mathbf{y} = f(\mathbf{x})$ 且 L 是标量损失，则：

$$\frac{\partial L}{\partial \mathbf{x}} = \left(\frac{\partial \mathbf{y}}{\partial \mathbf{x}}\right)^\top \frac{\partial L}{\partial \mathbf{y}} = \mathbf{J}^\top \nabla_{\mathbf{y}} L \tag{1.34}$$

反向传播有效地计算这个乘积而不显式形成完整 *Jacobian*。

1.4 概率论

概率论提供了理解不确定性、进行预测和设计损失函数的框架。

1.4.1 基本概率

定义 **1.17** (概率). 对于样本空间为 Ω 的随机实验，概率函数 P 为每个事件 $A \subseteq \Omega$ 分配一个数 $P(A)$，使得：

- 对所有事件 A，$0 \leq P(A) \leq 1$
- $P(\Omega) = 1$
- 如果 $A_1, A_2, \ldots$ 互斥，则 $P(\bigcup_i A_i) = \sum_i P(A_i)$

定义 **1.18** (随机变量). 随机变量 X 是将随机实验的结果映射到数值的函数：

- 离散型：可数值的 $P(X = x)$
- 连续型：由概率密度函数 $p(x)$ 描述

- 期望值： $E[X] = \sum_x x \cdot P(X = x)$ 或 $\int x \cdot p(x)dx$
- 方差： $\mathrm{Var}(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2$
- 标准差： $\sigma_X = \sqrt{\mathrm{Var}(X)}$

1.4.2 常见分布

Table 1.2 – 深度学习中的常见概率分布

分布	公式	应用
伯努利分布	$P(X = 1) = p, P(X = 0) = 1 - p$	二分类
类别分布	$P(X = k) = p_k$	多分类
高斯（正态）分布	$p(x) = \frac{1}{\sqrt{2\pi\sigma^2}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$	回归、权重初始化
多项分布	伯努利分布的推广	语言建模

示例 **1.9** (Softmax 分布). *Softmax* 函数将 *logits* 向量 $\mathbf{z} \in \mathbb{R}^K$ 转换为概率分布：

$$softmax(z_i) = \frac{e^{z_i}}{\sum_{j=1}^{K} e^{z_j}} \tag{1.35}$$

性质：

- $\sum_{i=1}^{K} softmax(z_i) = 1$

- 对所有 i，$softmax(z_i) \in (0,1)$
- $\arg\max_i softmax(z_i) = \arg\max_i z_i$

用于多分类输出。

1.4.3 贝叶斯概率

定义 **1.19** (贝叶斯规则).

$$P(A|B) = \frac{P(B|A) \cdot P(A)}{P(B)} \tag{1.36}$$

在机器学习背景下：

$$P(\text{假设}|\text{数据}) = \frac{P(\text{数据}|\text{假设}) \cdot P(\text{假设})}{P(\text{数据})} \tag{1.37}$$

- $P(\text{假设})$：先验概率
- $P(\text{数据}|\text{假设})$：似然
- $P(\text{假设}|\text{数据})$：后验概率

贝叶斯方法用于贝叶斯神经网络的的不确定性估计和贝叶斯优化的超参数调优。

1.4.4 最大似然估计

最大似然估计（MLE）是深度学习中大多数损失函数背后的统计原理。

定义 **1.20** (最大似然估计). 给定从未知分布独立同分布抽取的数据集 $\mathcal{D} = \{(\mathbf{x}_i, y_i)\}_{i=1}^N$，*MLE* 找到使观察数据的概率最大化的参数 θ：

$$\theta_{MLE} = \arg\max_\theta \prod_{i=1}^N p(y_i \mid \mathbf{x}_i; \theta) \tag{1.38}$$

等价地（为数值稳定性取负对数）：

$$\theta_{MLE} = \arg\min_\theta \underbrace{-\sum_{i=1}^N \log p(y_i \mid \mathbf{x}_i; \theta)}_{\text{负对数似然（}NLL\text{）}} \tag{1.39}$$

负对数似然（NLL）就是损失函数。不同的模型假设导致不同的损失函数：

示例 **1.10** (高斯噪声下的 MSE 作为 MLE)**.** 假设 $y = f(\mathbf{x};\theta) + \epsilon$，其中 $\epsilon \sim \mathcal{N}(0,\sigma^2)$。则：

$$p(y \mid \mathbf{x};\theta) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y - f(\mathbf{x};\theta))^2}{2\sigma^2}\right) \tag{1.40}$$

$$-\log p(y \mid \mathbf{x};\theta) = \frac{(y-\hat{y})^2}{2\sigma^2} + \frac{1}{2}\log(2\pi\sigma^2) \tag{1.41}$$

由于常数 $\frac{1}{2}\log(2\pi\sigma^2)$ 不依赖于 θ，最小化 *NLL* 等价于最小化 *MSE*。这解释了为什么 *MSE* 是回归的 "自然"损失——它对应于加性高斯噪声的假设。

MSE 损失的梯度：

$$\frac{\partial \mathcal{L}_{MSE}}{\partial \hat{y}} = \frac{2}{N}(\hat{y} - y) \tag{1.42}$$

与残差成正比，意味着大误差获得比例上大的梯度——这既是优点（在最优值附近快速收敛）也是缺点（对异常值敏感）。

示例 **1.11** (分类分布的交叉熵作为 MLE)**.** 对于 K 类分类，将输出建模为类别分布：

$$p(y = k \mid \mathbf{x};\theta) = \hat{y}_k = softmax(\mathbf{z})_k \tag{1.43}$$

负对数似然变为：

$$\mathcal{L}_{NLL} = -\sum_{k=1}^{K} y_k \log \hat{y}_k = -\log \hat{y}_c \tag{1.44}$$

其中 c 是真实类别。这正是交叉熵损失。梯度简化为优美的形式：

$$\frac{\partial \mathcal{L}}{\partial z_k} = \hat{y}_k - y_k \tag{1.45}$$

梯度就是预测与目标之间的差异——"误差信号"。这种简洁形式是 *softmax* + 交叉熵成为分类标准的原因之一。

1.4.5 贝叶斯推断与 MAP 估计

MLE 找到单个最佳参数，而贝叶斯推断将参数 θ 本身视为具有先验分布 $p(\theta)$ 的随机变量。

定义 **1.21** (最大后验（MAP）估计)**.**

$$\theta_{MAP} = \arg\max_{\theta}\ p(\theta \mid \mathcal{D}) = \arg\max_{\theta}\ \underbrace{\log p(\mathcal{D} \mid \theta)}_{\text{对数似然}} + \underbrace{\log p(\theta)}_{\text{对数先验}} \tag{1.46}$$

示例 **1.12** (高斯先验下的权重衰减作为 MAP). 如果我们对每个权重放置高斯先验 $\theta_j \sim \mathcal{N}(0, \lambda^{-1})$，则：

$$\log p(\theta) = \sum_j \log \frac{1}{\sqrt{2\pi/\lambda}} \exp\left(-\frac{\lambda\theta_j^2}{2}\right) = -\frac{\lambda}{2}\|\theta\|_2^2 + const \tag{1.47}$$

因此，带高斯先验的 *MAP* 估计等价于带 L_2 正则化（权重衰减）的 *MLE*：

$$\theta_{MAP} = \arg\min_\theta \ \mathscr{L}(\theta) + \frac{\lambda}{2}\|\theta\|_2^2 \tag{1.48}$$

类似地，拉普拉斯先验 $p(\theta_j) \propto \exp(-\lambda|\theta_j|)$ 给出 L_1 正则化，促进稀疏性。

1.4.6 共轭先验与指数族

深度学习中的许多常见分布属于指数族：

$$p(\mathbf{x} \mid \boldsymbol{\eta}) = h(\mathbf{x}) \exp\left(\boldsymbol{\eta}^\top T(\mathbf{x}) - A(\boldsymbol{\eta})\right) \tag{1.49}$$

其中 $\boldsymbol{\eta}$ 是自然参数，$T(\mathbf{x})$ 是充分统计量，$A(\boldsymbol{\eta})$ 是对数配分函数。

共轭先验

如果先验 $p(\boldsymbol{\eta})$ 与似然 $p(\mathbf{x} \mid \boldsymbol{\eta})$ 共轭，则后验 $p(\boldsymbol{\eta} \mid \mathbf{x})$ 与先验属于同一分布族。关键例子：

Table 1.3 – 常见共轭先验关系

似然	先验	后验	应用
伯努利	Beta	Beta	点击率
多项	Dirichlet	Dirichlet	主题模型（LDA）
高斯（已知 σ^2）	高斯	高斯	贝叶斯回归
高斯（已知 μ）	逆伽马	逆伽马	方差估计
泊松	Gamma	Gamma	计数数据

1.5 优化

神经网络训练本质上是一个优化问题：找到使损失函数最小化的参数。

1.5.1 优化问题

神经网络训练

给定：

- 带参数 θ 的神经网络 $f(\mathbf{x};\theta)$
- 数据集 $\mathscr{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$
- 损失函数 $\mathscr{L}(\theta)$

找到：

$$\theta^* = \arg\min_\theta \mathscr{L}(\theta) \tag{1.50}$$

其中典型情况下：

$$\mathscr{L}(\theta) = \frac{1}{N}\sum_{i=1}^N \ell(f(\mathbf{x}_i;\theta), \mathbf{y}_i) \tag{1.51}$$

1.5.2 梯度下降

Algorithm 1 批量梯度下降

1. 初始化参数 $\theta^{(0)}$
2. 对于 $t = 0, 1, 2, \ldots$：
   (a) 计算梯度：$\mathbf{g}_t = \nabla\mathscr{L}(\theta^{(t)})$
   (b) 更新参数：$\theta^{(t+1)} = \theta^{(t)} - \alpha\mathbf{g}_t$
3. 当满足收敛条件时停止

学习率 $\alpha > 0$ 控制步长。选择合适的学习率对成功训练至关重要。

1.5.3 优化中的挑战

- 局部最小值：损失景观是非凸的，包含许多局部最小值。然而，对于大型神经网络，局部最小值通常不是大问题；鞍点更常见。
- 梯度消失/爆炸：在深度网络中，梯度可能变得非常小（消失）或非常大（爆炸），使训练困难。
- 平台期：梯度接近零的平坦区域，导致进展缓慢。

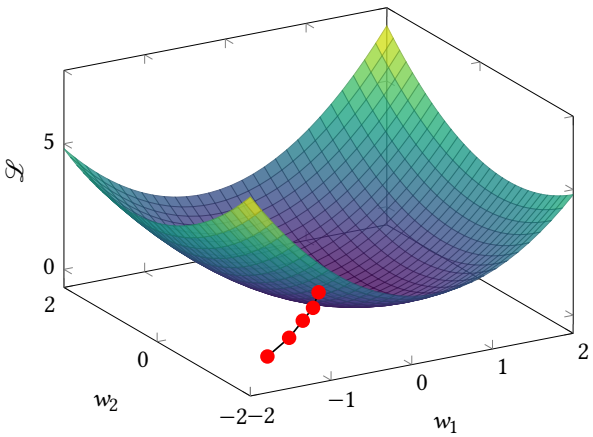


Figure 1.3 – 二次损失表面上的梯度下降

- 损失景观：神经网络损失景观复杂，包含许多局部最小值、鞍点和平坦区域。

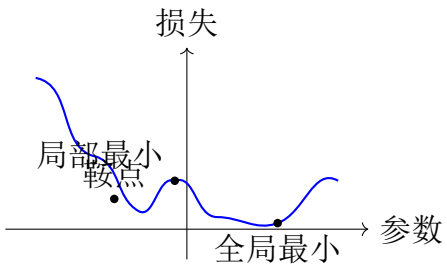


Figure 1.4 – 包含局部和全局最小值的损失景观示意图

1.6 信息论

信息论为理解损失函数（特别是交叉熵）提供了框架。

1.6.1 熵

定义 **1.22** (香农熵). 对于具有概率分布 $p(x)$ 的离散随机变量 X：

$$H(X) = -\sum_x p(x)\log p(x) = E[-\log p(X)] \tag{1.52}$$

熵衡量分布的 “不确定性” 或 “信息量”。对数底数通常是 2（比特）或 e（奈特，自然单位）。

历史背景：克劳德 • 香农在他 1948 年的论文《通信的数学理论》[Shannon 1948] 中引入了熵，创立了信息论。关键洞察是熵提供了满足自然 desiderata（连续性、对等概率结果的单调性以及独立事件的加性）的不确定性的唯一（常数倍）度量。

1.6.1.1 证明：通过 Jensen 不等式证明 $D_{KL}(p\|q) \geq 0$

KL 散度的非负性是信息论中最重要的结果之一，直接由 Jensen 不等式得出。

定理 1.4 (Jensen 不等式). 对于凸函数 ϕ 和随机变量 X：

$$\phi(E[X]) \leq E[\phi(X)] \tag{1.53}$$

对于凹函数，不等式反向。相等当且仅当 X 是确定性的（几乎处处常数）或 ϕ 在 X 的支撑上是仿射的。

KL 散度的非负性

由于 $-\log$ 是严格凸函数，根据 Jensen 不等式：

$$\begin{aligned}
-D_{KL}(p\|q) &= \sum_x p(x) \log \frac{q(x)}{p(x)} && (1.54)\\
&= E_p\left[\log \frac{q}{p}\right] && (1.55)\\
&\leq \log E_p\left[\frac{q}{p}\right] \quad \text{（Jensen 不等式，log 是凹的）} && (1.56)\\
&= \log\left(\sum_x p(x)\frac{q(x)}{p(x)}\right) && (1.57)\\
&= \log\left(\sum_x q(x)\right) = \log 1 = 0 && (1.58)
\end{aligned}$$

因此 $D_{KL}(p\|q) \geq 0$，当且仅当 $p = q$ 几乎处处相等时等号成立。

这个证明优雅但有力：它告诉我们交叉熵 $H(p,q)$ 始终至少与真实熵 $H(p)$ 一样大，差距正好是 $D_{KL}(p\|q)$。

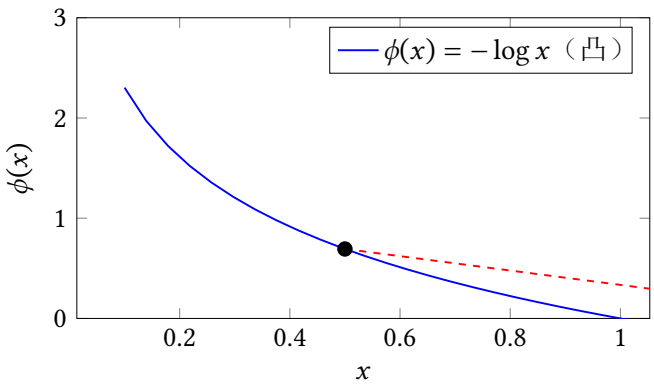


Figure 1.5 – 凸函数 $\phi(x) = -\log x$ 的 Jensen 不等式：在均值 $E[X]$ 处求值的函数位于期望值 $E[\phi(X)]$ 之下。

1.6.1.2 互信息

定义 **1.23** (互信息). 两个随机变量 X 和 Y 之间的互信息为：

$$I(X;Y) = D_{KL}(p(x,y)\|p(x)p(y)) = \sum_{x,y} p(x,y)\log\frac{p(x,y)}{p(x)p(y)} \tag{1.59}$$

互信息满足：

$$I(X;Y) = H(X) - H(X \mid Y) = H(Y) - H(Y \mid X) \tag{1.60}$$

$$I(X;Y) = H(X) + H(Y) - H(X,Y) \tag{1.61}$$

$$I(X;Y) \geq 0 \quad \text{当且仅当} X \perp Y \text{ 时等号成立} \tag{1.62}$$

深度学习中的应用：

- **InfoNCE** 损失（第 3 章）：对比学习最大化同一图像增强视图之间的互信息。
- 信息瓶颈理论 [Tishby and Zaslavsky 2015]：深度学习被视为将输入 **X** 逐步压缩为保留关于目标 **Y** 最大信息的表示 **T**：

$$\min_{p(\mathbf{t}|\mathbf{x})} I(\mathbf{X};\mathbf{T}) - \beta I(\mathbf{T};\mathbf{Y}) \tag{1.63}$$

- **VAE** [Kingma and Welling 2014]：证据下界（ELBO）可以写为：

$$\log p(\mathbf{x}) \geq \mathbb{E}_{q(\mathbf{z}|\mathbf{x})}[\log p(\mathbf{x}|\mathbf{z})] - D_{KL}(q(\mathbf{z}|\mathbf{x})\|p(\mathbf{z})) \tag{1.64}$$

熵的性质

- 对所有分布 $H(X) \geq 0$
- 当且仅当对某个 x 有 $p(x) = 1$ 时 $H(X) = 0$（确定性）
- $H(X)$ 在均匀分布时最大化
- 二元熵：$H_b(p) = -p \log p - (1-p)\log(1-p)$

1.6.2 交叉熵与 KL 散度

定义 **1.24** (交叉熵). 对于两个概率分布 p（真实）和 q（预测）：

$$H(p,q) = -\sum_x p(x) \log q(x) = H(p) + D_{KL}(p\|q) \tag{1.65}$$

定义 **1.25** (KL 散度).

$$D_{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \tag{1.66}$$

KL 散度衡量从 q 到 p 的 "距离"（不是真正的度量，因为它是非对称的）。

KL 散度性质

- $D_{KL}(p\|q) \geq 0$（非负性）
- 当且仅当 $p = q$ 时 $D_{KL}(p\|q) = 0$
- $D_{KL}(p\|q) \neq D_{KL}(q\|p)$（非对称）

示例 **1.13** (交叉熵损失). 对于 N 个样本和 K 个类的分类：

$$\mathscr{L}_{CE} = -\frac{1}{N} \sum_{i=1}^{N} \sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik} \tag{1.67}$$

其中 y_{ik} 是真实标签（独热），$\hat{y}_{ik} = softmax(z_{ik})$ 是预测。

最小化交叉熵等价于最小化真实分布与预测分布之间的 *KL* 散度，因为对于独热标签 $H(p)$ 是常数。

推导 ***softmax***–交叉熵梯度：这是深度学习中最重要的推导之一。对于真实

类为 c 的单个样本（独热：$y_c = 1$，$y_{k\neq c} = 0$）：

$$\mathscr{L} = -\sum_{k=1}^{K} y_k \log \hat{y}_k = -\log \hat{y}_c = -\log \frac{e^{z_c}}{\sum_j e^{z_j}} = -z_c + \log \sum_j e^{z_j} \tag{1.68}$$

$$\frac{\partial \mathscr{L}}{\partial z_k} = -y_k + \frac{e^{z_k}}{\sum_j e^{z_j}} = \hat{y}_k - y_k \tag{1.69}$$

这个非常简洁的梯度——预测减目标——是分类训练的基石。*Softmax* 与交叉熵对数 “抵消”，避免了分别计算它们时出现的数值问题。

1.7 本章小结

本章涵盖了神经网络的基本数学基础：

1. 线性代数：向量、矩阵、张量运算和范数是在神经网络中表示和操作数据和参数的构建块。
2. 微积分：导数、梯度和链式规则是计算反向传播梯度的基本工具。
3. 概率论：分布、期望值和贝叶斯规则为理解预测和设计损失函数提供了框架。
4. 优化：梯度下降及其变体是神经网络训练的主力。
5. 信息论：熵和 KL 散度为使用交叉熵损失提供了正当理由。

这些数学工具将在整本教材中应用，因为我们探索激活函数、损失函数、网络架构和训练算法。

1.8 扩展阅读

更深入地覆盖这些主题：

- [Bishop 2006] - 线性代数和概率的全面处理
- [Strang 2009] - 线性代数的优秀介绍
- [Cover and Thomas 2006] - 信息论的经典文本
- [Kingma and Welling 2014] - 变分推断与 ELBO
- [Tishby and Zaslavsky 2015] - 深度学习的信息瓶颈理论
- [Choromanska et al. 2015] - 神经网络损失表面分析
- [I. Goodfellow, Bengio, and Courville 2016] - 第 2 章涵盖深度学习的数学基础
- [E. J. Hu et al. 2022] - 现代模型中的低秩适应和低秩结构

1.9 练习

1. 线性代数：给定权重矩阵 $\mathbf{W} \in \mathbb{R}^{256\times512}$，如果我们包含偏置项，这一层有多少参数？写出单次前向传递的计算成本（FLOPs）。
2. 微积分：计算单层网络的梯度 $\frac{\partial \mathscr{L}}{\partial \mathbf{W}}$，输出为 $\mathbf{y} = \mathbf{W}\mathbf{x}$，其中 $\mathscr{L} = \|\mathbf{y} - \mathbf{t}\|^2$。
3. 概率：证明 softmax 函数对加性常数不变：$\text{softmax}(\mathbf{z} + c) = \text{softmax}(\mathbf{z})$ 对任意常数 c。
4. 优化：为简单二次函数 $f(x) = x^2$ 实现梯度下降，并分析不同学习率的影响。
5. 信息论：使用 Jensen 不等式证明 $D_{KL}(p\|q) \geq 0$。

Chapter 2

激活函数

2.1 引言

激活函数是应用于神经网络中每个神经元（或每层）输出的数学函数。它们向网络引入非线性，使其能够学习线性函数无法捕获的复杂模式。如果没有激活函数，无论网络有多少层，都只是执行线性变换，等同于一个线性层。

激活函数的选择会显著影响：

- 网络学习复杂模式的能力
- 训练稳定性和收敛速度
- 反向传播过程中的梯度流动
- 神经元的输出范围
- 计算效率

本章将探讨各种激活函数的数学性质、优缺点和实际应用。

2.2 为什么需要非线性？——通用逼近定理

在逐一介绍激活函数之前，值得提出一个基本问题：为什么我们需要非线性激活函数？答案在于神经网络理论中最著名的理论结果之一。

2.2.1 定理

定理 2.1 (通用逼近定理 [Hornik 1991]). 设 σ 是一个非常数的、有界的、连续的激活函数。则对于任意连续函数 $g : [0,1]^n \to \mathbb{R}$ 和任意 $\epsilon > 0$，存在整数 $N > 0$、实常数 $\alpha_i, b_i \in \mathbb{R}$ 和实向量 $\mathbf{w}_i \in \mathbb{R}^n$，使得：

$$F(\mathbf{x}) = \sum_{i=1}^{N} \alpha_i \, \sigma\left(\mathbf{w}_i^\top \mathbf{x} + b_i\right) \tag{2.1}$$

满足 $|F(\mathbf{x}) - g(\mathbf{x})| < \epsilon$ 对所有 $\mathbf{x} \in [0,1]^n$。

含义：具有一个隐藏层和非多项式激活函数（包括 ReLU、sigmoid、tanh）的网络可以以任意精度逼近任何连续函数，只要神经元数量足够。

2.2.2 直观解释

考虑一个我们希望在 1D 区间上逼近的函数。隐藏层中的每个神经元通过 $\sigma(\mathbf{w}^\top \mathbf{x} + b)$ 计算一个" 凸起" 或" 阶梯" 函数，输出层用权重 α_i 将这些组合起来构建目标函数。神经元越多，逼近越精细：

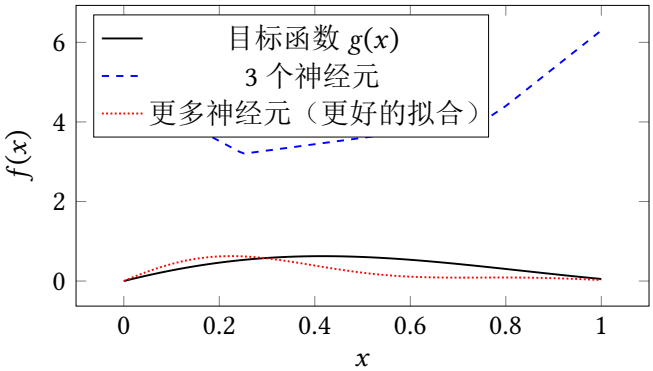


Figure 2.1 – 每个神经元贡献一个" 凸起" 或" 阶梯"。更多神经元对目标函数产生更精细的逼近。

2.2.3 注意事项和实际意义

通用逼近定理是一个存在性结果，而非构造性结果。它告诉我们存在一个合适的网络，但并未说明：

1. 需要多少神经元——对于高维函数，这个界是天文数字。
2. 梯度下降能否找到它——优化景观可能有糟糕的局部最小值。
3. 更深的网络是否更高效——Cybenko 和 Hornik 的结果适用于单隐藏层，但深度为组合函数提供了指数级的效率提升 [Montúfar et al. 2014]。

在实践中，深度网络（多层、适度宽度）远远优于浅层、宽网络。关键洞察是：深度实现了层次化特征组合——每一层构建越来越抽象的表示。

2.3 激活函数的历史发展

了解激活函数的历史可以阐明为什么某些选择成为主流，以及该领域是如何演变的：

ReLU 的崛起故事尤其具有启发性。在 2012 年之前，sigmoid 和 tanh 是标准选择。然而，在深度网络中，这些饱和函数导致了梯度消失——通过 L 个 sigmoid 导数的梯度乘积最多为 $(1/4)^L$，当 $L > 10$ 时变得可以忽略不计。AlexNet [Krizhevsky, Sutskever, and G. E. Hinton 2012] 的突破表明，ReLU 的非

Table 2.1 – 激活函数发展历史时间表

年份	函数	背景与动机	关键参考文献
1943	阈值（阶梯）函数	McCulloch-Pitts 神经元	McCulloch & Pitts
1957	感知机	线性阈值单元	Rosenblatt
1960	Adaline	线性激活 + 符号	Widrow & Hoff
1986	Sigmoid	可微分的反向传播	Rumelhart et al.
1991	Tanh	零中心化替代方案	
1998	ReLU	生物学的启发	Hahnloser et al.
2010	Softmax	概率多分类	
2011	ReLU（复兴）	AlexNet：避免梯度消失	[Krizhevsky, Sutskever, and G. E. Hinton 2012]
2015	ELU, SELU	平滑负区	Clevert et al.
2015	PReLU	可学习的负斜率	He et al.
2016	Swish/SiLU	自门控、非单调	Ramachandran et al.
2016	GELU	概率门控	Hendrycks & Gimpel
2019	Mish	自正则化	Misra
2022	StarReLU	高效 Transformer	Maulik et al.

饱和特性（$f'(x) = 1$ 当 $x > 0$）使得训练更深的网络成为可能，从根本上改变了这个领域。

2.4 神经网络神经元

在深入研究激活函数之前，让我们先了解它们在神经元中的作用。

单个人工神经元计算：

$$y = f(\mathbf{w}^\top \mathbf{x} + b) = f\left(\sum_{i=1}^{n} w_i x_i + b\right) \tag{2.2}$$

其中：

- $\mathbf{x} \in \mathbb{R}^n$ 是输入向量
- $\mathbf{w} \in \mathbb{R}^n$ 是权重向量
- $b \in \mathbb{R}$ 是偏置

- $f(\cdot)$ 是激活函数
- $y \in \mathbb{R}$ 是输出

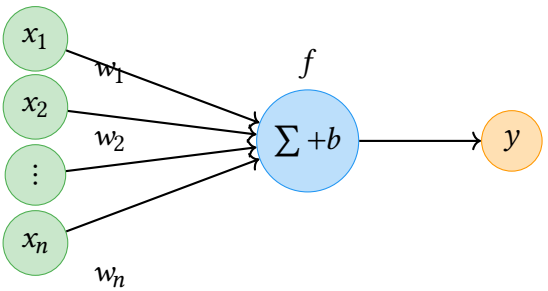


Figure 2.2 – 单个神经元计算加权和，然后应用激活函数

2.5 线性激活函数

虽然实际上不是严格的" 激活函数"，但线性函数有时用于输出层。

2.5.1 恒等函数

$$f(x) = x \tag{2.3}$$

- 输出范围：$(-\infty, \infty)$
- 导数：$f'(x) = 1$
- 用例：回归输出层
- 不推荐：隐藏层（会使网络变为线性）

2.6 Sigmoid 族

Sigmoid 族激活函数将输入压缩到范围 $(0, 1)$，使其适用于二分类输出层。

2.6.1 Sigmoid（逻辑斯蒂）

$$\sigma(x) = \frac{1}{1 + e^{-x}} = \frac{e^x}{e^x + 1} \tag{2.4}$$

Sigmoid 的性质

- 输出范围：$(0, 1)$
- 导数：$\sigma'(x) = \sigma(x)(1 - \sigma(x))$
- 单调递增
- 关于 **0.5** 对称
- 处处光滑且可微分

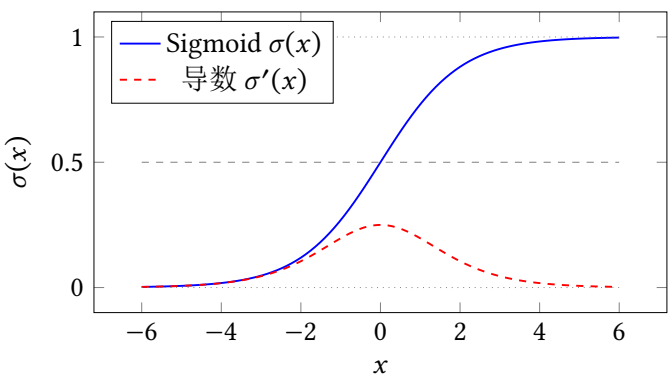


Figure 2.3 – Sigmoid 函数及其导数

优点

- 平滑的梯度
- 输出可以解释为概率
- 历史上广泛使用

缺点

- 梯度消失：对于大的 $|x|$，$\sigma'(x) \approx 0$
- 非零中心化：输出总是正的
- 指数计算：比简单函数慢
- 饱和：在 0 和 1 处饱和

2.6.1.1 数学分析：为什么 Sigmoid 导致梯度消失

考虑一个每层都使用 sigmoid 激活的 L 层网络。在反向传播过程中，流经第 l 层的梯度乘以每个中间层的导数：

$$\frac{\partial \mathscr{L}}{\partial z_1} = \underbrace{\frac{\partial \mathscr{L}}{\partial z_L}}_{\text{输出梯度}} \cdot \prod_{l=2}^{L} \underbrace{\frac{\partial a_l}{\partial z_l}}_{=\sigma'(z_l)} \cdot \underbrace{\frac{\partial z_l}{\partial a_{l-1}}}_{=\mathbf{W}_l} \tag{2.5}$$

由于 $\sigma'(z) = \sigma(z)(1-\sigma(z)) \leq \frac{1}{4}$（在 $z = 0$ 处最大），sigmoid 导数分量单独提供一个最多为 $(1/4)^{L-1}$ 的乘数因子：

$$\left\|\frac{\partial \mathscr{L}}{\partial z_1}\right\| \leq \left(\frac{1}{4}\right)^{L-1} \cdot \prod_{l=2}^{L} \|\mathbf{W}_l\| \cdot \left\|\frac{\partial \mathscr{L}}{\partial z_L}\right\| \tag{2.6}$$

对于 50 层网络：$(1/4)^{49} \approx 10^{-30}$。即使权重范数适中，梯度也有效地消失了。这个数学分析解释了为什么比几层更深的 sigmoid 网络在采用替代激活函数之前基本上是不可训练的。

2.6.1.2 导数推导

从 $\sigma(x) = \frac{1}{1+e^{-x}}$ 开始：

$$\sigma'(x) = \frac{d}{dx}\left((1+e^{-x})^{-1}\right) = (1+e^{-x})^{-2} \cdot e^{-x} \tag{2.7}$$

$$= \frac{1}{1+e^{-x}} \cdot \frac{e^{-x}}{1+e^{-x}} \tag{2.8}$$

$$= \sigma(x) \cdot (1-\sigma(x)) \tag{2.9}$$

这种优雅的形式意味着梯度可以直接从输出计算，无需再次求值指数。最大值 $\sigma'(0) = 0.25$ 出现在 $x = 0$ 处。

示例 **2.1** (二分类中的 Sigmoid). 对于二分类，*sigmoid* 输出可以解释为：

$$P(y = 1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) \tag{2.10}$$

带 *sigmoid* 的交叉熵损失为：

$$\mathscr{L} = -[y \log \hat{y} + (1-y)\log(1-\hat{y})] \tag{2.11}$$

其中 $\hat{y} = \sigma(\mathbf{w}^\top \mathbf{x} + b)$。

2.6.2 双曲正切（Tanh）

$$\tanh(x) = \frac{e^x - e^{-x}}{e^x + e^{-x}} = 2\sigma(2x) - 1 \tag{2.12}$$

Tanh 的性质

- 输出范围：$(-1, 1)$
- 导数：$\tanh'(x) = 1 - \tanh^2(x)$
- 零中心化输出
- 单调递增

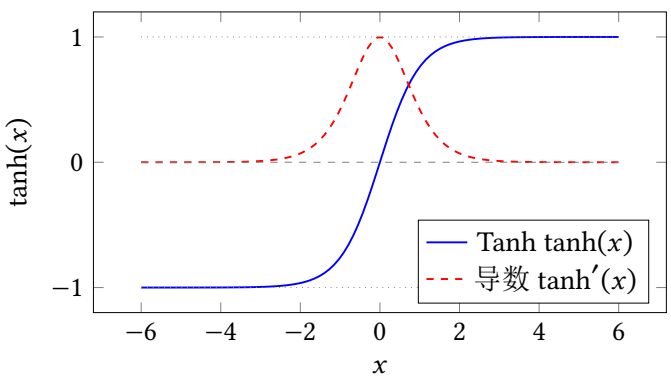


Figure 2.4 – Tanh 函数及其导数

优点

- 零中心化输出（更强的梯度）
- 比 sigmoid 更强的梯度
- 输出在 $(-1, 1)$，通常对隐藏层更好

缺点

- 梯度消失：对于大的 $|x|$ 仍然存在
- 指数计算
- 在 ± 1 处饱和

2.7 修正线性单元（ReLU）族

ReLU 族由于其计算效率和训练深度网络的有效性，已成为现代深度学习中隐藏层的主流选择。

2.7.1 ReLU

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x & \text{如果} x > 0 \\ 0 & \text{如果} x \leq 0 \end{cases} \tag{2.13}$$

ReLU 的性质

- 输出范围：$[0, \infty)$
- 导数：

$$\mathrm{ReLU}'(x) = \begin{cases} 1 & \text{如果} x > 0 \\ 0 & \text{如果} x \leq 0 \end{cases} \tag{2.14}$$

- 计算高效
- 稀疏激活

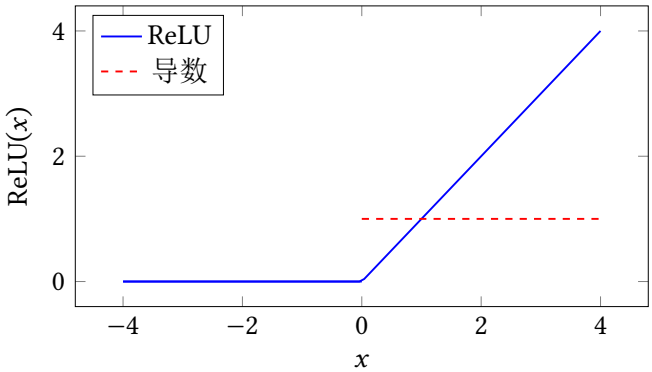


Figure 2.5 – ReLU 函数及其导数

优点

- 计算非常快
- 减少正激活的梯度消失问题
- 稀疏表示：并非所有神经元同时激活
- 在深度网络中表现优异
- 生物学合理性

2.7.1.1 为什么 ReLU 防止梯度消失：数学分析

ReLU 相对于 sigmoid 的关键优势是其非饱和特性。对于任何预激活 $z_l > 0$ 的神经元：

$$\mathrm{ReLU}'(z_l) = 1 \quad \text{（无衰减）} \tag{2.15}$$

在使用 ReLU 激活的深度网络中，”活跃”路径的梯度为：

$$\frac{\partial \mathscr{L}}{\partial z_1} = \frac{\partial \mathscr{L}}{\partial z_L} \cdot \prod_{l=2}^{L} \mathbf{1}_{\{z_l>0\}} \cdot \mathbf{W}_l \tag{2.16}$$

指示函数 $\mathbf{1}_{\{z_l>0\}}$ 要么是 0 要么是 1——没有指数衰减。如果每层大约一半神经元是活跃的，梯度幅度为：

$$\left\|\frac{\partial \mathscr{L}}{\partial z_1}\right\| \approx \left(\frac{1}{2}\right)^{L-1} \cdot \prod_{l=2}^{L} \|\mathbf{W}_l\| \cdot \left\|\frac{\partial \mathscr{L}}{\partial z_L}\right\| \tag{2.17}$$

虽然因子 $(1/2)^{L-1}$ 仍然指数衰减，但在实践中通过以下方式缓解：

- **He** 初始化 [He, X. Zhang, et al. 2015]：将权重方差设置为 $2/n_{\text{in}}$ 以补偿预期的不活跃神经元比例。
- 残差连接 [He, X. Zhang, et al. 2016a]：跳跃连接提供了一条绕过乘法衰减的替代梯度路径（第 6 章）。
- 批归一化 [Ioffe and Szegedy 2015]：稳定激活的分布，保持它们在非饱和区域。

2.7.1.2 ReLU、稀疏性与生物学动机

ReLU 的 $\max(0, x)$ 操作创建稀疏激活：对于典型的初始化，大约 50% 的神经元输出为零。这种稀疏性有几个好处：

1. 信息解耦：稀疏表示往往更易解释和鲁棒（类似于神经科学中的稀疏编码）。

2. 效率：零值神经元对下游计算没有贡献，支持硬件优化。
3. 生物学合理性：生物神经元表现出阈值行为——只有当输入超过阈值时才发放。ReLU 定性地捕获了这一点，不像平滑的 sigmoid。

缺点

- 死亡 **ReLU** 问题：如果神经元的权重更新使得 $\mathbf{w}^\top \mathbf{x} + b \leq 0$ 对所有输入，该神经元将永远输出零。
- 非零中心化
- 输出无界

死亡 ReLU 问题

如果神经元的权重更新使得 $\mathbf{w}^\top \mathbf{x} + b \leq 0$ 对所有输入，该神经元将永远输出零。这被称为" 死亡 ReLU" 或" 死神经元" 问题。当输入为负时梯度为零，神经元停止学习。
解决方案包括：

- Leaky ReLU
- PReLU
- ELU
- 正确的权重初始化
- 较低的学习率

2.7.2 Leaky ReLU

$$\mathrm{LeakyReLU}(x) = \begin{cases} x & \text{如果} x > 0 \\ \alpha x & \text{如果} x \leq 0 \end{cases} = \max(0, x) + \alpha \min(0, x) \tag{2.18}$$

其中 α 是一个小的正常数（通常 $\alpha = 0.01$）。

Leaky ReLU 的性质

- 输出范围：$(-\infty, \infty)$
- 导数：

$$\mathrm{LeakyReLU}'(x) = \begin{cases} 1 & \text{如果} x > 0 \\ \alpha & \text{如果} x \leq 0 \end{cases} \tag{2.19}$$

- 允许小的负值

优点

- 没有死亡 **ReLU** 问题
- 允许小的负梯度
- 近似零中心化

缺点

- 超参数 α 需要调整
- 性能可能不会显著优于 ReLU

2.7.3 参数化 ReLU（PReLU）

$$\mathrm{PReLU}(x) = \begin{cases} x & \text{如果} x > 0 \\ \alpha x & \text{如果} x \leq 0 \end{cases} \tag{2.20}$$

其中 α 是一个可学习参数（可以是每通道的）。

PReLU 允许网络学习负值的最佳斜率，可能提高性能。

2.7.4 指数线性单元（ELU）

$$\mathrm{ELU}(x) = \begin{cases} x & \text{如果} x > 0 \\ \alpha(e^x - 1) & \text{如果} x \leq 0 \end{cases} \tag{2.21}$$

其中 $\alpha > 0$ 是一个超参数。

ELU 的性质

- 输出范围：$(-\alpha, \infty)$
- 对于 $x < 0$ 平滑
- 负值使均值更接近零

优点

- 没有死亡 **ReLU** 问题
- 输出更接近零均值
- 更平滑的梯度

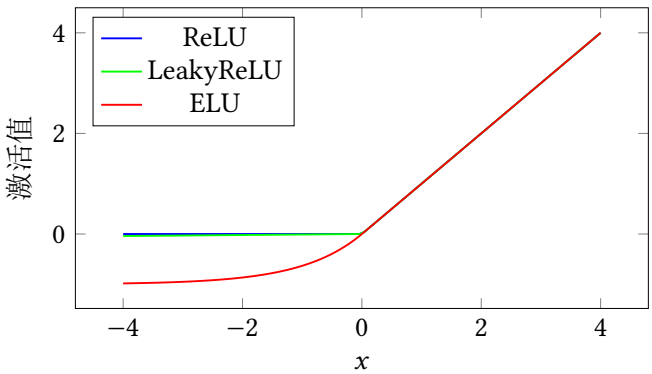


Figure 2.6 – ReLU 及其变体的比较

缺点

- 负值的指数计算
- 前向传播较慢

2.7.5 Swish / SiLU

$$\text{Swish}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{2.22}$$

Swish 已成为深度网络中 ReLU 的广泛使用的平滑替代方案 [Ramachandran, Zoph, and Q. V. Le 2017]。

Swish 的性质

- 输出范围：大约 $(-0.31, \infty)$
- 自门控
- 非单调（对于小的负值有凹陷）
- 导数：$\text{Swish}'(x) = \sigma(x) + x \cdot \sigma(x)(1 - \sigma(x)) = \text{Swish}(x) + \sigma(x)(1 - \text{Swish}(x))$

2.7.6 Mish

Mish [Misra 2019] 是一个自正则化的非单调激活函数：

$$\text{Mish}(x) = x \cdot \tanh(\text{softplus}(x)) = x \cdot \tanh(\ln(1 + e^x)) \tag{2.23}$$

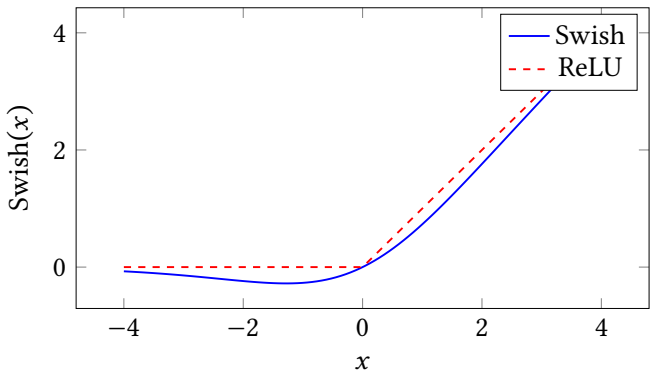


Figure 2.7 – Swish vs ReLU

Mish 的性质

- 输出范围：大约 $(-0.308, \infty)$（下方有界，上方无界）
- 非单调：类似于 Swish，对负的 x 略有凹陷
- 处处光滑：无限可微（C^∞）
- 自正则化：非单调形状作为隐式正则化器

导数是：

$$\text{Mish}'(x) = \frac{e^x \cdot (4(e^x + 2)) + 4x \cdot e^{2x} + 2 \cdot e^{3x} + x \cdot e^{4x} + 2}{(2e^x + 2)^2} \tag{2.24}$$

虽然导数很复杂，但关键洞察是 Mish 始终保持正梯度（像 ReLU），但对于负输入有更平滑、有界的梯度，提供温和的正则化。在经验上，Mish 在图像分类基准测试（ImageNet、CIFAR-10/100）和目标检测任务上始终匹配或优于 ReLU 和 Swish。

优点

- 在视觉任务上比 ReLU/Swish 表现更强
- 平滑、非单调形状提供隐式正则化
- 无需额外超参数

缺点

- 计算更昂贵（tanh + softplus）
- 导数公式复杂

2.7.7 StarReLU

StarReLU，在 [Maulik et al. 2024] 中引入，是一种专为 transformer 设计的新型自门控激活函数：

$$\text{StarReLU}(x) = s \cdot \text{ReLU}(x)^2 + \beta \tag{2.25}$$

其中 s 和 β 是可学习参数（初始化为 $s = 1, \beta = 0$）。

动机：在 transformer 中，计算瓶颈是应用于 $N \times d$ 隐藏状态张量每个元素的激活函数。StarReLU 的二次形式 $\text{ReLU}(x)^2$ 比 GELU/Swish 更简单（无 tanh 或 σ），同时提供：

- 自门控：平方运算自然地将负输入归零。
- 可学习的尺度和偏移：s 和 β 调整激活范围。
- 计算效率：在 transformer 块中比 GELU 减少 60% 的 FLOPs，没有精度损失。

StarReLU 代表了设计硬件感知激活函数的趋势——利用这样一个事实：在现代硬件上，简单操作（乘法、加法）比超越函数（exp、tanh、erf）便宜得多。

2.7.8 GELU（高斯误差线性单元）

$$\text{GELU}(x) = x \cdot \Phi(x) \tag{2.26}$$

其中 $\Phi(x)$ 是标准正态分布的累积分布函数：

$$\Phi(x) = \frac{1}{2}\left[1 + \text{erf}\left(\frac{x}{\sqrt{2}}\right)\right] \tag{2.27}$$

一个计算高效的近似：

$$\text{GELU}(x) \approx \frac{x}{2} \cdot \left(1 + \tanh\left(\sqrt{\frac{2}{\pi}}\left(x + 0.044715x^3\right)\right)\right) \tag{2.28}$$

GELU 用于 BERT、GPT 和许多 transformer 架构。

2.7.8.1 概率解释：GELU、Dropout 和高斯 CDF

GELU 函数有一个连接它与 dropout [Clark 2022] 的深层概率解释。考虑一个随机神经元，其中输入 x 乘以随机伯努利掩码 $m \sim \text{Bernoulli}(\Phi(x))$：

$$y = x \cdot m, \quad P(m = 1 \mid x) = \Phi(x) \tag{2.29}$$

这个随机神经元的期望输出为：

$$\mathbb{E}[y \mid x] = x \cdot \Phi(x) = \text{GELU}(x) \tag{2.30}$$

关键洞察：GELU 是一个随机门控神经元的确定性期望，其中门控概率等于高斯 CDF。这提供了一种统一视图：

- **Dropout** 以与输入无关的概率 p 随机将激活置零。
- **GELU** 根据激活幅度自适应地门控激活（与输入相关）。
- 两者都可以看作乘性噪声，但 GELU 的噪声是自适应的——较大的输入更可能通过。

这个解释解释了为什么 GELU 在 transformer 中效果这么好：它提供了一种依赖于输入幅度的平滑、”软” 版本的 dropout，提供隐式正则化，而不像注意力机制中随机 dropout 那样有训练不稳定性。

2.7.8.2 GELU 的导数

从 $\text{GELU}(x) = x\Phi(x)$，通过乘积规则：

$$\text{GELU}'(x) = \Phi(x) + x\phi(x) \tag{2.31}$$

其中 $\phi(x) = \frac{1}{\sqrt{2\pi}}e^{-x^2/2}$ 是标准正态分布的概率密度函数。注意 $\text{GELU}'(x) > 0$ 对所有 x（函数严格递增），但 $\text{GELU}'(x) < 1$（不像 ReLU），提供自然的梯度缩放。

GELU 的性质

- 与 **dropout** 结合时是随机正则化器
- 在所有区域非线性
- 大约等于：$0.5x(1 + \tanh(\sqrt{2/\pi}(x + 0.044715x^3)))$
- 零中心化输出

优点

- Transformer 中的最先进性能
- 结合了 ReLU 和 dropout 的优点
- 平滑且非单调

2.8 特殊激活函数

2.8.1 Softmax

$$\text{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{K} e^{x_j}} \quad \text{对于} i = 1, \ldots, K \tag{2.32}$$

Softmax 用于多分类输出层。

Softmax 的性质

- 输出范围：每个输出在 $(0, 1)$
- 求和约束：$\sum_i \text{softmax}(x_i) = 1$
- 概率解释
- 平移不变：对于任意常数 c，$\text{softmax}(x + c) = \text{softmax}(x)$

优点

- 输出是有效的概率分布
- 支持交叉熵损失优化
- 自然处理多分类

缺点

- 仅用于输出层（不是隐藏层）
- 当 logits 非常大时可能饱和

实践中的 Softmax

为数值稳定性，实现 softmax 为：

$$\text{softmax}(x_i) = \frac{e^{x_i - \max(x)}}{\sum_j e^{x_j - \max(x)}} \tag{2.33}$$

这减去最大值以防止指数溢出。

2.9 选择激活函数

激活函数的选择取决于具体任务和网络架构。以下是一般指南：

Table 2.2 – 激活函数选择指南

层类型	推荐函数	注意事项
隐藏层（通用）	ReLU, GELU	ReLU 是默认选择
隐藏层（深度）	ReLU, GELU, ELU	避免深度网络中的 sigmoid/tanh
隐藏层（**CNN**）	ReLU, GELU	医学成像用 LeakyReLU
隐藏层（**RNN**）	Tanh	RNN 的通常默认选择
隐藏层（**Transformer**）	GELU	用于 BERT、GPT 等
输出（二分类）	Sigmoid	用于概率输出
输出（多分类）	Softmax	用于概率分布
输出（回归）	恒等函数 / 无	用于无界输出
输出（有界）	Sigmoid, Tanh	用于 $[0, 1]$ 或 $[-1, 1]$ 输出

2.10 实践注意事项

2.10.1 权重初始化

不同的激活函数与适当的权重初始化配合使用效果最佳：

- **Sigmoid/Tanh**：Xavier/Glorot 初始化（$\mathcal{N}(0, \sqrt{2/(n_{in} + n_{out})})$）
- **ReLU**（及其变体）：He 初始化（$\mathcal{N}(0, \sqrt{2/n_{in}})$）
- **GELU**：He 初始化效果良好

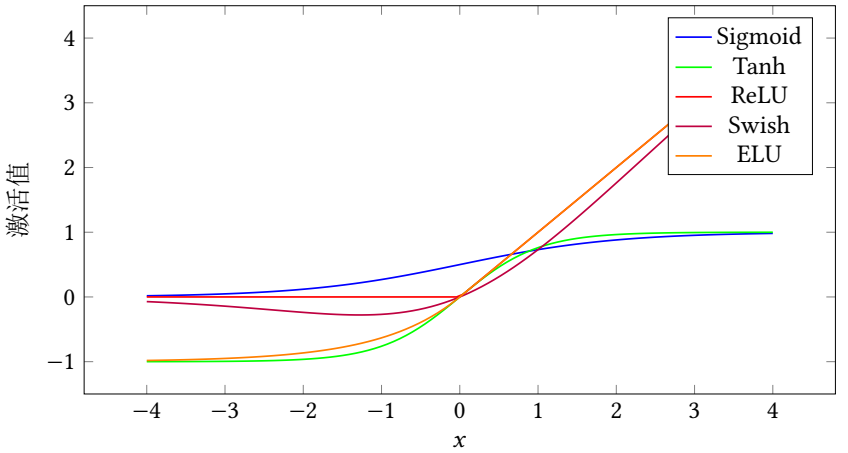


Figure 2.8 – 各种激活函数的比较

详见第 7 章优化部分。

2.10.2 批归一化与激活函数的结合

使用批归一化时，激活函数通常在批归一化之后：

$$\mathbf{y} = f(\mathrm{BN}(\mathbf{W}\mathbf{x} + \mathbf{b})) \tag{2.34}$$

这种顺序是原始 BatchNorm 论文推广的，现在已成为标准实践。

2.11 PyTorch 实现

代码 2.1 – PyTorch 中的激活函数

```python
import torch
import torch.nn as nn

# 常用激活函数
relu = nn.ReLU()
leaky_relu = nn.LeakyReLU(0.01)  # 负值斜率
prelu = nn.PReLU()  # 可学习斜率
elu = nn.ELU(alpha=1.0)
selu = nn.SELU()  # 自归一化神经网络
gelu = nn.GELU()  # 用于Transformer
silu = nn.SiLU()  # 与Swish相同
sigmoid = nn.Sigmoid()
tanh = nn.Tanh()
softmax = nn.Softmax(dim=1)

```

```
# 示例用法
x = torch.randn(32, 128)  # 批次32, 128个特征
y = relu(x)  # 应用ReLU

# 带维度规范的Softmax
logits = torch.randn(32, 10)  # 10个类别
probs = softmax(dim=1)(logits)

# Transformer中的GELU
transformer_layer = nn.TransformerEncoderLayer(
    d_model=512,
    nhead=8,
    activation='gelu'  # 或直接使用nn.GELU()
)
```

2.12 本章小结

1. 激活函数将非线性引入神经网络，使其能够学习复杂模式。
2. **Sigmoid** 和 **Tanh** 将输入压缩到有界范围，但存在梯度消失问题。
3. **ReLU** 计算高效且减少梯度消失，但可能遭受神经元死亡问题。
4. 变体如 **Leaky ReLU**、**PReLU** 和 **ELU** 解决了死亡 ReLU 问题。
5. **GELU** 在 transformer 架构中变得流行。
6. **Softmax** 用于多分类输出层。
7. 选择取决于任务、架构和训练稳定性。

Table 2.3 – 激活函数快速参考

函数	公式	范围	常见用途
Sigmoid	$1/(1+e^{-x})$	$(0,1)$	二分类输出
Tanh	$(e^x - e^{-x})/(e^x+e^{-x})$	$(-1,1)$	RNN，很少用于 CNN
ReLU	$\max(0,x)$	$[0,\infty)$	默认隐藏层
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty,\infty)$	防止死亡
ELU	x if $x>0$, else $\alpha(e^x-1)$	$(-\alpha,\infty)$	更好的梯度
GELU	$x\cdot\Phi(x)$	变化	Transformer
Softmax	$e^{x_i}/\sum e^{x_j}$	$(0,1)^K$	多分类输出

2.13 扩展阅读

- [Hornik 1991] - 通用逼近定理：理论基础
- [Clevert, Unterthiner, and Hochreiter 2015] - 指数线性单元快速准确深度网络学习
- [He, X. Zhang, et al. 2015] - 深入修正器：He 初始化
- [Hendrycks and Gimpel 2016] - 高斯误差线性单元
- [Ramachandran, Zoph, and Q. V. Le 2017] - 激活函数的系统比较
- [Misra 2019] - Mish：一种自正则化非单调神经激活函数
- [Clark 2022] - GELU 激活和 dropout 的统一视图
- [Maulik et al. 2024] - StarReLU：高效 transformer 的自门控激活函数
- [I. Goodfellow, Bengio, and Courville 2016] - 深度学习，第 6 章

2.14 习题

1. 梯度分析：计算 ELU 函数在正输入和负输入处的梯度。
2. 比较：使用 Python/Matplotlib 绘制本章涵盖的激活函数及其导数。
3. 死亡 **ReLU**:实现一个简单实验来演示死亡 ReLU 问题，并展示 LeakyReLU 如何解决它。
4. 数值稳定性：说明为什么在计算 softmax 前从 logits 减去 $\max(x)$ 可以提高数值稳定性。
5. 自定义激活：设计并实现一个结合 ReLU 和 GELU 特性的自定义激活函数。在简单的图像分类任务上测试它。
6. 自归一化：研究 SELU（缩放指数线性单元）并解释它如何实现自归一化神经网络。

Chapter 3

损失函数

3.1 引言

损失函数（也称为目标函数、成本函数或误差函数）是量化神经网络预测与真实目标匹配程度的数学度量。训练过程旨在通过基于梯度的优化调整网络参数来最小化此损失。

损失函数的选择取决于：

- 任务类型（分类、回归等）
- 目标变量的分布
- 对解决方案的期望属性（稀疏性、鲁棒性等）
- 计算考虑

3.2 损失函数的作用

给定数据集 $\mathscr{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$ 和模型 $f(\mathbf{x};\theta)$，经验风险为：

$$\mathscr{R}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell(f(\mathbf{x}_i;\theta), \mathbf{y}_i) \tag{3.1}$$

其中 $\ell(\cdot,\cdot)$ 是损失函数。

训练寻求找到：

$$\theta^* = \arg\min_{\theta} \mathscr{R}(\theta) \tag{3.2}$$

损失函数应该：

1. 可微分（用于基于梯度的优化）
2. 适当惩罚较大误差
3. 鼓励特定任务的正确行为
4. 具有有用的统计解释

3.3 回归损失函数

回归任务预测连续值。损失函数的选择编码了关于噪声分布的假设，并决定了模型如何处理异常值。

3.3.1 均方误差（MSE）

均方误差（L2 损失）是最常见的回归损失：

$$\mathscr{L}_{MSE} = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2 = \frac{1}{N}\sum_{i=1}^{N} e_i^2 \tag{3.3}$$

其中 $e_i = y_i - \hat{y}_i$ 是残差。

3.3.1.1 推导：MSE 作为高斯噪声下的最大似然估计

MSE 的统计证明来自最大似然估计。假设数据由以下方式生成：

$$y_i = f(\mathbf{x}_i;\theta) + \epsilon_i, \quad \epsilon_i \overset{\text{i.i.d.}}{\sim} \mathscr{N}(0,\sigma^2) \tag{3.4}$$

单个观测的似然为：

$$p(y_i \mid \mathbf{x}_i;\theta,\sigma^2) = \frac{1}{\sqrt{2\pi\sigma^2}}\exp\left(-\frac{(y_i-\hat{y}_i)^2}{2\sigma^2}\right) \tag{3.5}$$

完整数据集的负对数似然（NLL）：

$$\mathscr{L}_{\text{NLL}} = -\sum_{i=1}^{N}\log p(y_i \mid \mathbf{x}_i;\theta,\sigma^2) \tag{3.6}$$

$$= -\sum_{i=1}^{N}\left[-\frac{1}{2}\log(2\pi\sigma^2) - \frac{(y_i-\hat{y}_i)^2}{2\sigma^2}\right] \tag{3.7}$$

$$= \frac{N}{2}\log(2\pi\sigma^2) + \frac{1}{2\sigma^2}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2 \tag{3.8}$$

由于 $\frac{N}{2}\log(2\pi\sigma^2)$ 是不依赖于 θ 的常数：

$$\boxed{\arg\min_{\theta}\mathscr{L}_{\text{NLL}} = \arg\min_{\theta}\sum_{i=1}^{N}(y_i-\hat{y}_i)^2 = \arg\min_{\theta}\mathscr{L}_{MSE}} \tag{3.9}$$

要点：最小化 MSE 等同于在加性高斯噪声假设下拟合模型。这就是为什么当误差大致呈高斯分布时 MSE 效果良好，但当噪声分布具有重尾（例如拉普拉斯噪声）时失败的原因。

3.3.1.2 梯度分析

MSE 相对于预测的梯度：

$$\frac{\partial \mathscr{L}_{MSE}}{\partial \hat{y}_i} = -\frac{2}{N}(y_i - \hat{y}_i) = \frac{2}{N}e_i \tag{3.10}$$

梯度与误差呈线性关系——这是一把双刃剑：

- 接近最优（$e_i \approx 0$）：梯度很小，实现精确收敛。
- 远离最优或异常值（$|e_i| \gg 0$）：梯度非常大，导致超调和对异常值的过度敏感。

3.3.1.3 与方差和 R^2 的联系

对于模型 $\hat{y}_i = f(\mathbf{x}_i)$，总方差分解成立：

$$\underbrace{\sum_i (y_i - \bar{y})^2}_{\text{总 SS}} = \underbrace{\sum_i (\hat{y}_i - \bar{y})^2}_{\text{解释 SS}} + \underbrace{\sum_i (y_i - \hat{y}_i)^2}_{\text{残差 SS（MSE} \times N\text{）}} \tag{3.11}$$

这给出了 R^2 分数：$R^2 = 1 - \frac{\text{MSE}}{\text{Var}(y)}$。

3.3.2 平均绝对误差（MAE）

平均绝对误差（L1 损失）：

$$\mathscr{L}_{MAE} = \frac{1}{N}\sum_{i=1}^{N} |y_i - \hat{y}_i| \tag{3.12}$$

3.3.2.1 推导：MAE 作为拉普拉斯噪声下的 MLE

如果噪声服从拉普拉斯（双指数）分布：

$$\epsilon_i \sim \text{Laplace}(0, b), \quad p(\epsilon) = \frac{1}{2b}\exp\left(-\frac{|\epsilon|}{b}\right) \tag{3.13}$$

那么：

$$-\log p(y_i \mid \mathbf{x}_i; \theta) = \log(2b) + \frac{|y_i - \hat{y}_i|}{b} \tag{3.14}$$

$$\propto |y_i - \hat{y}_i| \tag{3.15}$$

因此 MAE 是拉普拉斯噪声下的 MLE 损失。拉普拉斯分布比高斯分布有更重的尾部，使 MAE 本质上对异常值更鲁棒。

3.3.2.2 比较：MSE vs MAE

Table 3.1 – MSE vs MAE：综合比较

性质	**MSE (L_2)**	**MAE (L_1)**
噪声假设	高斯 $\mathcal{N}(0, \sigma^2)$	拉普拉斯 Laplace$(0, b)$
异常值敏感性	高（二次惩罚）	低（线性惩罚）
关于 $\hat{y}$ 的梯度	$-2(y - \hat{y})/N$	$-\text{sign}(y - \hat{y})/N$
最优 $\hat{y}^*$	均值 $\bar{y}$	中位数 $\tilde{y}$
光滑性	处处光滑	在 0 处不可微
收敛速度	更快（二次近最优）	更慢（常数梯度）
统计效率	更高（对于高斯数据）	更低

关键洞察：MSE 和 MAE 之间的差异最终归结为关于噪声分布的信念。如果你期望偶尔出现大的异常值，MAE 更合适。如果误差确实是高斯分布的，MSE 在统计上是最优的。

3.3.3 Huber 损失：两全其美

Huber 损失 [Huber 1964] 在小误差时平滑地插值于 MSE，在大误差时插值于 MAE：

$$\mathscr{L}_{\text{Huber}}(y, \hat{y}) = \begin{cases} \frac{1}{2}(y - \hat{y})^2 & \text{如果} |y - \hat{y}| \leq \delta \\ \delta \cdot |y - \hat{y}| - \frac{1}{2}\delta^2 & \text{否则} \end{cases} \tag{3.16}$$

超参数 δ（通常 $\delta = 1$）控制过渡点：

- 对于 $|e| < \delta$：二次惩罚（像 MSE）——在最优附近高效优化。
- 对于 $|e| \geq \delta$：线性惩罚（像 MAE）——对异常值鲁棒。

示例 **3.1** (Huber 损失梯度). *Huber* 损失的梯度是连续的：

$$\frac{\partial \mathscr{L}_{Huber}}{\partial \hat{y}} = \begin{cases} \hat{y} - y & \text{如果} |y - \hat{y}| \leq \delta \\ \delta \cdot sign(\hat{y} - y) & \text{否则} \end{cases} \tag{3.17}$$

对于大误差，梯度被限制在 $\pm\delta$，防止异常值导致梯度爆炸，同时在最优附近保持平滑的二次行为。

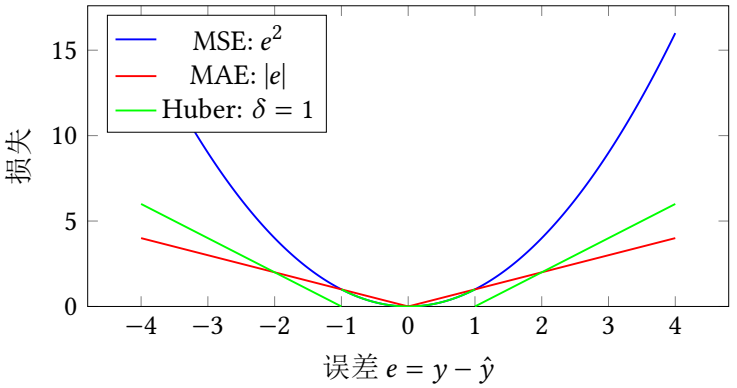


Figure 3.1 – MSE、MAE 和 Huber 损失。Huber 损失在零附近是二次的（快速收敛），对大误差是线性的（异常值鲁棒）。

3.4 分类损失函数

分类任务预测离散类别标签。损失函数必须鼓励模型为正确类别分配高概率。

3.4.1 从似然到交叉熵：完整推导链

要真正理解为什么交叉熵是标准分类损失，我们必须从头开始追踪完整推导。

3.4.1.1 第 1 步：生成模型

对于 K 类分类问题，我们将输出建模为由网络参数化的分类分布：

$$p(y = k \mid \mathbf{x}; \theta) = \hat{y}_k, \quad k = 1, \dots, K \tag{3.18}$$

其中 $\hat{y}_k = \mathrm{softmax}(\mathbf{z})_k$ 且 $\mathbf{z} = f(\mathbf{x}; \theta)$ 是 logits。

3.4.1.2 第 2 步：最大似然

给定 N 个独立同分布样本，似然为：

$$\mathscr{L}(\theta) = \prod_{i=1}^{N} p(y_i \mid \mathbf{x}_i; \theta) = \prod_{i=1}^{N} \prod_{k=1}^{K} \hat{y}_{ik}^{y_{ik}} \tag{3.19}$$

其中 $y_{ik} \in \{0, 1\}$ 是 one-hot 编码。

3.4.1.3 第 3 步：负对数似然 = 交叉熵

取负对数：

$$-\log \mathscr{L}(\theta) = -\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik}\log \hat{y}_{ik} \tag{3.20}$$

$$= -\sum_{i=1}^{N}\log \hat{y}_{i,c_i} \quad （因为 y_{ik} = \delta_{k,c_i}） \tag{3.21}$$

其中 c_i 是样本 i 的真实类别。这就是分类交叉熵损失。

3.4.1.4 第 4 步：与信息理论的联系

回忆第 1 章，真实分布 p（one-hot：$p_{c_i} = 1$，对于 $k \neq c_i$ 时 $p_k = 0$）和预测分布 $q = \hat{\mathbf{y}}$ 之间的交叉熵为：

$$H(p,q) = -\sum_{k=1}^{K} p_k \log q_k = -\log q_{c_i} \tag{3.22}$$

$$H(p) = 0 \quad （真实分布是确定性的） \tag{3.23}$$

因此：

$$D_{KL}(p\|q) = H(p,q) - H(p) = -\log \hat{y}_{c_i} = \mathscr{L}_{\mathrm{CE}} \tag{3.24}$$

最小化交叉熵 ≡ 最小化 KL 散度 ≡ 最大似然。

3.4.1.5 第 5 步：美丽的梯度

单个样本的 softmax-交叉熵梯度：

$$\frac{\partial \mathscr{L}}{\partial z_j} = \frac{\partial}{\partial z_j}\left(-\log \frac{e^{z_c}}{\sum_k e^{z_k}}\right) \tag{3.25}$$

$$= -\frac{\partial}{\partial z_j}\left(z_c - \log \sum_k e^{z_k}\right) \tag{3.26}$$

$$= \begin{cases} \hat{y}_j - 1 = \hat{y}_j - y_j & 如果 j = c \\ \hat{y}_j - 0 = \hat{y}_j - y_j & 如果 j \neq c \end{cases} \tag{3.27}$$

$$= \boxed{\hat{y}_j - y_j} \tag{3.28}$$

梯度简单地是预测减真实。这个优雅的结果意味着：

- 如果 $\hat{y}_c$（正确类别的预测概率）太低，梯度向上推。
- 如果 $\hat{y}_j$（错误类别的预测概率）太高，梯度向下推。
- 梯度自动按置信度缩放——非常错误的预测产生更大的梯度。

3.4.2 二元交叉熵

对于 $y \in \{0, 1\}$ 的二分类，带有 sigmoid 输出 $\hat{y} = \sigma(z)$：

$$\mathscr{L}_{BCE} = -\frac{1}{N}\sum_{i=1}^{N}[y_i \log \hat{y}_i + (1 - y_i)\log(1 - \hat{y}_i)] \tag{3.29}$$

这是 $K = 2$ 时分类交叉熵的特殊情况（数值上更稳定的实现）。关于 logit z 的梯度同样优雅：

$$\frac{\partial \mathscr{L}_{BCE}}{\partial z} = \hat{y} - y = \sigma(z) - y \tag{3.30}$$

3.4.3 分类交叉熵

对于多分类（K 个类别）：

$$\mathscr{L}_{CE} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} y_{ik} \log \hat{y}_{ik} \tag{3.31}$$

其中 $\hat{y}_{ik} = \mathrm{softmax}(z_{ik})$。

数值稳定性：在实践中，我们直接计算 log-softmax：

$$\log \hat{y}_k = z_k - \log \sum_{j=1}^{K} e^{z_j} = z_k - \mathrm{logsumexp}(\mathbf{z}) \tag{3.32}$$

logsumexp 函数使用 log-sum-exp 技巧实现：

$$\mathrm{logsumexp}(\mathbf{z}) = m + \log \sum_{j} e^{z_j - m}, \quad m = \max(\mathbf{z}) \tag{3.33}$$

这防止了指数的下溢/上溢。

3.5 高级损失函数

3.5.1 Focal 损失

Focal 损失 [T.-Y. Lin, Goyal, et al. 2017] 解决了单阶段检测器密集目标检测中类别不平衡的基本问题。在目标检测中，绝大多数提议的锚框是背景（简单负样本），因此标准交叉熵损失被容易分类的示例所主导，这些示例提供很少的学习信号。

3.5.1.1 动机：类别不平衡问题

考虑一个每张图像有 N 个锚框的单阶段目标检测器，通常只有 $O(1)$ 个正（目标）锚框。交叉熵损失：

$$\mathscr{L}_{CE} = -\log p_t \tag{3.34}$$

其中如果 $y = 1$（正样本）$p_t = p$，如果 $y = 0$（负样本）$p_t = 1 - p$。问题是：

- 简单负样本（$p_t \geq 0.9$）：尽管有数百万个，但每个贡献 $-\log(0.9) \approx 0.105$——个体很小但总体很大。
- 困难正样本（$p_t \leq 0.1$）：稀有但关键，每个贡献 $-\log(0.1) \approx 2.3$——大约是简单负样本的 22 倍。
- 综合效果：困难示例的梯度信号被简单示例的数量所淹没。

3.5.1.2 调制因子

Focal 损失添加了一个调制因子 $(1 - p_t)^\gamma$，下放容易分类示例的损失：

$$\mathscr{L}_{\text{FL}} = -(1 - p_t)^\gamma \log p_t \tag{3.35}$$

3.5.1.3 调制因子分析

对于不同的 p_t 和 γ 值：

关键观察：

- 当 $\gamma = 0$ 时，Focal 损失退化为标准交叉熵。
- 当 $\gamma = 2$ 时：$p_t = 0.9$ 的示例比 $p_t = 0.1$ 的示例贡献少 100 倍。
- 当 $\gamma = 5$ 时：容易示例基本贡献零损失。

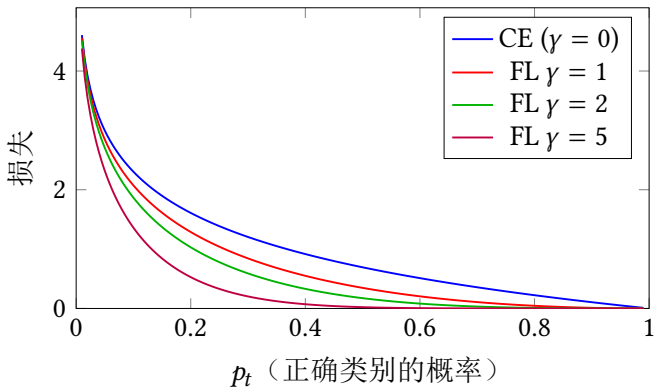


Figure 3.2 – 不同 γ 值的 Focal 损失。随着 γ 增加，容易示例（$p_t \to 1$）的损失被越来越多地抑制。

Table 3.2 – Focal 损失调制因子 $(1 - p_t)^\gamma$

p_t	$\gamma = 0$	$\gamma = 1$	$\gamma = 2$	$\gamma = 5$
0.1（困难）	1.000	0.900	0.810	0.590
0.3（中等）	1.000	0.700	0.490	0.168
0.5（模糊）	1.000	0.500	0.250	0.031
0.9（容易）	1.000	0.100	0.010	~0
0.99（非常容易）	1.000	0.010	0.0001	~0

3.5.1.4 Alpha 平衡 Focal 损失

在实践中，Focal 损失还包含类别平衡权重 $\alpha_t \in [0, 1]$：

$$\mathscr{L}_{\text{FL}} = -\alpha_t (1 - p_t)^\gamma \log p_t \tag{3.36}$$

α 和 γ 之间的相互作用很微妙。在 Lin 等人的实验中，$\alpha = 0.25$ 和 $\gamma = 2$ 取得了最佳性能。作者发现随着 γ 增加，α 的重要性降低——因为调制因子已经有效地降低多数类的权重。

3.5.2 标签平滑

标签平滑 [Szegedy, Vanhoucke, et al. 2016]，作为 Inception-v2 架构的一部分引入，用软标签替换硬 one-hot 标签。尽管简单，它已成为现代深度学习（尤其是大型语言模型 GPT、BERT）的标准技术。

3.5.2.1 机制

给定硬标签 $\mathbf{y} = [0, \dots, 1, \dots, 0]$（在位置 c 的 one-hot），标签平滑构建：

$$y_k^{\text{LS}} = \begin{cases} 1 - \epsilon + \frac{\epsilon}{K} & \text{如果} k = c \\ \frac{\epsilon}{K} & \text{如果} k \neq c \end{cases} \tag{3.37}$$

其中 $\epsilon \in [0, 1)$ 是平滑参数（通常 $\epsilon = 0.1$）。例如，对于 $K = 10$ 个类别和 $\epsilon = 0.1$：

$$\mathbf{y}^{\text{LS}} = [0.01, 0.01, \dots, 0.91, \dots, 0.01] \tag{3.38}$$

3.5.2.2 为什么标签平滑有效？三个视角

视角 **1**：防止过度自信。

硬标签鼓励模型将 logits 推到极值（$z_c \to +\infty$，$z_{k \neq c} \to -\infty$）。这导致：

- 大的 **logit** 幅度，增加 softmax 输出差距。
- 校准差：模型的置信度与其准确性不匹配。
- 泛化能力下降：[R. Müller, Kornblith, and G. E. Hinton 2019] 表明，带有硬标签交叉熵的神经网络在 $\|\mathbf{w}_c\| \gg \|\mathbf{w}_{k \neq c}\|$ 处有最小值，即正确类别的权重向量比其他类别增长大得多。

标签平滑通过为所有类别分配非零概率来防止这种情况，将最大可实现置信度限制在 $1 - \epsilon + \epsilon/K$。

视角 **2**：通过 **KL** 散度的正则化。

带平滑标签的交叉熵分解为：

$$\mathscr{L}_{\text{LS}} = -\sum_k y_k^{\text{LS}} \log \hat{y}_k \tag{3.39}$$

$$= -(1 - \epsilon) \underbrace{\log \hat{y}_c}_{\text{硬标签的 CE}} - \frac{\epsilon}{K} \underbrace{\sum_{k=1}^{K} \log \hat{y}_k}_{\text{负熵}} \tag{3.40}$$

$$= (1 - \epsilon)\mathscr{L}_{CE} - \epsilon H(\hat{\mathbf{y}}) \tag{3.41}$$

第二项 $H(\hat{\mathbf{y}})$ 是预测分布的熵。最大化熵鼓励预测更均匀——一种熵正则化形式。这防止模型崩溃到非常尖锐的分布。

视角 **3**：贝叶斯解释。

标签平滑可以看作是对类别概率引入狄利克雷先验。如果 $p \sim \text{Dir}(\alpha \mathbf{1})$（对

称狄利克雷），则：

$$\mathbb{E}[\log p_k] = \psi(\alpha) - \psi(K\alpha) \tag{3.42}$$

其中 ψ 是双伽玛函数。对于小的 α，这近似于 $\log \frac{1}{K}$，对应于均匀标签平滑。

3.6 对比学习和自监督损失函数

自监督学习的最新进展（2020-2024）引入了一类新的损失函数，通过对比正样本对和负样本对来学习表示，无需人工标签。

3.6.1 对比损失

最简单的对比损失形式是为度量学习引入的（例如人脸验证、孪生网络）：

$$\mathscr{L}_{\text{contrastive}} = \frac{1}{2}\left[y \cdot d^2 + (1-y) \cdot \max(0, m-d)^2\right] \tag{3.43}$$

其中：

- $d = \|f(\mathbf{x}_1) - f(\mathbf{x}_2)\|_2$ 是两个嵌入之间的距离
- $y \in \{0, 1\}$：$y = 1$ 表示正样本对（同类别），$y = 0$ 表示负样本对
- $m > 0$ 是边界超参数

直觉：正样本对被拉近（$d \to 0$），而负样本对被推远直到它们至少相距 m（超过则不再有惩罚）。

3.6.2 三元组损失

三元组损失通过同时比较锚、正样本和负样本来扩展对比学习：

$$\mathscr{L}_{\text{Triplet}} = \max\left(0, \underbrace{\|f_a - f_p\|_2^2}_{\text{正样本距离}} - \underbrace{\|f_a - f_n\|_2^2}_{\text{负样本距离}} + \alpha\right) \tag{3.44}$$

几何解释：当正样本至少比负样本更接近锚 α 时损失为零。边界 α 控制强制执行的分离程度。

3.6.3 InfoNCE 损失

InfoNCE [Oord, Li, and Vinyals 2018]，现代对比学习（SimCLR [Ting Chen et al. 2020]、MoCo、CLIP）的基础，将表示学习框定为多选题问题：

定义 3.1 (InfoNCE 损失)**.** 给定一个查询 $\mathbf{q}$，一个正样本键 $\mathbf{k}_+$，和 $N-1$ 个负

样本键 $\{\mathbf{k}_-\}$：

$$\mathscr{L}_{InfoNCE} = -\log \frac{\exp(\mathbf{q}^\top \mathbf{k}_+/\tau)}{\sum_{i=1}^{N} \exp(\mathbf{q}^\top \mathbf{k}_i/\tau)} \tag{3.45}$$

其中 $\tau > 0$ 是温度超参数。

3.6.3.1 与互信息的联系

[Oord, Li, and Vinyals 2018] 中的显著理论结果表明，InfoNCE 提供了互信息的下界：

$$I(\mathbf{x}; \mathbf{y}) \geq \log N - \mathscr{L}_{\text{InfoNCE}} \tag{3.46}$$

这意味着最小化 InfoNCE 最大化查询和正样本键之间的互信息，鼓励编码器捕获同一图像增强视图之间的所有共享信息。

3.6.3.2 温度 τ 的作用

温度参数 τ 控制分布的" 锐度"：

- 小的 τ（例如 CLIP 中的 0.07）：使分布尖锐，增加对太接近的负样本的惩罚。鼓励模型更" 有区分力"。
- 大的 τ（例如 0.5-1.0）：更软的分布，负样本之间更均匀的梯度。

对于相似度分数 $s = \mathbf{q}^\top \mathbf{k}/\tau$ 和从维度 d 球体上的均匀分布绘制的 N 个负样本：

$$\mathbb{E}[\mathscr{L}_{\text{InfoNCE}}] \approx -\log \sigma\left(\frac{s_+ - \mathbb{E}[s_-]}{\sqrt{\text{Var}(s)}}\right) + \text{const} \tag{3.47}$$

这表明损失有效地衡量了正样本相似度与预期负样本相似度的可分离性。

3.6.4 Dice 损失

对于类别不平衡严重的分割任务（例如医学图像分割），Dice 损失 [Milletari, Navab, and Ahmadi 2016] 直接优化 Dice 系数：

$$\mathscr{L}_{\text{Dice}} = 1 - \frac{2\sum_i \hat{y}_i \cdot y_i + \epsilon}{\sum_i \hat{y}_i + \sum_i y_i + \epsilon} \tag{3.48}$$

3.6.4.1 推导：Dice 系数和 Jaccard 指数

Sørensen–Dice 系数衡量预测和真实分割掩码之间的重叠：

$$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|} \tag{3.49}$$

Jaccard 指数（并交比，IoU）与之相关：

$$\text{IoU} = \frac{|A \cap B|}{|A \cup B|} = \frac{\text{Dice}}{2 - \text{Dice}}, \quad \text{Dice} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}} \tag{3.50}$$

为什么 **Dice** 损失对类别不平衡有效：与独立对待每个像素的交叉熵不同，Dice 损失直接优化用于评估的重叠度量。这意味着：

- 单个假正样本像素将分子减少 1 并将分母增加 1——双重惩罚。
- 损失固有地被预测和真实区域的大小归一化，使其具有尺度不变性。
- 小物体比在交叉熵下贡献更大比例的损失。

实践技巧：Dice 损失经常与交叉熵结合为 $\mathscr{L} = \mathscr{L}_{CE} + \lambda \mathscr{L}_{\text{Dice}}$，其中 $\lambda \in [0.5, 1.0]$。交叉熵提供稳定的像素级梯度，而 Dice 损失确保区域级重叠。

3.6.5 KL 散度损失

当目标本身是一个概率分布（而不是 one-hot 标签）时使用 KL 散度损失：

$$\mathscr{L}_{KL}(p \| q) = \sum_k p_k \log \frac{p_k}{q_k} \tag{3.51}$$

应用：

- 知识蒸馏：目标 p 是教师的软输出分布（温度缩放后），q 是学生的预测。
- 变分推断（VAE）：将近似后验 $q(\mathbf{z}|\mathbf{x})$ 匹配到先验 $p(\mathbf{z})$。
- 域适应：对齐源域和目标域的特征分布。

重要：**KL** 散度是非对称的。$D_{KL}(p \| q) \neq D_{KL}(q \| p)$。在知识蒸馏中：

- 正向 **KL** $D_{KL}(p_{\text{teacher}} \| q_{\text{student}})$：”均值寻求”——学生覆盖教师的所有模态。用于蒸馏。
- 反向 **KL** $D_{KL}(q_{\text{student}} \| p_{\text{teacher}})$：”模态寻求”——学生对少数模态拟合良好但可能错过其他。在变分推断中使用。

3.7 选择损失函数

3.8 本章小结

损失函数是数学模型和学习目标之间的桥梁：

Table 3.3 – 损失函数选择指南

任务	损失	注意事项
二分类	BCE	标准选择
多分类	交叉熵	带标签平滑
回归	MSE	对异常值敏感
鲁棒回归	MAE/Huber	抗异常值
不平衡分类	Focal 损失	关注困难示例
分割	Dice + CE	处理类别不平衡

1. **MSE** = 高斯噪声下的 **MLE**：最小化 MSE 等同于在加性高斯噪声假设下拟合模型。二次惩罚使其对异常值敏感但在最优附近高效。
2. **MAE** = 拉普拉斯噪声下的 **MLE**：线性惩罚提供对异常值的鲁棒性但收敛较慢。Huber 损失结合了两者的优点。
3. 交叉熵 = 分类分布的 **MLE**：softmax-交叉熵组合产生优雅的梯度 $\hat{y}_k - y_k$，实现高效的分类训练。
4. **Focal** 损失：通过调制因子 $(1 - p_t)^\gamma$ 动态降低简单示例的权重，解决类别不平衡。对于密集预测任务必不可少。
5. 标签平滑：通过在类别间分布概率质量防止过度自信，与贝叶斯先验和熵正则化有联系。
6. 对比损失（**InfoNCE**）：自监督学习的基础，绑定同一数据点视图之间的互信息。
7. **Dice** 损失：直接优化分割的评估指标，自然处理类别不平衡。

3.9 扩展阅读

- [Huber 1964] — 位置参数的鲁棒估计（Huber 损失）
- [Bishop 2006] — 第 1-2 章概率基础
- [I. Goodfellow, Bengio, and Courville 2016] — 第 5 章（机器学习基础）和第 6 章（深度前馈网络）
- [Milletari, Navab, and Ahmadi 2016] — V-Net：分割的 Dice 损失
- [Szegedy, Vanhoucke, et al. 2016] — 重新思考 Inception 架构（标签平滑）
- [T.-Y. Lin, Goyal, et al. 2017] — 密集目标检测的 Focal 损失
- [Oord, Li, and Vinyals 2018] — 对比预测编码（InfoNCE）

Table 3.4 – 完整损失函数决策框架

场景	损失	原因	关键参考
干净回归	MSE	MLE，高斯噪声	
回归 + 异常值	Huber ($\delta = 1$)	鲁棒 + 高效	[Huber 1964]
二分类	BCE	MLE，伯努利	
多分类	交叉熵	MLE，分类	
不平衡检测	Focal 损失 ($\gamma = 2$)	降低简单负样本权重	[T.-Y. Lin, Goyal, et al. 2017]
医学分割	Dice + CE	重叠优化	[Milletari, Navab, and Ahmadi 2016]
自监督	InfoNCE ($\tau = 0.07$)	MI 最大化	[Ting Chen et al. 2020]
知识蒸馏	KL 散度	分布匹配	
NLP/大模型	CE + 标签平滑	校准	[Szegedy, Vanhoucke, et al. 2016]

- [Ting Chen et al. 2020] — SimCLR：对比学习简单框架
- [He, X. Chen, et al. 2022b] — 用于自监督视觉的掩码自编码器（MAE）

3.10 习题

1. **Huber vs MSE**：对于残差 $r \in \{-3, -1, 0, 1, 3\}$ 和 Huber 阈值 $\delta = 1$，计算每个残差的 Huber 损失，并将其与 MSE 和 MAE 比较。哪种损失对大异常值的惩罚最强？
2. **Softmax-交叉熵梯度**：从 $\mathscr{L} = -\sum_k y_k \log \hat{y}_k$ 且 $\hat{y}_k = \mathrm{softmax}(z_k)$ 开始，推导梯度 $\frac{\partial \mathscr{L}}{\partial z_k} = \hat{y}_k - y_k$。
3. 类别不平衡：考虑一个 95% 负样本和 5% 正样本的二分类数据集。解释为什么标准 BCE 可能表现不佳。Focal 损失如何在训练期间改变简单负样本的贡献？
4. 标签平滑：对于带平滑参数 $\varepsilon = 0.1$ 的 5 类分类任务，写出正确类别的平滑目标分布，并解释为什么这可以改善校准。
5. 对比学习：在批次大小 N 的 InfoNCE 中，每个样本有一个正样本和 $2N - 2$ 个负样本。增加批次大小如何影响目标分母和任务难度？
6. 分割损失：展示 Dice 损失在预测掩码过度分割与分割不足时的变化。为什么 Dice 损失在实践中经常与交叉熵结合使用？

Chapter 4

神经网络架构

4.1 引言

神经网络架构定义了神经元的组织方式和连接方式，以处理信息。不同的架构适用于不同类型的数据和任务。本章涵盖三种基本神经网络架构：深度神经网络（DNN）、卷积神经网络（CNN）和循环神经网络（RNN）。

4.2 深度神经网络（DNN）

深度神经网络由多层神经元组成，信息从输入流向输出，没有循环。

4.2.1 架构

全连接 DNN 执行：

$$\mathbf{h}^{(l)} = f^{(l)}(\mathbf{W}^{(l)}\mathbf{h}^{(l-1)} + \mathbf{b}^{(l)}) \tag{4.1}$$

其中 $l = 1, \ldots, L$ 是层的索引。

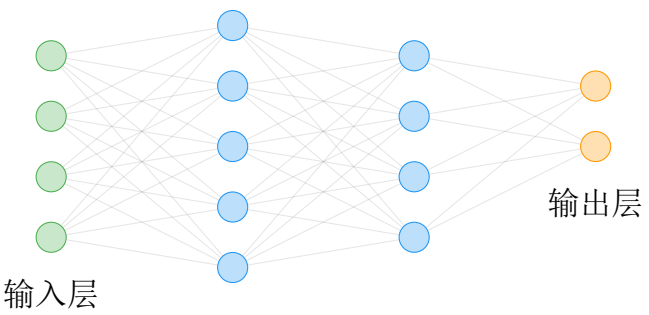


Figure 4.1 – 深度神经网络架构

DNN 特性

- 参数量：$\sum_{l=1}^{L}(n^{(l-1)} \cdot n^{(l)} + n^{(l)})$
- 通用逼近：在足够宽度/深度下可以逼近任何连续函数
- 适用于：表格数据、基于特征的分类

4.3 卷积神经网络（CNN）

CNN 通过局部连接和参数共享来利用数据的空间结构。

4.3.1 卷积层

卷积操作：

$$y[i,j] = \sum_m \sum_n w[m,n] \cdot x[i+m, j+n] \tag{4.2}$$

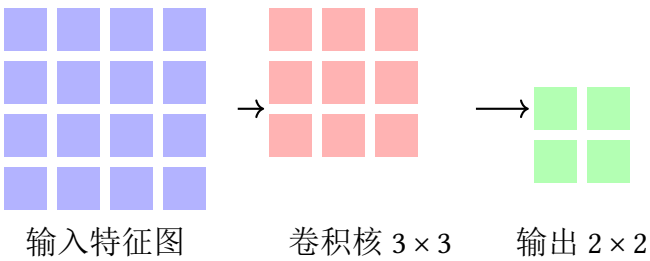


Figure 4.2 – 卷积操作

关键参数：

- 卷积核大小：$K \times K$（通常 3×3、5×5、7×7）
- 步长：s（步长大小，通常 1 或 2）
- 填充：p（用于控制输出大小的零填充）

输出尺寸：

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - K}{s} + 1 \right\rfloor \tag{4.3}$$

CNN 优势

- 局部感受野：每个神经元只看到局部区域
- 参数共享：跨空间位置使用相同权重
- 平移等变性：输入移位 → 输出移位
- 适用于：图像、视频、空间数据

4.3.2 感受野分析

感受野（RF）是影响其输出的输入图像区域。理解感受野增长对架构设计至关重要。

4.3.2.1 单层感受野

具有卷积核 K、步长 s 和填充 p 的单层卷积产生的输出感受野就是 K。输出尺寸为：

$$o_i = \left\lfloor \frac{i_{in} + 2p - K}{s} + 1 \right\rfloor \tag{4.4}$$

4.3.2.2 多层感受野

对于 L 层卷积堆叠，输出的有效感受野（ERF）增长为：

$$\mathrm{RF} = 1 + \sum_{l=1}^{L} \left((K_l - 1) \cdot \prod_{l'=1}^{l-1} s_{l'} \right) \tag{4.5}$$

示例 **4.1** (两个 3×3 卷积). 两个步长为 *1* 堆叠的 3×3 卷积：

$$RF = 1 + (3-1) \times 1 + (3-1) \times 1 = 5 \times 5 \tag{4.6}$$

这与单个 5×5 卷积的感受野匹配，但参数更少（$2 \times 9C^2$ *vs* $25C^2$）。

4.3.2.3 有效感受野

虽然理论感受野覆盖上述整个区域，但有效感受野（实际影响输出的区域）在感受野中心呈高斯类分布 [Luo et al. 2016]。边界附近的像素比中心像素对输出的贡献少得多。

这有重要的设计含义：

- 简单增加理论感受野不会按比例增加有效感受野
- 空洞/带孔卷积可以在不增加参数的情况下扩展有效感受野
- 相对于大卷积核的浅层网络，更偏好小卷积核的深层网络

4.3.3 1×1 卷积：跨通道混合

1×1 卷积，也称为逐点卷积，独立地对每个像素进行操作，但混合跨通道的信息。尽管简单，它是现代架构中最通用的操作之一。

数学解释。具有 C_{in} 输入通道和 C_{out} 输出通道的 1×1 卷积等效于在每个空间位置应用全连接层：

$$y_{c_{out}}[i,j] = \sum_{c_{in}=1}^{C_{in}} w_{c_{out},c_{in}} \cdot x_{c_{in}}[i,j] + b_{c_{out}} \tag{4.7}$$

这可以写为矩阵乘法：$\mathbf{Y}[i,j] = \mathbf{W} \cdot \mathbf{X}[i,j]$，其中 $\mathbf{W} \in \mathbb{R}^{C_{out} \times C_{in}}$。

1×1 卷积的关键用途

- 降维（瓶颈）：在昂贵的 3×3 卷积前将 $C_{in} \to C_{in}/4$，然后投影回来。用于 Inception、ResNet 瓶颈和 MobileNet。参数：$C_{in} \times C_{out}$（vs $K^2 \times C_{in} \times C_{out}$ 的 $K \times K$ 卷积）。
- 通道混合：启用跨通道信息流而不进行空间聚合。当需要不同卷积核的特征交互时必不可少。
- 非线性注入：每个 1×1 卷积后，激活函数（ReLU/GELU）增加表示能力。在 ResNeXt 和 MLP-Mixer 中，这些" 通道 MLP" 被堆叠。
- 增加深度而不增加空间成本：增加网络深度而不影响空间分辨率或显著增加参数。

4.3.4 转置卷积（反卷积）

对于上采样任务（分割、生成），转置卷积（有时称为反卷积）反转标准卷积的空间缩减。

给定由步长 s、填充 p 和卷积核 K 定义的卷积，转置卷积将输入尺寸 H_{in} 映射到：

$$H_{out} = (H_{in} - 1) \times s - 2p + K \tag{4.8}$$

转置卷积 $\neq$ 逆卷积

转置卷积不是卷积的数学逆。它是卷积算子的转置（因此得名）。给定 $\mathbf{Y} = \mathbf{C}\mathbf{X}$ 其中 $\mathbf{C}$ 是作为矩阵的卷积，转置卷积计算 $\mathbf{C}^\top\mathbf{Y}$。这具有相同参数但不能精确恢复 $\mathbf{X}$。

4.4 循环神经网络（RNN）

RNN 通过维护隐藏状态来处理序列数据。

4.4.1 架构

对于输入序列 $\mathbf{x}^{(1)}, \mathbf{x}^{(2)}, \ldots, \mathbf{x}^{(T)}$：

$$\mathbf{h}^{(t)} = f(\mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{b}_h) \tag{4.9}$$

$$\mathbf{y}^{(t)} = g(\mathbf{W}_{hy}\mathbf{h}^{(t)} + \mathbf{b}_y) \tag{4.10}$$

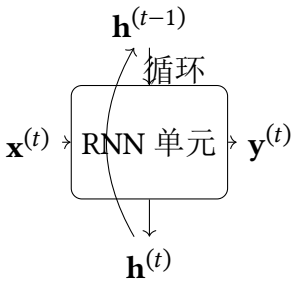


Figure 4.3 – RNN 架构

RNN 特性

- 序列处理：可以处理可变长度序列
- 共享参数：跨时间步使用相同权重
- 记忆：隐藏状态捕获过去信息
- 适用于：文本、时间序列、音频、视频

梯度消失问题

标准 RNN 在时间反向传播（BPTT）期间遭受梯度消失，使得学习长期依赖变得困难。这导致了 LSTM 和 GRU 架构的发展。

4.4.2 RNN 中的梯度消失：数学证明

理解为什么 vanilla RNN 在长序列上失败需要检查通过时间的梯度流动。我们推导出梯度消失的精确条件。

4.4.2.1 时间反向传播（BPTT）

考虑序列长度 T 上的总损失：

$$\mathscr{L} = \sum_{t=1}^{T} \ell(\mathbf{y}^{(t)}, \hat{\mathbf{y}}^{(t)}) \tag{4.11}$$

$\mathscr{L}$ 关于早期时间步 k 的 $\mathbf{h}^{(k)}$ 的梯度涉及雅可比矩阵链：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{h}^{(k)}} = \frac{\partial \mathscr{L}}{\partial \mathbf{h}^{(T)}} \prod_{t=k+1}^{T} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \tag{4.12}$$

隐藏状态转换的雅可比矩阵（带 $\mathbf{h}^{(t)} = \tanh(\mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \mathbf{W}_{xh}\mathbf{x}^{(t)} + \mathbf{b})$）为：

$$\frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} = \operatorname{diag}\left(1 - (\mathbf{h}^{(t)})^2\right) \cdot \mathbf{W}_{hh} \tag{4.13}$$

4.4.2.2 谱分析

$T - k$ 个雅可比矩阵的乘积可以通过它们的谱特性来分析：

$$\left\| \prod_{t=k+1}^{T} \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \right\| \leq \prod_{t=k+1}^{T} \left\| \frac{\partial \mathbf{h}^{(t)}}{\partial \mathbf{h}^{(t-1)}} \right\| \leq \prod_{t=k+1}^{T} \gamma_t \tag{4.14}$$

其中 $\gamma_t = \lambda_{\max}(\mathbf{W}_{hh}) \cdot \sup_i |1 - (h_i^{(t)})^2| \leq \lambda_{\max}(\mathbf{W}_{hh})$ 因为 $|1 - \tanh^2(x)| \leq 1$。

定理 4.1 (梯度消失/爆炸条件)**.** 通过时间的梯度满足：

$$\left\| \frac{\partial \mathscr{L}}{\partial \mathbf{h}^{(k)}} \right\| \approx \left\| \frac{\partial \mathscr{L}}{\partial \mathbf{h}^{(T)}} \right\| \cdot \gamma^{T-k} \tag{4.15}$$

其中 $\gamma = \lambda_{\max}(\mathbf{W}_{hh})$。因此：

- 如果 $\gamma < 1$：梯度指数消失为 $\gamma^{T-k} \to 0$
- 如果 $\gamma > 1$：梯度指数爆炸
- 如果 $\gamma = 1$（在边界上）：梯度被保留，但这是一个零测度条件

为什么这在实践中很重要

- 典型的随机初始化以高概率产生 $\gamma \ll 1$ [Bengio, Simard, and Frasconi 1994]
- 对于序列长度 $T = 100$ 和 $\gamma = 0.9$：梯度幅度 $\approx 0.9^{100} \approx 2.7 \times 10^{-5}$ —— 有效地为零
- $\mathbf{W}_{hh}$ 的特征值被 tanh 导数限制在 $[-1, 1]$，推向消失
- 这本质上是动力系统问题：RNN 是一个离散时间动力系统，其不动点要么是吸引的（$\gamma < 1$）要么是排斥的（$\gamma > 1$）

4.5 长短期记忆网络（LSTM）

LSTM [Hochreiter and Schmidhuber 1997] 用门控机制解决梯度消失问题。

4.5.1 LSTM 门

$$\mathbf{f}^{(t)} = \sigma(\mathbf{W}_f[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_f) \quad \text{(遗忘门)} \tag{4.16}$$

$$\mathbf{i}^{(t)} = \sigma(\mathbf{W}_i[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_i) \quad \text{(输入门)} \tag{4.17}$$

$$\mathbf{o}^{(t)} = \sigma(\mathbf{W}_o[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_o) \quad \text{(输出门)} \tag{4.18}$$

$$\tilde{\mathbf{c}}^{(t)} = \tanh(\mathbf{W}_c[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}] + \mathbf{b}_c) \quad \text{(细胞候选)} \tag{4.19}$$

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)} \quad \text{(细胞状态)} \tag{4.20}$$

$$\mathbf{h}^{(t)} = \mathbf{o}^{(t)} \odot \tanh(\mathbf{c}^{(t)}) \quad \text{(隐藏状态)} \tag{4.21}$$

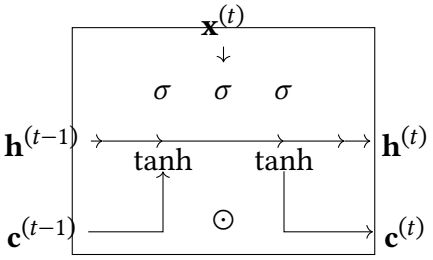


Figure 4.4 – LSTM 单元结构

LSTM 优势

- 选择性记忆：门控制记住/遗忘什么
- 长期依赖：细胞状态实现梯度流动
- 广泛使用：NLP、语音、时间序列

4.5.2 为什么 LSTM 解决梯度消失

关键洞察是细胞状态 $\mathbf{c}^{(t)}$ 提供了一条加性（而非乘性）的梯度流动路径。

4.5.2.1 通过细胞状态的梯度流动

与 vanilla RNN 中 $\mathbf{h}^{(t)} = f(\mathbf{W}_{hh}\mathbf{h}^{(t-1)} + \cdots)$ 不同，LSTM 细胞状态更新通过加法：

$$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \mathbf{i}^{(t)} \odot \tilde{\mathbf{c}}^{(t)} \tag{4.22}$$

损失关于时间 k 的细胞状态的梯度为：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{c}^{(k)}} = \frac{\partial \mathscr{L}}{\partial \mathbf{c}^{(T)}} \prod_{t=k+1}^{T} \left(\mathbf{f}^{(t)} \odot \frac{\partial \mathbf{c}^{(t)}}{\partial \mathbf{c}^{(t-1)}} + \cdots \right) \tag{4.23}$$

关键雅可比矩阵为：

$$\frac{\partial \mathbf{c}^{(t)}}{\partial \mathbf{c}^{(t-1)}} = \mathrm{diag}\left(\mathbf{f}^{(t)}\right) \tag{4.24}$$

当遗忘门 $\mathbf{f}^{(t)} \approx \mathbf{1}$ 时（网络可以学习设置为），梯度被以近恒等映射传递：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{c}^{(k)}} \approx \frac{\partial \mathscr{L}}{\partial \mathbf{c}^{(T)}} \tag{4.25}$$

直觉：每个门做什么

- 遗忘门 $\mathbf{f}^{(t)}$：“我应该擦除这个信息吗？”接近 0 的值丢弃旧内容；接近 1 的值保留它。这是解决梯度消失的关键——网络学习何时保持信息流动。
- 输入门 $\mathbf{i}^{(t)}$：“这个新输入值得写吗？”对细胞候选进行门控。防止不相关输入覆盖细胞状态。
- 输出门 $\mathbf{o}^{(t)}$：“我应该现在暴露这个信息吗？”控制细胞状态何时贡献到隐藏状态。允许细胞存储信息而不立即使用它。
- 细胞状态 $\mathbf{c}^{(t)}$：“传送带”。当遗忘门接近 1 时，信息沿这条路径基本不受阻碍地流动。这是绕过梯度消失问题的信息高速公路。

4.5.2.2 比较：RNN vs LSTM 梯度

Table 4.1 – 梯度流动：Vanilla RNN vs LSTM

方面	**Vanilla RNN**	**LSTM**
状态更新	$\mathbf{h}^{(t)} = \tanh(\mathbf{W}\mathbf{h}^{(t-1)} + \cdots)$	$\mathbf{c}^{(t)} = \mathbf{f}^{(t)} \odot \mathbf{c}^{(t-1)} + \cdots$
梯度路径	乘性	加性 + 门控
雅可比矩阵	$\mathrm{diag}(1 - \mathbf{h}^2)\mathbf{W}_{hh}$	$\mathrm{diag}(\mathbf{f}^{(t)})$
基于学习	无法控制梯度流动	网络学习 $\mathbf{f}^{(t)}$
长期（$T = 100$）	$\gamma^{100} \approx 0$	$1.0^{100} = 1.0$

4.6 门控循环单元（GRU）

GRU [Cho et al. 2014] 以更少的门简化 LSTM，同时保持性能。

4.6.1 GRU 方程

$$\mathbf{z}^{(t)} = \sigma(\mathbf{W}_z[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(更新门)} \tag{4.26}$$

$$\mathbf{r}^{(t)} = \sigma(\mathbf{W}_r[\mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \quad \text{(重置门)} \tag{4.27}$$

$$\tilde{\mathbf{h}}^{(t)} = \tanh(\mathbf{W}[\mathbf{r}^{(t)} \odot \mathbf{h}^{(t-1)}, \mathbf{x}^{(t)}]) \tag{4.28}$$

$$\mathbf{h}^{(t)} = (1 - \mathbf{z}^{(t)}) \odot \mathbf{h}^{(t-1)} + \mathbf{z}^{(t)} \odot \tilde{\mathbf{h}}^{(t)} \tag{4.29}$$

与 LSTM 的比较：

- 更少参数（2 个门 vs 3 个）
- 没有单独的细胞状态
- 通常性能与 LSTM 相当

4.7 选择架构

Table 4.2 – 架构选择指南

架构	最佳用途	示例
DNN	表格数据	分类、回归
CNN	图像、空间数据	目标检测、分割
RNN/LSTM	序列	NLP、时间序列
Transformer	长序列	翻译、生成

4.8 本章小结

1. DNN 是用于通用学习目的的全连接网络
2. CNN 通过局部连接和参数共享利用空间结构
3. 感受野随网络深度增长；有效 RF 呈高斯类分布
4. 1×1 卷积实现高效的跨通道混合和降维
5. RNN 通过循环连接处理序列数据

6. Vanilla RNN 遭受指数级梯度消失/爆炸（γ^{T-k}）
7. LSTM 通过加性细胞状态路径和可学习遗忘门解决此问题
8. GRU 以更少参数简化 LSTM，同时保持竞争性能
9. 架构选择取决于数据类型、序列长度和任务要求

4.9 扩展阅读

- [Bengio, Simard, and Frasconi 1994] — 用梯度下降学习长期依赖的困难
- [Hochreiter and Schmidhuber 1997] — 长短期记忆
- [Cho et al. 2014] — 使用 RNN 编码器-解码器学习短语表示
- [Luo et al. 2016] — 理解深度 CNN 中的有效感受野
- [He, X. Zhang, et al. 2016a] — 用于图像识别的深度残差学习

4.10 习题

1. **DNN** 参数计数：一个全连接网络有层宽度 $784 \to 256 \to 128 \to 10$。计算包括偏置在内的可训练参数总数。
2. 感受野增长：计算三个步长为 1 堆叠的 3×3 卷积的感受野。与一个 7×7 卷积在感受野和参数数量方面如何比较？
3. 1×1 卷积：证明具有 C_{in} 输入通道和 C_{out} 输出通道的 1×1 卷积等效于位置逐点的全连接层。它使用多少参数？
4. **RNN** 梯度衰减：如果循环雅可比矩阵范数被限制为 $\gamma = 0.92$，估计 50 个时间步的梯度衰减。为什么这使得长期学习困难？
5. **LSTM** 门：解释遗忘门在维护长期记忆中的作用。当 $\mathbf{f}^{(t)} \approx 0$ 和 $\mathbf{f}^{(t)} \approx 1$ 时会发生什么？
6. 架构选择：对于以下每个任务，选择首选架构并说明理由：图像分类、机器翻译、表格信用评分和多变量时间序列预测。

Chapter 5

神经网络核心操作

5.1 引言

本章涵盖现代神经网络的基本构建模块：卷积、池化和归一化操作。理解这些操作对于设计和实现有效的神经网络架构至关重要。

5.2 卷积操作

卷积是卷积神经网络（CNN）的基石。

5.2.1 2D 卷积

对于输入 $\mathbf{X} \in \mathbb{R}^{H \times W}$ 和卷积核 $\mathbf{K} \in \mathbb{R}^{K \times K}$：

$$y[i,j] = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} k[m,n] \cdot x[i+m, j+n] \tag{5.1}$$

在实践中，我们通常添加偏置并使用多个卷积核产生多个输出通道。

卷积参数

- 卷积核大小：$K \times K$（常见：3×3、5×5、7×7）
- 步长（s）：滑动窗口的步长
- 填充（p）：输入周围的零填充
- 膨胀（d）：卷积核元素之间的间距

带填充和步长的输出尺寸：

$$H_{out} = \left\lfloor \frac{H_{in} + 2p - d(K-1) - 1}{s} + 1 \right\rfloor \tag{5.2}$$

5.2.2 分组卷积和深度卷积

- 分组卷积：将输入通道分成组，分别对每组应用卷积
- 深度卷积：每个输入通道一个卷积核（分组数 $G = C_{in}$）
- 逐点卷积：1×1 卷积（用于混合通道）

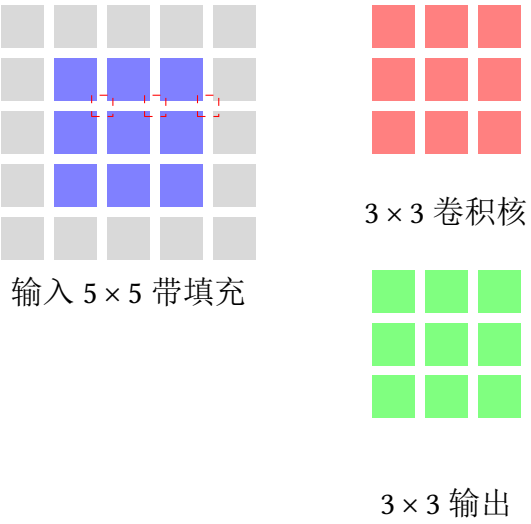


Figure 5.1 – 带填充的卷积

MobileNet 使用：深度可分离卷积 = 深度卷积 + 逐点卷积

深度可分离卷积

- 标准卷积 **FLOPs**：$H \cdot W \cdot K^2 \cdot C_{in} \cdot C_{out}$
- 可分离卷积 **FLOPs**：$H \cdot W \cdot K^2 \cdot C_{in} + H \cdot W \cdot C_{in} \cdot C_{out}$
- 加速比：大约 $\frac{1}{K^2} + \frac{1}{C_{out}}$

5.2.3 空洞（带孔）卷积

空洞卷积在卷积核元素之间插入零，在不增加参数数量或计算量的情况下扩展感受野：

$$y[i,j] = \sum_{m=0}^{K-1} \sum_{n=0}^{K-1} k[m,n] \cdot x[i + m \cdot d, j + n \cdot d] \tag{5.3}$$

其中 d 是膨胀率。

空洞卷积特性

- 带膨胀 d 的感受野：有效卷积核大小 $= d(K-1)+1$
- **FLOPs**：与标准卷积相同（只有采样模式改变）
- 参数：与标准卷积相同
- 关键应用：语义分割（DeepLab 系列），需要大感受野而不使用池化或步长

示例 **5.1** (堆叠空洞卷积). 三个膨胀率 $d = 1, 2, 4$ 的 3×3 卷积产生总感受野：

$$RF = 1 + (3-1) \times 1 + (3-1) \times 2 + (3-1) \times 4 = 15 \tag{5.4}$$

这与单个 15×15 标准卷积的感受野匹配，但只有 $3 \times 9C^2$ 参数而非 $225C^2$。这是 ***DeepLab*** *[L.-C. Chen et al. 2017]* 的核心思想。

5.3 池化操作

池化减少空间维度并提供平移不变性。

5.3.1 最大池化

$$y[i,j] = \max_{m,n \in \mathcal{N}(i,j)} x[m,n] \tag{5.5}$$

其中 $\mathcal{N}(i,j)$ 是邻域。

- 保留最强激活
- 提供局部平移不变性
- 常见：2×2 步长 2

5.3.2 平均池化

$$y[i,j] = \frac{1}{K \cdot K} \sum_{m,n \in \mathcal{N}(i,j)} x[m,n] \tag{5.6}$$

- 对邻域平均
- 比最大池化保守
- 用于全局池化

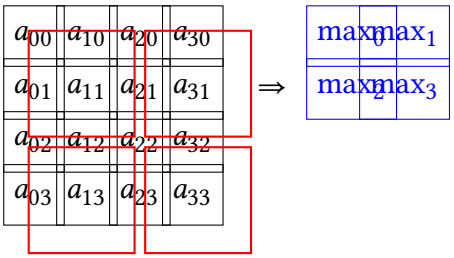


Figure 5.2 – 最大池化 2×2，步长 2

5.3.3 全局池化

应用于整个特征图以产生每个通道的单个值：

- 全局平均池化（**GAP**）：$\frac{1}{HW}\sum_{i,j} x_{ij}$
- 全局最大池化：$\max_{i,j} x_{ij}$
- 替代 **FC** 层在现代架构中

5.4 归一化技术

归一化通过控制层输入分布来稳定训练。

5.4.1 批归一化

在批次维度上归一化激活：

$$\hat{x}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \tag{5.7}$$

其中：

$$\mu_B = \frac{1}{m}\sum_{i=1}^{m} x_i \tag{5.8}$$

$$\sigma_B^2 = \frac{1}{m}\sum_{i=1}^{m}(x_i - \mu_B)^2 \tag{5.9}$$

BatchNorm 特性

- 减少内部协变量偏移
- 允许更高的学习率
- 提供轻微的正则化
- 依赖批次大小，小批次可能有问题

BatchNorm vs 推理

训练期间，BatchNorm 使用批次统计。推理期间，使用运行统计：

$$\hat{x} = \frac{x - \tilde{\mu}}{\sqrt{\tilde{\sigma}^2 + \epsilon}} \tag{5.10}$$

其中 $\tilde{\mu}$ 和 $\tilde{\sigma}^2$ 是运行均值。

5.4.2 层归一化

对单个样本内的特征进行归一化：

$$\mu = \frac{1}{D}\sum_{i=1}^{D} x_i, \quad \sigma = \frac{1}{D}\sum_{i=1}^{D}(x_i - \mu)^2 \tag{5.11}$$

- 独立于批次大小
- 用于 **RNN** 和 **Transformer**
- 训练和推理时相同

Table 5.1 – 归一化比较

方法	归一化维度	依赖批次	用例
BatchNorm	(H, W, C)	是	CNN
LayerNorm	(H, W, N)	否	Transformer
InstanceNorm	(H, W)	否	风格迁移
GroupNorm	(H, W, G)	否	小批次 CNN

5.4.3 实例归一化

独立归一化每个样本和通道：

$$y_{nchw} = \frac{x_{nchw} - \mu_{nc}}{\sqrt{\sigma_{nc}^2 + \epsilon}} \tag{5.12}$$

用于风格迁移，其中实例特定的统计信息很重要。

5.4.4 组归一化

组归一化（GN）将通道分成 G 组，分别独立地对每组进行归一化 [Wu and He 2018]。这提供了一个统一框架，将 LayerNorm 和 InstanceNorm 作为特例。

对于形状为 (N, C, H, W) 的张量，每组 C/G 个通道：

$$\mu_g = \frac{1}{(C/G) \cdot H \cdot W} \sum_{c \in \mathcal{G}_g} \sum_{h,w} x_{nchw} \tag{5.13}$$

$$\sigma_g^2 = \frac{1}{(C/G) \cdot H \cdot W} \sum_{c \in \mathcal{G}_g} \sum_{h,w} (x_{nchw} - \mu_g)^2 \tag{5.14}$$

$$\hat{x}_{nchw} = \frac{x_{nchw} - \mu_g}{\sqrt{\sigma_g^2 + \epsilon}} \tag{5.15}$$

GN 作为统一框架

- $G = 1$：GroupNorm 变为 **LayerNorm**（跨所有通道归一化）
- $G = C$：GroupNorm 变为 **InstanceNorm**（逐通道归一化）
- $G = 32$ 是大多数实现中的默认值（一个好的权衡）
- 独立于批次大小：与批次大小 1 一起工作（例如医学成像）
- 计算量与 BatchNorm 等价；只有分组改变

5.5 正则化操作

除了归一化之外，几种操作通过在训练期间引入随机性或约束来防止过拟合。

Table 5.2 – 归一化方法：统一视图

方法	统计量计算维度	依赖批次	主要用途
BatchNorm	(N, H, W) 每 C	是	CNN（大批次）
LayerNorm	(H, W, C) 每 N	否	Transformer, RNN
InstanceNorm	(H, W) 每 N, C	否	风格迁移
GroupNorm	$(H, W, C/G)$ 每 N	否	检测、小批次

5.5.1 Dropout

Dropout [Srivastava et al. 2014] 在训练期间以概率 p 随机将激活置零：

$$\tilde{x}_i = \frac{m_i \cdot x_i}{1-p}, \quad m_i \sim \text{Bernoulli}(1-p) \tag{5.16}$$

缩放因子 $1/(1-p)$（倒置 dropout）确保 $\mathbb{E}[\tilde{x}_i] = x_i$，因此推理时不需要调整。

5.5.1.1 为什么 Dropout 有效：贝叶斯解释

Dropout 可以被视为近似贝叶斯推断 [Gal and Ghahramani 2016]。每个 dropout 掩码定义一个不同的子网络，训练带 dropout 等同于：

$$p(\mathbf{y} \mid \mathbf{x}, \mathcal{D}) \approx \frac{1}{T} \sum_{t=1}^{T} p(\mathbf{y} \mid \mathbf{x}, \mathbf{W} \odot \mathbf{m}^{(t)}) \tag{5.17}$$

这是在指数数量的稀疏网络（N 个单元为 2^N）上进行模型平均，通过方差减少提高泛化。

5.5.1.2 Dropout 变体

Table 5.3 – Dropout 变体

变体	操作	用例
标准 Dropout	以概率 p 将激活置零	FC 层
DropConnect	以概率 p 将权重置零	FC 层
Spatial Dropout	将整个特征图置零	CNN
DropPath	跳过整个路径/块	深度网络
DropBlock	丢弃连续区域	CNN（正则化）

DropPath（随机深度）[Huang, Y. Sun, et al. 2016] 在训练期间随机跳过整

个残差块。对于输入 $\mathbf{x}$ 和输出 $\mathscr{F}(\mathbf{x})$ 的块：

$$\tilde{\mathbf{y}} = \begin{cases} \mathbf{x} & \text{以概率} p_L \\ \mathscr{F}(\mathbf{x}) + \mathbf{x} & \text{以概率} 1 - p_L \end{cases} \tag{5.18}$$

丢弃率 p_L 通常随深度线性增加，创建在训练期间作为集成起作用的更短有效网络。

5.6 组合操作

现代网络将这些操作组合成标准化的块。

5.6.1 基本卷积块

典型的卷积块：

1. 卷积
2. 批归一化
3. 激活（ReLU/GELU）

常见模式：

- Conv-BN-ReLU（VGG 风格）
- BN-ReLU-Conv（ResNet 风格，预激活）

代码 5.1 – PyTorch 中的卷积块

```python
import torch.nn as nn

class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, kernel_size=3, stride=1):
        super().__init__()
        padding = kernel_size // 2
        self.block = nn.Sequential(
            nn.Conv2d(in_ch, out_ch, kernel_size, stride, padding, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True)
        )

    def forward(self, x):
        return self.block(x)

```

```python
# 预激活块（ResNet风格）
class PreActBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
        super().__init__()
        self.block = nn.Sequential(
            nn.BatchNorm2d(in_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False),
            nn.BatchNorm2d(out_ch),
            nn.ReLU(inplace=True),
            nn.Conv2d(out_ch, out_ch, 3, 1, 1, bias=False)
        )
        self.shortcut = nn.Identity() if stride==1 and in_ch==out_ch else \
                        nn.Conv2d(in_ch, out_ch, 1, stride, bias=False)

    def forward(self, x):
        return self.block(x) + self.shortcut(x)
```

5.7 本章小结

1. 卷积用共享权重提取局部特征
2. 深度可分离卷积为移动部署减少计算
3. 空洞卷积在不增加额外参数的情况下扩展感受野
4. 池化提供空间缩减和不变性
5. 归一化稳定训练动态（CNN 用 BN，Transformer 用 LN，小批次用 GN）
6. Dropout 及其变体通过随机深度提供正则化
7. 现代架构将这些组合成标准化的块
8. 操作的选择取决于任务和数据特性

5.8 扩展阅读

- [Srivastava et al. 2014] — Dropout：防止神经网络过拟合的简单方法
- [Ioffe and Szegedy 2015] — 批归一化：通过减少内部协变量偏移加速深度网络训练
- [J. L. Ba, Kiros, and G. E. Hinton 2016] — 层归一化
- [Gal and Ghahramani 2016] — Dropout 作为贝叶斯近似
- [Huang, Y. Sun, et al. 2016] — 带随机深度的深度网络

- [L.-C. Chen et al. 2017] — 重新思考用于语义图像分割的空洞卷积
- [Howard et al. 2017] — MobileNets：用于移动视觉应用的高效 CNN
- [Wu and He 2018] — 组归一化

5.9 习题

1. 卷积输出尺寸：对于输入尺寸 32×32、卷积核大小 5、填充 2、步长 1 和膨胀 1，计算输出高度和宽度。对步长 2 重复计算。
2. 深度可分离 **FLOPs**：对于输入形状 $56 \times 56 \times 64$ 和输出通道 128，比较标准 3×3 卷积与深度可分离卷积的 FLOPs。
3. 空洞感受野：三个 3×3 卷积使用膨胀率 $1, 2, 4$。推导有效感受野并解释为什么这在语义分割中有用。
4. 池化选择：比较包含一个强激活和许多弱激活的特征图上的最大池化和平均池化。哪种操作更好地保留显著的局部证据？
5. 归一化策略：你的 GPU 内存只允许检测模型的批次大小为 2。你会选择 BatchNorm、LayerNorm 还是 GroupNorm？解释权衡。
6. **Dropout** 实践：为什么通常在推理时禁用 dropout？推导为什么倒置 dropout 在训练期间保持预期激活不变。

Chapter 6

跳跃连接与注意力机制

6.1 引言

深度网络面临两个根本性挑战：(1) 梯度在传播过程中会消失或爆炸；(2) 模型需要能够选择性地关注输入中最相关的部分。跳跃连接通过提供梯度高速公路来解决第一个挑战，而注意力机制通过实现动态的、依赖于输入的加权来解决第二个挑战 Together, these techniques form the backbone of modern architectures from ResNet to the Transformer.

6.2 跳跃连接

跳跃连接（shortcuts）绕过层，从而实现深度网络中的梯度流动。

6.2.1 残差连接（ResNet）

残差学习框架 [He, X. Zhang, et al. 2016a]：

$$\mathbf{y} = \mathscr{F}(\mathbf{x}) + \mathbf{x} \tag{6.1}$$

其中 $\mathscr{F}$ 是学习的残差映射。

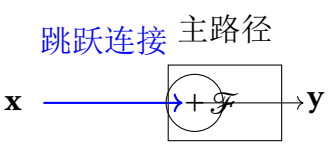


Figure 6.1 – 残差连接

如果维度匹配：$\mathbf{y} = \mathscr{F}(\mathbf{x}, \{W_i\}) + \mathbf{x}$
如果维度不匹配：$\mathbf{y} = \mathscr{F}(\mathbf{x}, \{W_i\}) + W_s\mathbf{x}$

6.2.2 为什么残差连接有效：理论分析

6.2.2.1 梯度流视角

考虑一个具有 L 个残差块的网络。损失 $\mathscr{L}$ 关于中间激活 $\mathbf{x}_l$ 的梯度为：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{x}_l} = \frac{\partial \mathscr{L}}{\partial \mathbf{x}_L}\left(1 + \frac{\partial}{\partial \mathbf{x}_l}\sum_{i=l}^{L-1}\mathscr{F}(\mathbf{x}_i, W_i)\right) \tag{6.2}$$

关键洞察：梯度包含一个恒等项（“1”），它被加到残差梯度上。即使残差梯度 $\frac{\partial \mathscr{F}}{\partial \mathbf{x}}$ 消失，恒等项也能确保信息直接传播：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{x}_l} \geq \frac{\partial \mathscr{L}}{\partial \mathbf{x}_L} \cdot 1 \tag{6.3}$$

这与普通网络形成鲜明对比，后者涉及 $L - l$ 个雅可比矩阵的乘积：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{x}_l}^{\text{plain}} = \frac{\partial \mathscr{L}}{\partial \mathbf{x}_L}\prod_{i=l}^{L-1}\frac{\partial \mathbf{x}_{i+1}}{\partial \mathbf{x}_i} \tag{6.4}$$

每个绝对值小于 1 的因子都会导致指数级衰减。

6.2.2.2 恒等映射假设

如果一层的最优函数接近恒等映射（对于深度网络通常如此），那么学习残差 $\mathscr{F}(\mathbf{x}) = \mathscr{H}(\mathbf{x}) - \mathbf{x}$ 比从头学习 $\mathscr{H}(\mathbf{x})$ 更容易。网络只需要将残差推向零，而不是通过非线性层学习近恒等变换。

ResNet 的优势

- 使极深网络（100+ 层）的训练成为可能
- 通过恒等快捷方式缓解梯度消失/爆炸问题
- 减少退化问题（更深 ≠ 更高训练误差）
- 易于实现，计算开销最小
- 理论保证：普通网络可学习的任何函数，其残差版本也可学习

6.2.3 预激活 ResNet

文献 [He, X. Zhang, et al. 2016b] 中的预激活设计将 BatchNorm 和 ReLU 移到卷积层之前而非之后：

$$\text{后激活：} \quad \mathbf{y} = \text{ReLU}(\text{BN}(\mathscr{F}(\mathbf{x}) + \mathbf{x})) \tag{6.5}$$

$$\text{预激活：} \quad \mathbf{y} = \mathscr{F}(\text{ReLU}(\text{BN}(\mathbf{x}))) + \mathbf{x} \tag{6.6}$$

预激活提供了更清晰的信号传播路径：恒等快捷路径完全是 “干净” 的（路径中没有非线性或 BN），这改善了梯度流动。

6.2.4 瓶颈架构

用于更深的 ResNet 以降低复杂度：

- 1×1 卷积降维
- 3×3 卷积（瓶颈）
- 1×1 卷积恢复维度

对于输入 C，瓶颈输出 C：

1. Conv $1 \times 1, C \to C//4$
2. Conv $3 \times 3, C//4 \to C//4$
3. Conv $1 \times 1, C//4 \to C$

6.2.5 ResNeXt：残差块中的分组卷积

ResNeXt [Xie et al. 2017] 将分组卷积引入残差块。不是使用单个宽卷积，而是使用 G 条并行的 “基数” 路径，每条路径宽度为 d：

$$\mathscr{F}(\mathbf{x}) = \sum_{i=1}^{G} \mathscr{T}_i(\mathbf{x}) \tag{6.7}$$

其中每个 $\mathscr{T}_i$ 是具有 C/G 个通道的变换。

ResNeXt 与 ResNet 的对比

- 相同的 **FLOPs** 和参数，但精度更高
- 增加 G 可提高精度，即使总宽度固定
- 关键在于基数（路径数量）而非仅深度或宽度
- $G = 32$，宽度 $d = 4$ 是常见配置

6.2.6 随机深度（DropPath）

随机深度 [Huang, Y. Sun, et al. 2016] 在训练期间随机丢弃整个残差块。对于线性增加的丢弃率 $p_l = 1 - \frac{l}{L}(1 - p_L)$：

$$\mathbf{y}_l = \begin{cases} \mathbf{x}_l & \text{概率} p_l \text{（跳过块）} \\ \mathscr{F}_l(\mathbf{x}_l) + \mathbf{x}_l & \text{概率} 1 - p_l \text{（保留块）} \end{cases} \tag{6.8}$$

这通过训练不同深度网络集合来提供正则化，无需额外参数。它现在是 EfficientNet 和 Vision Transformer 等现代架构的标准组件。

6.2.7 密集连接（DenseNet）

每一层接收所有前面层的特征图 [Huang, Zhuang Liu, et al. 2017]：

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \dots, \mathbf{x}_{l-1}]) \tag{6.9}$$

DenseNet 的特性

- 特征复用：每一层可访问所有前面的特征
- 强梯度流：从任意层到损失的直接连接
- 参数效率：在相当精度下比 ResNet 参数更少
- 内存成本：由于拼接导致更高（通过小增长率 k 的密集连接来缓解）
- **DenseNet** 块中的总参数：$\mathcal{O}(k \times k \times (l-1) \times K^2)$，其中 k 是增长率

6.3 注意力机制

注意力允许模型关注输入的相关部分。

6.3.1 自注意力（缩放点积）

缩放点积注意力函数 [Vaswani et al. 2017]：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V} \tag{6.10}$$

其中：

- $\mathbf{Q} = \mathbf{X}\mathbf{W}_Q$（查询），$\mathbf{Q} \in \mathbb{R}^{n \times d_k}$
- $\mathbf{K} = \mathbf{X}\mathbf{W}_K$（键），$\mathbf{K} \in \mathbb{R}^{n \times d_k}$

- $\mathbf{V} = \mathbf{X}\mathbf{W}_V$（值），$\mathbf{V} \in \mathbb{R}^{n \times d_v}$

- $\mathbf{X} \in \mathbb{R}^{n \times d_{\text{model}}}$ 是输入序列

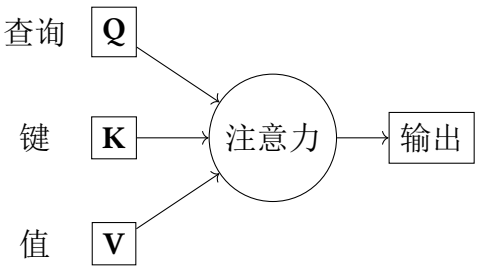


Figure 6.2 – 自注意力机制

6.3.1.1 为什么要按 $\sqrt{d_k}$ 缩放？

如果不缩放，当 d_k 很大时，点积 $\mathbf{q}_i \cdot \mathbf{k}_j = \sum_{m=1}^{d_k} q_{im} k_{jm}$ 的幅度会增长（单位方差随机向量的方差为 d_k）。这会将 softmax 推到梯度极小的区域：

$$\text{如果} d_k = 64: \quad \text{softmax}\begin{pmatrix}8.0\\8.1\end{pmatrix} \approx \begin{pmatrix}0.475\\0.525\end{pmatrix}, \quad \text{softmax}\begin{pmatrix}800\\810\end{pmatrix} \approx \begin{pmatrix}0.000\\1.000\end{pmatrix} \tag{6.11}$$

按 $\sqrt{d_k}$ 缩放可确保点积具有单位方差，使 softmax 保持在其 “敏感” 区域。

6.3.1.2 计算复杂度

对于序列长度 n、模型维度 d：

$$\text{Q、K、V 投影：} \quad \mathcal{O}(n \cdot d^2) \tag{6.12}$$

$$\text{注意力分数} \mathbf{Q}\mathbf{K}^\top: \quad \mathcal{O}(n^2 \cdot d) \tag{6.13}$$

$$\text{输出 softmax}(\cdot)\mathbf{V}: \quad \mathcal{O}(n^2 \cdot d) \tag{6.14}$$

$$\text{总计：} \quad \mathcal{O}(n^2 \cdot d + n \cdot d^2) \tag{6.15}$$

当 $n > d$ 时，主导项是 $\mathcal{O}(n^2 \cdot d)$，使得自注意力的序列长度呈二次方——这是长序列的关键瓶颈。

自注意力的特性

- 全局感受野：在一层中每个 token 都可以关注所有其他 token
- 计算复杂度 $O(n^2 d)$：序列长度的二次方
- 可解释：注意力权重显示成对相关性
- 排列等变：自注意力默认是顺序不变的（位置编码添加顺序信息）

6.3.2 多头注意力

多头注意力（MHA）不是使用单一注意力函数，而是并行运行 h 个具有不同学习投影的注意力函数 [Vaswani et al. 2017]：

$$\text{head}_i = \text{Attention}(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V) \tag{6.16}$$

$$\text{MHA}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}(\text{head}_1, \dots, \text{head}_h)\mathbf{W}^O \tag{6.17}$$

其中每个 $\mathbf{W}_i^Q, \mathbf{W}_i^K \in \mathbb{R}^{d \times d_k}$，$\mathbf{W}_i^V \in \mathbb{R}^{d \times d_v}$，$\mathbf{W}^O \in \mathbb{R}^{hd_v \times d}$。通常 $d_k = d_v = d/h$。

为什么要用多个头？

- 多样化表示：每个头可以学习不同的注意力模式（句法、语义、位置）
- 相同总成本：$h \times \mathcal{O}(n^2 \cdot d_k) = \mathcal{O}(n^2 \cdot d)$ ——与单头相同
- 线性投影：输出投影 $\mathbf{W}^O$ 混合来自所有头的信息
- 经验：$h = 8$ 或 $h = 16$ 是标准；在 ~32 以上收益递减

6.3.3 交叉注意力

交叉注意力允许模型关注与正在生成的序列不同的序列。查询来自一个序列，而键和值来自另一个序列：

$$\text{Cross-Attention}(\mathbf{Q}_{\text{target}}, \mathbf{K}_{\text{source}}, \mathbf{V}_{\text{source}}) \tag{6.18}$$

这是编码器-解码器架构（如机器翻译）的核心机制：

- 编码器：通过自注意力处理源序列
- 解码器：使用交叉注意力查询编码器的表示
- 解码器 “查找” 每个生成目标 token 的相关源 token

6.3.4 位置编码

由于自注意力是排列等变的，我们必须显式注入位置信息。

6.3.4.1 正弦位置编码

原始 Transformer 使用固定正弦编码 [Vaswani et al. 2017]：

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d}}\right) \tag{6.19}$$

$$\mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d}}\right) \tag{6.20}$$

正弦 PE 的特性

- 每个维度对应不同波长（从 2π 到 $10000 \cdot 2\pi$）的正弦波
- 允许模型学习相对位置：PE_{pos+k} 可以表示为 PE_{pos} 的线性函数
- 无需学习参数——适用于各种序列长度
- 在外推到比训练中看到的更长序列时有限制

6.3.4.2 现代替代方案

Table 6.1 – 位置编码方法对比

方法	机制	关键特性
可学习 PE	可训练嵌入	灵活，受限于最大长度
RoPE	旋转 Q、K 角度	通过点积实现相对位置
ALiBi	在分数上添加线性偏置	外推到未见过的长度
T5 Bias	在注意力分数上添加偏置	简单有效

旋转位置嵌入（**RoPE**）[Su et al. 2021] 通过旋转查询和键向量来编码位置：

$$\langle \mathbf{q}_m, \mathbf{k}_n \rangle_{\mathrm{RoPE}} = \langle R_m \mathbf{q}, R_n \mathbf{k} \rangle = f(\mathbf{q}, \mathbf{k}, n-m) \tag{6.21}$$

这自然地将相对位置 $n-m$ 编码到注意力分数中。RoPE 是 LLaMA 和许多现代 LLM 的默认选择。

ALiBi [Press, N. A. Smith, and Lewis 2022] 根据相对距离在注意力分数上添

加固定线性偏置：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}} + \mathbf{m} \cdot [-n + m]\right)\mathbf{V} \tag{6.22}$$

其中 $\mathbf{m}$ 是一个非学习斜率向量。ALiBi 的关键优势是长度外推：在短序列上训练的模型可以在推理时处理更长的序列。

6.3.5 压缩激励（SE）注意力

通道注意力机制 [J. Hu, Shen, and G. Sun 2018]：

1. 压缩：全局平均池化 $\mathbf{z} = \frac{1}{H \times W} \sum_{i,j} \mathbf{X}_{:,i,j}$
2. 激励： FC-ReLU-FC-Sigmoid $\mathbf{s} = \sigma(W_2 \delta(W_1 \mathbf{z}))$
3. 缩放： $\tilde{\mathbf{X}} = \mathbf{s} \cdot \mathbf{X}$

SE 块

- 建模通道依赖关系
- 轻量：压缩比 $r = 16$ 意味着仅增加 $2C^2/r$ 个额外参数
- 即插即用：可添加到任何现有架构
- 在 **ImageNet** 上实现超过 **1%** 的 **Top-1** 改进，开销可忽略

6.3.6 CBAM：卷积块注意力模块

CBAM [Woo et al. 2018] 将通道注意力和空间注意力按顺序组合：
通道注意力模块（**CAM**）：

$$\mathbf{M}_c(\mathbf{X}) = \sigma\big(\text{MLP}(\text{AvgPool}(\mathbf{X})) + \text{MLP}(\text{MaxPool}(\mathbf{X}))\big) \tag{6.23}$$

空间注意力模块（**SAM**）：

$$\mathbf{M}_s(\mathbf{X}') = \sigma\big(f^{7\times7}([\text{AvgPool}(\mathbf{X}'); \text{MaxPool}(\mathbf{X}')])\big) \tag{6.24}$$

完整过程：$\mathbf{X}'' = \mathbf{M}_s(\mathbf{M}_c(\mathbf{X}) \odot \mathbf{X}) \odot \mathbf{X}$。

通过同时关注什么（通道）和哪里（空间位置），CBAM 捕获比仅通道 SE 更丰富的注意力模式。

6.3.7 窗口注意力（Swin Transformer）

为了解决全局自注意力的 $O(n^2)$ 复杂度问题，Swin Transformer [Ze Liu et al. 2021] 将注意力限制在局部窗口内：

$$\text{WindowAttention}(\mathbf{x}) = \text{softmax}\left(\frac{\mathbf{Q}_w\mathbf{K}_w^\top}{\sqrt{d}} + \mathbf{B}\right)\mathbf{V}_w \tag{6.25}$$

其中 $\mathbf{B}$ 是相对位置偏置（可学习），下标 w 表示在大小为 $M \times M$ 的窗口内计算。

窗口注意力 vs 全局注意力

- 复杂度：$O(n \cdot M^2 \cdot d)$ vs $O(n^2 \cdot d)$ ——当 M 固定时对图像大小是线性的
- 跨窗口连接：交替层中的移位窗口划分使信息能在窗口间流动
- 先局部：低层特征本质上是局部的；窗口注意力是视觉的自然归纳偏置

6.3.8 高效注意力变体

Table 6.2 – 注意力复杂度对比

方法	复杂度	全局？	参考文献
标准 MHA	$O(n^2d)$	是	Vaswani 2017
Linformer	$O(nkd)$	近似	Wang et al. 2020
Linear Attn	$O(nd^2)$	近似	Katharopoulos 2020
Performer	$O(nd)$	近似	Choromanski 2021
Swin（窗口）	$O(nM^2d)$	移位	Liu et al. 2021
Flash Attention	$O(n^2d)$	是	Dao et al. 2022

代码 6.1 – PyTorch 多头自注意力实现

```python
import torch
import torch.nn as nn
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, n_heads, dropout=0.1):
        super().__init__()
        self.d_model = d_model
```

```
        self.n_heads = n_heads
        self.d_k = d_model // n_heads

        self.W_q = nn.Linear(d_model, d_model)
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)
        self.dropout = nn.Dropout(dropout)

    def forward(self, x, mask=None):
        B, N, D = x.shape

        # 投影并重塑: (B, N, D) -> (B, H, N, d_k)
        Q = self.W_q(x).view(B, N, self.n_heads, self.d_k).transpose(1, 2)
        K = self.W_k(x).view(B, N, self.n_heads, self.d_k).transpose(1, 2)
        V = self.W_v(x).view(B, N, self.n_heads, self.d_k).transpose(1, 2)

        # 缩放点积注意力
        scores = Q @ K.transpose(-2, -1) / math.sqrt(self.d_k)
        if mask is not None:
            scores = scores.masked_fill(mask == 0, float('-inf'))
        attn = torch.softmax(scores, dim=-1)
        attn = self.dropout(attn)

        # 拼接多头
        out = (attn @ V).transpose(1, 2).contiguous().view(B, N, D)
        return self.W_o(out)
```

6.4 本章小结

1. 跳跃连接通过提供梯度高速公路使深度网络成为可能
2. 残差连接添加恒等快捷方式；梯度始终有一条幅度 ≥ 1 的路径
3. 预激活 ResNet 保持快捷路径 “干净” 以获得更好的梯度流
4. ResNeXt 表明基数（分组路径）比宽度更有效
5. 随机深度通过训练子网络集合提供正则化
6. 自注意力以 $O(n^2 d)$ 时间计算成对交互
7. 多头注意力以相同的计算成本学习多样化的注意力模式
8. 交叉注意力连接序列到序列模型中的编码器和解码器
9. 位置编码（正弦、RoPE、ALiBi）注入顺序信息

10. 通道注意力（SE、CBAM）和窗口注意力（Swin）使注意力适应特定领域
11. 高效注意力变体旨在降低二次复杂度

6.5 扩展阅读

- [He, X. Zhang, et al. 2016a] ——图像识别的深度残差学习
- [Huang, Y. Sun, et al. 2016] ——带随机深度的深度网络
- [He, X. Zhang, et al. 2016b] ——深度残差网络中的恒等映射
- [Xie et al. 2017] ——用于深度神经网络的聚合残差变换
- [Huang, Zhuang Liu, et al. 2017] ——密集连接的卷积网络
- [Vaswani et al. 2017] ——注意力就是你所需要的一切
- [J. Hu, Shen, and G. Sun 2018] ——压缩激励网络
- [Woo et al. 2018] ——CBAM：卷积块注意力模块
- [Ze Liu et al. 2021] ——Swin Transformer：使用移位窗口的分层视觉 Transformer
- [Su et al. 2021] ——RoFormer：带旋转位置嵌入的增强型 Transformer
- [Press, N. A. Smith, and Lewis 2022] ——短训练、长测试：线性偏置注意力实现输入长度外推

6.6 习题

1. 残差梯度流：从 $\mathbf{y} = \mathscr{F}(\mathbf{x}) + \mathbf{x}$ 开始，推导 $\frac{\partial \mathscr{L}}{\partial \mathbf{x}}$ 并解释恒等项如何帮助深度优化。
2. 基数 **vs** 宽度：在 ResNeXt 中，当保持总 FLOPs 大致固定时增加基数会发生什么变化？为什么多个分组路径可以胜过简单地加宽单一路径？
3. 密集连接：一个 DenseNet 块有 4 层，增长率 $k = 32$。如果输入有 64 个通道，块中最后一层之前有多少个通道？
4. 注意力复杂度：比较全局自注意力与 Swin 窗口注意力对于序列长度 n 和窗口大小 M^2 的时间复杂度。何时窗口注意力更有吸引力？
5. 位置编码：解释 RoPE 与正弦位置编码有何不同。为什么 RoPE 在现代大语言模型中更受青睐？
6. 通道 **vs** 空间注意力：比较 SE 和 CBAM。哪个模块可以显式关注空间位置，为什么这对于检测或分割任务很重要？

Chapter 7

优化器与学习率调度器

7.1 梯度下降优化

训练神经网络通过在减少损失的方向上更新参数来最小化损失函数。

7.1.1 批量梯度下降

使用整个数据集的梯度更新参数：

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathscr{L}(\theta_t) \tag{7.1}$$

- 优点：收敛稳定，梯度精确
- 缺点：大数据集上慢，内存占用大

7.1.2 随机梯度下降（SGD）

使用单个样本的梯度更新参数：

$$\theta_{t+1} = \theta_t - \alpha \nabla_\theta \mathscr{L}(\theta_t; \mathbf{x}^{(i_t)}, \mathbf{y}^{(i_t)}) \tag{7.2}$$

- 优点：快，能跳出局部最小值，可在线学习
- 缺点：梯度噪声大，需要较小的学习率

7.1.3 小批量梯度下降

使用 B 个样本的批次更新：

$$\theta_{t+1} = \theta_t - \alpha \frac{1}{B} \sum_{i=1}^{B} \nabla_\theta \mathscr{L}(\theta_t; \mathbf{x}^{(i)}, \mathbf{y}^{(i)}) \tag{7.3}$$

这是深度学习中的标准方法。

7.2 带动量的梯度下降

动量通过累积速度来加速收敛：

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t + (1 - \beta) \nabla_\theta \mathscr{L}(\theta_t) \tag{7.4}$$

$$\theta_{t+1} = \theta_t - \alpha \mathbf{v}_{t+1} \tag{7.5}$$

- $\beta \in [0, 1)$ 是动量系数（通常为 0.9）
- 减少陡峭方向的振荡
- 加速平缓方向的收敛

Nesterov 加速梯度

Nesterov 动量向前看：

$$\mathbf{v}_{t+1} = \beta \mathbf{v}_t - \alpha \nabla_\theta \mathscr{L}(\theta_t + \beta \mathbf{v}_t) \tag{7.6}$$

$$\theta_{t+1} = \theta_t + \mathbf{v}_{t+1} \tag{7.7}$$

这提供了更好的收敛保证，通常训练更快。

7.3 自适应学习率方法

这些方法根据梯度历史调整每个参数的学习率。

7.3.1 AdaGrad

学习率与累积平方梯度成反比地调整：

$$\mathbf{g}_t = \nabla_\theta \mathscr{L}(\theta_t) \tag{7.8}$$

$$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t \odot \mathbf{g}_t \tag{7.9}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\mathbf{r}_t + \epsilon}} \odot \mathbf{g}_t \tag{7.10}$$

- 适合稀疏特征
- 学习率随时间递减
- 在深度学习中可能过早停止

7.3.2 RMSProp

使用平方梯度的指数加权移动平均：

$$\mathbf{g}_t = \nabla_\theta \mathscr{L}(\theta_t) \tag{7.11}$$

$$\mathbf{r}_t = \beta \mathbf{r}_{t-1} + (1 - \beta) \mathbf{g}_t \odot \mathbf{g}_t \tag{7.12}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha}{\sqrt{\mathbf{r}_t + \epsilon}} \odot \mathbf{g}_t \tag{7.13}$$

- 解决了 **AdaGrad** 学习率递减的问题

- **RNN** 的常用默认值
- $\beta = 0.9$ 是典型值

7.3.3 Adam（自适应矩估计）

Adam [Kingma and J. Ba 2015] 结合了动量和 RMSProp：

$$\mathbf{m}_t = \beta_1 \mathbf{m}_{t-1} + (1-\beta_1)\mathbf{g}_t \quad \text{（一阶矩）} \tag{7.14}$$

$$\mathbf{v}_t = \beta_2 \mathbf{v}_{t-1} + (1-\beta_2)\mathbf{g}_t^2 \quad \text{（二阶矩）} \tag{7.15}$$

$$\hat{\mathbf{m}}_t = \frac{\mathbf{m}_t}{1-\beta_1^t} \quad \text{（偏差校正）} \tag{7.16}$$

$$\hat{\mathbf{v}}_t = \frac{\mathbf{v}_t}{1-\beta_2^t} \tag{7.17}$$

$$\theta_{t+1} = \theta_t - \frac{\alpha \hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \tag{7.18}$$

Adam 的特性

- 大多数深度学习任务的默认优化器
- 对超参数鲁棒
- 默认参数： $\alpha = 0.001, \beta_1 = 0.9, \beta_2 = 0.999, \epsilon = 10^{-8}$
- 早期时间步的偏差校正

代码 7.1 – PyTorch 中的优化器

```python
import torch.optim as optim

# 带动量的 SGD
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9)

# 带 Nesterov 动量的 SGD
optimizer = optim.SGD(model.parameters(), lr=0.01, momentum=0.9, nesterov=True
    )

# Adam
optimizer = optim.Adam(model.parameters(), lr=0.001, betas=(0.9, 0.999))

# AdamW（解耦权重衰减）
optimizer = optim.AdamW(model.parameters(), lr=0.001, weight_decay=0.01)

# RMSProp
optimizer = optim.RMSprop(model.parameters(), lr=0.01, alpha=0.99)
```

7.3.4 AdamW（带权重衰减的 Adam）

AdamW [Loshchilov and Hutter 2019a] 将权重衰减与梯度更新解耦：

$$\theta_{t+1} = \theta_t - \alpha \frac{\mathbf{m}_t}{\sqrt{\mathbf{v}_t} + \epsilon} - \alpha\lambda\theta_t \tag{7.19}$$

- 与原始 **Adam** 不同的正确 **L2** 正则化
- 在许多任务中泛化更好
- **Transformer** 的标准配置

7.3.5 AdaDelta

AdaDelta [Zeiler 2012] 通过将累积过去梯度的窗口限制为固定大小 w 来解决 AdaGrad 单调递减学习率的问题，使用指数衰减平均：

$$\mathbf{r}_t = \rho\mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t^2 \tag{7.20}$$

$$\Delta\theta_t = -\frac{\sqrt{\mathbf{s}_{t-1} + \epsilon}}{\sqrt{\mathbf{r}_t + \epsilon}}\mathbf{g}_t \tag{7.21}$$

$$\mathbf{s}_t = \rho\mathbf{s}_{t-1} + (1-\rho)(\Delta\theta_t)^2 \tag{7.22}$$

其中 ρ 是衰减常数（通常为 0.95）。

AdaDelta 的特性

- 无需手动设置学习率：分子 $\sqrt{\mathbf{s}_{t-1} + \epsilon}$ 替代了 α，使该方法基本无超参数
- 对不同梯度尺度鲁棒：使用累积更新自适应调整每个参数
- 现在很少使用：被 Adam 及其变体取代

7.3.6 自适应方法的统一视角

7.3.7 收敛速率分析

定理 7.1 (SGD 收敛速率). 对于平滑（*L-Lipschitz* 梯度）和 μ-强凸函数 f，学习率 $\alpha = 1/L$ 的梯度下降满足：

$$f(\theta_t) - f(\theta^*) \leq \left(1 - \frac{\mu}{L}\right)^t \left(f(\theta_0) - f(\theta^*)\right) \tag{7.23}$$

这是速率 $1 - \mu/L$ 的线性收敛速率。

Table 7.1 – 自适应优化器家族

方法	一阶矩	二阶矩	**LR** 缩放
AdaGrad	无	$\mathbf{r}_t = \mathbf{r}_{t-1} + \mathbf{g}_t^2$	$\alpha/\sqrt{\mathbf{r}_t}$
RMSProp	无	$\mathbf{r}_t = \rho\mathbf{r}_{t-1} + (1-\rho)\mathbf{g}_t^2$	$\alpha/\sqrt{\mathbf{r}_t}$
AdaDelta	无	与 RMSProp 相同	$\sqrt{\mathbf{s}}/\sqrt{\mathbf{r}_t}$
Adam	$\mathbf{m}_t$（偏差校正）	$\mathbf{v}_t$（偏差校正）	$\alpha\hat{\mathbf{m}}_t/\sqrt{\hat{\mathbf{v}}_t}$
AdamW	与 Adam 相同	与 Adam 相同	+ 解耦 $\lambda\theta_t$

不同优化器的收敛速率

- 凸 + 平滑： SGD 达到 $O(1/T)$ 速率；加速方法达到 $O(1/T^2)$
- 非凸：所有方法收敛到 $O(1/\sqrt{T})$ 的驻点
- **Adam** 在非凸设置下不能保证收敛 [Reddi, Kale, and Kumar 2018]；**AdamW** 和 **AMSGrad** 修复了这个问题
- 在实践中，Adam 和 AdamW 初始收敛更快，但 SGD+ 动量通常能达到更好的最终泛化

7.4 梯度裁剪

防止梯度爆炸。两种主要策略：

7.4.1 值裁剪

逐个裁剪每个梯度分量：

$$\tilde{g}_i = \text{clip}(g_i, -c, c) = \max(\min(g_i, c), -c) \tag{7.24}$$

其中 c 是裁剪值（通常为 0.5–5.0）。

7.4.2 范数裁剪

如果整个梯度向量的范数超过阈值，则按比例缩放：

$$\tilde{\mathbf{g}} = \begin{cases} \mathbf{g} & \text{如果} \|\mathbf{g}\| \le c \\ \frac{c}{\|\mathbf{g}\|}\mathbf{g} & \text{如果} \|\mathbf{g}\| > c \end{cases} \tag{7.25}$$

这保留了梯度的方向，同时限制其幅度。

何时使用梯度裁剪

- **RNN 和 Transformer**：对于防止长序列中的梯度爆炸必不可少
- 微调预训练模型：防止大梯度更新造成的失稳
- 范数裁剪通常优于值裁剪（保留梯度方向）
- 典型最大范数：1.0（Transformer）到 5.0（RNN）
- PyTorch 中：`torch.nn.utils.clip_grad_norm_`(model.parameters(), max_norm=1.0)

7.5 学习率调度器

学习率调度器在训练过程中调整学习率。

7.5.1 步进衰减

$$\alpha_t = \alpha_0 \cdot \gamma^{\lfloor t/T \rfloor} \tag{7.26}$$

其中 γ 是衰减因子（如 0.5），T 是周期。

7.5.2 指数衰减

$$\alpha_t = \alpha_0 \cdot e^{-\lambda t} \tag{7.27}$$

7.5.3 余弦退火

$$\alpha_t = \alpha_{min} + \frac{1}{2}(\alpha_{max} - \alpha_{min})\left(1 + \cos\left(\frac{t\pi}{T}\right)\right) \tag{7.28}$$

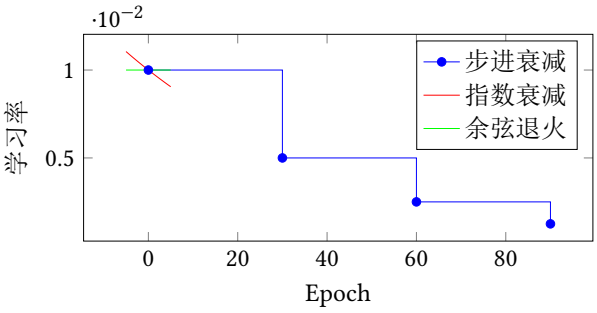


Figure 7.1 – 学习率调度器

7.5.4 预热

在开始时逐渐增加学习率：

$$\alpha_t = \alpha_{max} \cdot \frac{t}{T_{warmup}} \quad \text{当} t < T_{warmup} \tag{7.29}$$

- 防止开始时的大梯度
- 对 **Transformer** 很重要
- 常见：**500-10000** 预热步数

7.5.5 预热 + 余弦衰减

现代 Transformer 训练的标准调度结合了预热和余弦退火：

$$\alpha_t = \begin{cases} \alpha_{max} \cdot \frac{t}{T_{warmup}} & \text{如果} t < T_{warmup} \\ \alpha_{min} + \frac{1}{2}(\alpha_{max} - \alpha_{min})\left(1 + \cos\left(\frac{(t-T_{warmup})\pi}{T_{total}-T_{warmup}}\right)\right) & \text{否则} \end{cases} \tag{7.30}$$

这种预热加余弦的调度是许多 Transformer 训练方法的默认配置；见 [Loshchilov and Hutter 2017; Goyal et al. 2017]。

7.5.6 ReduceLROnPlateau

监控指标（通常是验证损失），当改进停滞时降低学习率：

$$\alpha_t = \alpha_{t-1} \cdot \gamma \quad \text{如果连续} p \text{ 个 epoch 无改进} \tag{7.31}$$

其中 γ 是衰减因子（通常为 0.1），p 是耐心值（通常为 5–10 个 epoch）。

- 自适应：无需预先定义总 epoch 数
- 当训练时间未知时有用（如早停）
- 常用于训练 CNN 进行图像分类

7.5.7 OneCycleLR

Smith [L. N. Smith 2018] 提出，OneCycleLR 使用单个周期，分三个阶段：

1. 增加 LR 从 α_{min} 到 α_{max}（约前 30% 的步数）
2. 减少 LR 从 α_{max} 到 α_{min}（约接下来 60% 的步数）

3. 微调：进一步减少到 $\alpha_{min}/100$（最后约 10% 的步数）

可选地，动量遵循逆模式（LR 低时动量高）。

OneCycleLR 的优势

- 收敛更快：通常用更少的 epoch 达到相同精度
- 正则化效果：高到低的 LR 周期起到隐式正则化的作用
- 无需预热：内置于调度中
- 在 fastai 和迁移学习任务中很受欢迎

7.6 优化器选择

Table 7.2 – 优化器选择指南

优化器	适用场景	备注
SGD + 动量	图像分类	泛化最好
Adam	通用	默认选择
AdamW	Transformer、NLP	微调最佳
RMSProp	RNN	循环模型适用

7.7 本章小结

1. 梯度下降是神经网络优化的基础
2. 动量通过累积速度加速收敛
3. 自适应方法（AdaGrad、RMSProp、AdaDelta、Adam）调整每个参数的学习率
4. Adam 结合动量和自适应学习率；AdamW 添加正确的权重衰减
5. 梯度裁剪防止梯度爆炸（优先使用范数裁剪）
6. 学习率调度器（步进、余弦、plateau、OneCycleLR）改善收敛
7. 预热对 Transformer 训练很重要
8. SGD+ 动量通常泛化更好；Adam 收敛更快

7.8 扩展阅读

- [Zeiler 2012] ——ADADELTA：一种自适应学习率方法

- [Pascanu, Mikolov, and Bengio 2013] ——训练循环神经网络的困难
- [Kingma and J. Ba 2015] ——Adam：一种随机优化方法
- [Loshchilov and Hutter 2017] ——带热重启的随机梯度下降
- [Reddi, Kale, and Kumar 2018] ——关于 Adam 的收敛性及以后
- [L. N. Smith 2018] ——神经网络超参数的有原则方法
- [Loshchilov and Hutter 2019a] ——解耦权重衰减正则化
- [Paszke et al. 2019] ——PyTorch：一种命令式风格的高性能深度学习库

7.9 习题

1. 动量更新：从 $\mathbf{v}_{t+1} = \beta\mathbf{v}_t + (1-\beta)\mathbf{g}_t$ 开始，解释动量如何与普通 SGD 相比平滑噪声梯度。
2. **Adam** 偏差校正：为什么 Adam 中需要偏差校正的矩估计 $\hat{\mathbf{m}}_t$ 和 $\hat{\mathbf{v}}_t$？如果省略它们，前几次迭代会发生什么？
3. **AdamW vs Adam**：展示在使用 Adam 时，解耦权重衰减与在损失上添加 L2 惩罚有何不同。为什么 AdamW 通常更适合 Transformer？
4. 梯度裁剪：假设梯度向量的范数为 12，裁剪阈值为 3。写出范数裁剪后的梯度，并解释保留了什么特性。
5. 余弦退火：绘制在一个训练周期中余弦学习率衰减的定性形状。为什么大的初始学习率可以帮助优化？
6. 调度器选择：为以下场景选择调度器并说明理由：长期预训练运行、未知停止时间、快速迁移学习微调。

Chapter 8

经典网络架构

8.1 引言

本章综述了塑造现代深度学习的里程碑式卷积网络和视觉架构。我们按时间顺序呈现它们，以展示每一代网络如何在前代局限的基础上发展：从 1998 年开创性的 LeNet-5 [LeCun, Bottou, et al. 1998] 到 2020 年的 Vision Transformer [Dosovitskiy et al. 2021] 及更晚的架构。理解这一演进过程不仅能帮助我们理解这些架构本身，也能揭示持续指导当今研究的关键设计原则。

8.2 神经网络的发展历程

深度学习架构的历史揭示了一条清晰的演进路径：

- **1958–1998** 年：感知机和早期神经网络；由于梯度消失和硬件限制，网络深度受限。
- **1998–2012** 年：LeNet 证明了 CNN 的可行性；在 GPU 加速的 AlexNet 出现之前，该领域在图像识别方面基本处于休眠状态。
- **2012–2015** 年：AlexNet 点燃了深度学习革命；VGGNet、GoogLeNet/Inception 和 ResNet 迅速推动了网络深度的增加。
- **2015–2019** 年：ResNet、DenseNet 和 MobileNet 变体主导；效率变得至关重要；SE 模块引入通道注意力机制。
- **2019** 年至今：EfficientNet、ViT、Swin Transformer 和 ConvNeXt 模糊了 CNN 和注意力机制范式之间的界限。

8.3 感知机与早期神经网络

8.3.1 感知机（1958 年）

Frank Rosenblatt 引入的感知机 [Rosenblatt 1958] 是第一个可训练的线性分类器：

$$y = \begin{cases} 1 & \text{若} \mathbf{w} \cdot \mathbf{x} + b > 0 \\ 0 & \text{其他情况} \end{cases} \tag{8.1}$$

其根本局限——无法解决非线性可分问题如 XOR——在 [Minsky and Papert 1969] 中被正式证明，并一度使神经网络研究停滞不前。

8.4 LeNet-5（1998 年）

Yann LeCun 及其同事引入 LeNet-5 [LeCun, Bottou, et al. 1998] 作为第一个成功的深度卷积网络，通过端到端反向传播训练用于手写数字识别（MNIST）。

LeNet-5 架构

1. 输入：32×32 灰度图像
2. C1：6 个 5×5 滤波器，步长 1 $\rightarrow 28 \times 28 \times 6$
3. S2：2×2 平均池化，步长 2 $\rightarrow 14 \times 14 \times 6$
4. C3：16 个 5×5 滤波器 $\rightarrow 10 \times 10 \times 16$
5. S4：2×2 平均池化 $\rightarrow 5 \times 5 \times 16$
6. C5：120 个 5×5 滤波器 $\rightarrow 120$
7. F6：全连接层，84 个单元，tanh 激活
8. 输出：10 类（数字）

总参数量：~60,000
关键创新：

- 通过卷积实现权重共享：与全连接网络相比大大减少了参数量。
- 局部感受野：利用图像的空间结构。
- 空间二次采样（池化）：实现平移不变性。
- 端到端训练：整个流水线（特征提取 + 分类）联合训练。

8.5 AlexNet（2012 年）

AlexNet [Krizhevsky, Sutskever, and G. E. Hinton 2012] 在 ImageNet LSVRC-2012 竞赛中获胜，前 5 名误差为 15.3%，而第二名 Entry 为 26.2%。这一差距革新了计算机视觉领域，开启了现代深度学习时代。

AlexNet 架构

1. 输入：$224 \times 224 \times 3$ RGB 图像
2. Conv1：96 个 11×11 滤波器，步长 4 → $55 \times 55 \times 96$
3. Pool1：3×3 最大池化，步长 2 → $27 \times 27 \times 96$
4. Norm1：局部响应归一化（LRN）
5. Conv2：256 个 5×5 滤波器 → $27 \times 27 \times 256$
6. Pool2：3×3 最大池化 → $13 \times 13 \times 256$
7. Conv3：384 个 3×3 滤波器
8. Conv4：384 个 3×3 滤波器
9. Conv5：256 个 3×3 滤波器
10. Pool5：3×3 最大池化 → $6 \times 6 \times 256$
11. FC6：4096 个神经元
12. FC7：4096 个神经元
13. FC8：1000 类（ImageNet）

参数量：~6000 万
关键创新：

- **ReLU** 激活：取代 tanh/sigmoid，加快训练速度并缓解梯度消失问题。
- **GPU** 训练：在两块 NVIDIA GTX 580 GPU 上并行训练，耗时 5-6 天。
- 数据增强：随机裁剪、水平翻转和基于 PCA 的颜色抖动。
- **Dropout** 正则化 [Srivastava et al. 2014]：应用于全连接层以减少过拟合。
- 局部响应归一化（**LRN**）：批量归一化的前身。

8.6 VGGNet（2014 年）

Simonyan 和 Zisserman [Simonyan and Zisserman 2015] 在牛津大学视觉几何组（Visual Geometry Group）表明，使用非常小（3×3）的卷积核增加网络深度是实现高精度的重要因素。VGG-16 和 VGG-19 在 ILSVRC 2014 中获得第二名。

VGG-16 架构

1. 输入：$224 \times 224 \times 3$
2. 卷积块（均使用 3×3 滤波器、ReLU，然后最大池化）：
   - 块 1：2× 卷积 64，池化 $\rightarrow 112 \times 112 \times 64$
   - 块 2：2× 卷积 128，池化 $\rightarrow 56 \times 56 \times 128$
   - 块 3：3× 卷积 256，池化 $\rightarrow 28 \times 28 \times 256$
   - 块 4：3× 卷积 512，池化 $\rightarrow 14 \times 14 \times 512$
   - 块 5：3× 卷积 512，池化 $\rightarrow 7 \times 7 \times 512$
3. 全连接层：4096, 4096, 1000

关键洞察：两个 3×3 卷积具有与一个 5×5 卷积相同的有效感受野，但参数量更少，且多了一层非线性：

- 一个 5×5 卷积：$25C^2$ 个参数
- 两个 3×3 卷积：$2 \times 9C^2 = 18C^2$ 个参数（减少 28%）
- 两个 ReLU 激活 vs 一个：更大的表示能力

参数量：~1.38 亿（大部分在 FC 层；骨干网络仅 ~15M）

8.7 GoogLeNet / Inception（2014 年）

Szegedy 等人 [Szegedy, W. Liu, et al. 2015] 引入 GoogLeNet（Inception v1），以 6.7% 的前 5 名误差赢得 ILSVRC 2014。其关键创新是 Inception 模块，它并行应用多种卷积大小并拼接它们的输出。

Inception 模块（朴素版）

不同尺度的并行操作：

- 1×1 卷积（通道压缩/瓶颈）
- 3×3 卷积（在 1×1 瓶颈之后）
- 5×5 卷积（在 1×1 瓶颈之后）
- 3×3 最大池化（在 1×1 瓶颈之后）

所有输出沿通道维度拼接。

关键创新：

- 多尺度特征提取：同时捕获细粒度和粗粒度模式。
- 1×1 卷积降维：显著降低计算成本。
- 全局平均池化（**GAP**）：取代大型全连接层，大大减少参数量。
- 辅助分类器：两个中间 softmax 头在训练期间提供额外的梯度信号，对抗梯度消失。

参数量：~400 万（比 AlexNet 少 15 倍，但精度更高）

Inception 架构在 Inception v2/v3 [Szegedy, Vanhoucke, et al. 2016] 中进一步优化，将大卷积分解（如 $5 \times 5 \to$ 两个 3×3，$n \times n \to 1 \times n + n \times 1$）以进一步提高效率。

8.8 ResNet（2015 年）

He 等人 [He, X. Zhang, et al. 2016a] 引入残差网络（ResNet），解决了退化问题：反直觉地，简单地增加网络层数会使训练更加困难，即使在理论上。其解决方案是残差（跳跃）连接：

$$\mathbf{y} = \mathcal{F}(\mathbf{x}, \{W_i\}) + \mathbf{x} \tag{8.2}$$

ResNet 以 3.57% 的前 5 名误差赢得 ILSVRC 2015，超越了人类水平（~5.1%）。

ResNet-50（瓶颈块）

1. Conv1：7×7，64 个滤波器，步长 $2 \to 112 \times 112 \times 64$
2. 池化：3×3 最大池化，步长 $2 \to 56 \times 56 \times 64$
3. 阶段 1：3 个瓶颈块，64/256 通道
4. 阶段 2：4 个瓶颈块，128/512 通道，步长 2
5. 阶段 3：6 个瓶颈块，256/1024 通道，步长 2
6. 阶段 4：3 个瓶颈块，512/2048 通道，步长 2
7. 全局平均池化
8. FC：1000 类

每个瓶颈块：1×1（压缩）$\to 3 \times 3$（卷积）$\to 1 \times 1$（扩展）+ 跳跃连接。

关键创新：

- 残差/跳跃连接：梯度可以直接从输出流向早期层。
- 恒等快捷方式：相同维度的快捷方式不增加额外参数。

- 瓶颈设计（$1 \times 1, 3 \times 3, 1 \times 1$）：使深度网络具有可控的计算量。
- 可扩展性：ResNet-18, 34, 50, 101, 152；后来扩展到 ResNet-1001+ [He, X. Zhang, et al. 2016b]。

参数量：~2500 万（ResNet-50）

8.9 U-Net（2015 年）

Ronneberger、Fischer 和 Brox [Ronneberger, Fischer, and Brox 2015] 为生物医学图像分割引入 U-Net。其 U 形编码器-解码器结构，结合保留空间细节的跳跃连接，已成为密集预测任务的主导架构。

U-Net 架构

1. 收缩路径（编码器）：4 个块，每个块包含两个 3×3 卷积 +ReLU，然后是 2×2 最大池化；通道数：$1 \to 64 \to 128 \to 256 \to 512$
2. 瓶颈层：在最低分辨率处的 3×3 卷积（1024 通道）
3. 扩展路径（解码器）：2×2 转置卷积 → 与对应编码器特征图拼接 → 两个 3×3 卷积 +ReLU
4. 输出：1×1 卷积映射到类别数

跳跃连接直接将编码器特征图拼接到解码器输入。

关键设计原则：

- 跳跃连接保留在下采样过程中丢失的空间细节。
- 数据效率：原始论文仅用 30 张带标注的图像就取得了优异效果，使用了激进的数据增强。
- 对称架构：编码器和解码器相互镜像。
- 广泛扩展：Attention U-Net、U-Net++、V-Net [Milletari, Navab, and Ahmadi 2016]、nnU-Net。
- 用于医学成像、卫星分析、自动驾驶和图像生成（如 Stable Diffusion [Rombach et al. 2022]）。

8.10 DenseNet（2017 年）

Huang 等人 [Huang, Zhuang Liu, et al. 2017] 提出密集连接卷积网络（DenseNet），其中每一层接收同一密集块中所有前面层的拼接特征图：

$$\mathbf{x}_l = H_l([\mathbf{x}_0, \mathbf{x}_1, \ldots, \mathbf{x}_{l-1}]) \tag{8.3}$$

其中 $[\cdot]$ 表示通道维度拼接。对于有 L 层、增长率为 k 的块，第 l 层接收 $k_0 + k(l-1)$ 个输入通道。

关键创新：

- 特征复用：每层直接访问所有前面的特征，减少冗余。
- 强梯度流：密集连接充当隐式深度监督。
- 更好的参数效率：在 ImageNet 上，DenseNet-121（8M 参数）与 ResNet-50（25M 参数）达到相当的精度。
- 过渡层：密集块之间的 1×1 卷积 $+2 \times 2$ 平均池化，控制特征图大小。

参数量：~800 万（DenseNet-121）

8.11 MobileNet（2017 年）

Howard 等人 [Howard et al. 2017] 引入 MobileNets，用深度可分离卷积替代标准卷积，以实现移动和嵌入式视觉应用的大幅降低计算成本。

深度可分离卷积将标准 $K \times K$ 卷积分解为：

1. 深度卷积： $K \times K$ 卷积独立应用于每个输入通道（C_{in} 个深度为 1 的滤波器）。
2. 逐点卷积： 1×1 卷积组合通道输出（C_{out} 个滤波器）。

相对于标准卷积的 FLOPs 减少：

$$\frac{\text{深度可分离 FLOPs}}{\text{标准 FLOPs}} = \frac{1}{C_{out}} + \frac{1}{K^2} \tag{8.4}$$

对于 $K = 3, C_{out} = 256$：减少因子约为 8–9×。

MobileNet 系列

- **MobileNetV1** [Howard et al. 2017]：深度可分离卷积；宽度乘数 α 和分辨率乘数 ρ 实现精度-效率权衡。（~4.2M 参数，~0.58 GMACs）
- **MobileNetV2** [Sandler et al. 2018]：添加倒残差块（扩展通道 → 深度卷积 → 压缩），具有线性瓶颈以防止信息破坏。（~3.4M 参数）
- **MobileNetV3**：神经架构搜索（NAS）优化，硬件感知激活（h-swish）；V3-Small：2.5M 参数，77.7% Top-1。

8.12 ShuffleNet（2018 年）

Zhang 等人 [X. Zhang et al. 2018] 引入 ShuffleNet 以克服 MobileNet 昂贵的 1×1 逐点卷积（占总计算量的 93%）的限制。

ShuffleNet 对深度卷积和逐点卷积步骤都使用分组卷积，并引入通道重排操作以允许跨组信息流动：

$$\text{Shuffle}(\mathbf{X})_{g,c,h,w} = \mathbf{X}_{\lfloor c/(C/G) \rfloor, (g+c \cdot G) \bmod C, h, w} \tag{8.5}$$

没有重排，堆叠的分组卷积只能在组内通信。通道重排在不增加 1×1 卷积成本的情况下提供了高效的组间混合。

ShuffleNet 变体

- ShuffleNetV1：在 ARM 设备上比 MobileNet 快 1.5 倍，精度相当。
- ShuffleNetV2：引入通道分割和四个硬件效率准则：（1）相等的通道宽度最小化内存访问成本；（2）过多的分组卷积是有害的；（3）碎片化网络结构损害并行性；（4）逐元素操作不可忽视。ShuffleNetV2 1.5×：3.5M 参数，299M MACs，73.3% Top-1。

8.13 SENet（2018 年）

Hu、Shen 和 Sun [J. Hu, Shen, and G. Sun 2018] 引入压缩激励网络（SENet），以 2.25% 的前 5 名误差赢得 ILSVRC 2017。SE 块向任何现有架构添加了轻量级通道注意力机制：

SE 块

1. 压缩：全局平均池化 $\rightarrow 1 \times 1 \times C$ 通道描述符

$$s_c = \frac{1}{H \cdot W} \sum_{i=1}^{H} \sum_{j=1}^{W} x_c(i,j) \tag{8.6}$$

2. 激励：两个具有瓶颈的全连接层（缩减比 $r = 16$）和 sigmoid 门控

$$\mathbf{e} = \sigma(\mathbf{W}_2\, \delta(\mathbf{W}_1 \mathbf{s})), \quad \mathbf{W}_1 \in \mathbb{R}^{(C/r) \times C}, \ \mathbf{W}_2 \in \mathbb{R}^{C \times (C/r)} \tag{8.7}$$

3. 缩放：逐通道特征重新校准 $\tilde{x}_c = e_c \cdot x_c$

关键洞察：不同通道对不同输入携带不同的语义信息。SE 块学习强调判别

性通道并抑制不太有用的通道。增加的开销很小（< 总 FLOPs 的 1%），但在 ImageNet 上始终能提高 1-2% 的精度。SE 块已被整合到 ResNet（SE-ResNet）、MobileNet（SE-MobileNet）和许多其他架构中。同年引入的泛化版本——结合空间注意力的 CBAM（卷积块注意力模块）[Woo et al. 2018]——进一步扩展了这一思想。

8.14 EfficientNet（2019 年）

Tan 和 Le [Tan and Q. Le 2019] 提出 EfficientNet，采用原则性的复合缩放方法，使用单一复合系数 ϕ 同时缩放网络深度、宽度和输入分辨率：

$$\text{深度：} \quad d = \alpha^{\phi} \tag{8.8}$$

$$\text{宽度：} \quad w = \beta^{\phi} \tag{8.9}$$

$$\text{分辨率：} \quad r = \gamma^{\phi} \tag{8.10}$$

$$\text{约束：} \quad \alpha \cdot \beta^2 \cdot \gamma^2 \approx 2 \quad \text{（FLOPs 按 } 2^{\phi} \text{ 缩放）} \tag{8.11}$$

其中 $\alpha = 1.2, \beta = 1.1, \gamma = 1.15$ 是通过在 EfficientNet-B0 上进行网格搜索得到的。

Table 8.1 – EfficientNet 系列（ImageNet Top-1 精度）

变体	参数	**FLOPs**	分辨率	**Top-1**
B0	5.3M	0.39B	224^2	77.1%
B1	7.8M	0.70B	240^2	79.1%
B2	9.2M	1.0B	260^2	80.1%
B3	12M	1.8B	300^2	81.6%
B4	19M	4.2B	380^2	82.9%
B7	66M	37B	600^2	84.3%

EfficientNet 以显著更少的参数和 FLOPs 达到最先进的精度。复合缩放原则已成为架构缩放的标准指南。

8.15 Vision Transformer（ViT，2020 年）

Dosovitskiy 等人 [Dosovitskiy et al. 2021] 证明，当在大规模预训练时，直接应用于图像块序列的纯 Transformer 编码器——不包含任何卷积——可以在图像分类上达到具有竞争力的性能。

ViT 架构

1. 块嵌入：将 224×224 图像分割为 16×16 块（196 个块）；将每个块展平为向量；应用线性投影得到 d 维嵌入。
2. **[CLS]** 令牌：在块序列前添加一个可学习的分类令牌。
3. 位置编码：向每个令牌添加学习到的 1D 位置嵌入。
4. **Transformer** 编码器：L 层的堆叠；每层包含多头自注意力（MHSA）+ 层归一化 +MLP（GELU）+ 残差连接。
5. 分类头：对最后一层的 [CLS] 令牌输出应用线性层。

ViT 关键洞察

- 无归纳偏置：与 CNN 不同，ViT 没有内置的平移等变性或局部性。它必须从数据中学习这些，这需要大规模预训练。
- 数据饥饿：ViT 仅在大数据集（ImageNet-21k、JFT-300M）上预训练时才能超越 CNN。单独在 ImageNet-1k 上，它不如 ResNet [Dosovitskiy et al. 2021]。
- 良好的可扩展性：性能和更多数据与计算一致地改进；ViT-L/16 在 JFT-300M 预训练下达到 87.1% Top-1。
- **DeiT** [Touvron, Cord, et al. 2021]：使用教师（RegNet）蒸馏，使 ViT 能够仅在 ImageNet-1k 上从头开始训练，ViT-B/16 达到 85.2%。

CNN vs. ViT：当前格局（2022 年至今）

截至 2024 年，该领域已收敛到混合和现代化方法：

- **Swin Transformer** [Ze Liu et al. 2021]：具有移动窗口的分层 ViT，实现线性复杂度并支持密集预测。Swin-L：84.5% Top-1。
- **ConvNeXt** [Zhuang Liu et al. 2022]：纯 CNN 重新设计，纳入 Transformer 设计选择（大核、更少激活、层归一化）。ConvNeXt-L：85.1% Top-1，在相似 FLOPs 下与 Swin 持平。
- CNN 和 ViT 之间的竞争推动了快速进展。现在的选择往往取决于下游任务、数据规模和部署硬件，而不是一种范式普遍优于另一种。

8.16 总结与比较

8.17 章节小结

1. **LeNet-5**（1998 年）开创了用于字符识别的端到端 CNN 训练。

Table 8.2 – 按时间排序的架构比较（ImageNet Top-1 精度）

网络	年份	参数	**Top-1** 精度	关键创新
LeNet-5	1998	60K	N/A (MNIST)	CNN 权重共享
AlexNet	2012	60M	62.5%	GPU + ReLU + Dropout
VGG-16	2014	138M	71.5%	统一的 3×3 卷积堆叠
GoogLeNet	2014	4M	72.0%	Inception 多尺度
ResNet-50	2015	25M	76.1%	残差跳跃连接
U-Net	2015	~31M	N/A (分割)	编码器-解码器 + 跳跃
DenseNet-121	2017	8M	74.4%	密集特征复用
MobileNetV2	2018	3.4M	72.0%	倒残差 + 线性瓶颈
SENet (SE-ResNet-50)	2018	28M	77.6%	通道注意力
ShuffleNetV2 1.5×	2018	3.5M	73.3%	通道重排
EfficientNet-B7	2019	66M	84.3%	复合缩放
ViT-B/16	2020	86M	79.9%*	基于块的注意力
Swin-L	2021	197M	84.5%	分层移动窗口注意力
ConvNeXt-L	2022	198M	85.1%	现代化 CNN

* ImageNet-21k 预训练；仅 ImageNet-1k 为 77.9%。

2. **AlexNet**（2012 年）证明，在 GPU 上使用 ReLU 和 dropout 训练的深度 CNN 可以大大优于手工特征，开启了现代深度学习时代。
3. **VGGNet**（2014 年）展示了通过统一的 3×3 卷积堆叠实现深度的重要性。
4. **GoogLeNet/Inception**（2014 年）引入了多尺度特征提取和降维，以 15 倍的参数减少达到 AlexNet 精度。
5. **ResNet**（2015 年）用残差连接解决了退化问题，使数百层成为可能，并超越了人类水平的 ImageNet 性能。
6. **U-Net**（2015 年）为密集预测任务定义了带跳跃连接的编码器-解码器范式。
7. **DenseNet**（2017 年）通过跨所有层的密集特征复用提高了参数效率。
8. **MobileNet**（2017 年）引入了深度可分离卷积以实现高效的移动推理。
9. **ShuffleNet**（2018 年）通过分组卷积和通道重排进一步提高了移动效率。
10. **SENet**（2018 年）纳入通道注意力以自适应地重新校准特征响应。

11. **EfficientNet**（2019 年）证明了复合缩放始终优于单维度缩放。
12. **ViT**（2020 年）证明了纯注意力在足够规模下可以匹配或超越 CNN。
13. 该领域继续收敛：现代 CNN（ConvNeXt）和 ViT（Swin）在相似计算预算下达到相似性能。

8.18 延伸阅读

以下参考文献按时间顺序排列。

- [LeCun, Bottou, et al. 1998] — LeCun 等（1998 年）：LeNet-5 和基于梯度的文档识别学习。
- [Krizhevsky, Sutskever, and G. E. Hinton 2012] — Krizhevsky、Sutskever & Hinton（2012 年）：AlexNet 和 ImageNet 突破。
- [Simonyan and Zisserman 2015] — Simonyan & Zisserman（2014/2015 年）：VGGNet 深度研究。
- [Ronneberger, Fischer, and Brox 2015] — Ronneberger、Fischer & Brox（2015 年）：用于生物医学分割的 U-Net。
- [Szegedy, W. Liu, et al. 2015] — Szegedy 等（2015 年）：GoogLeNet / Inception v1。
- [He, X. Zhang, et al. 2016a] — He 等（2016 年）：ResNet 和深度残差学习。
- [He, X. Zhang, et al. 2016b] — He 等（2016 年）：深度残差网络中的恒等映射。
- [Szegedy, Vanhoucke, et al. 2016] — Szegedy 等（2016 年）：Inception v3 和标签平滑。
- [Huang, Zhuang Liu, et al. 2017] — Huang 等（2017 年）：DenseNet。
- [Howard et al. 2017] — Howard 等（2017 年）：MobileNets。
- [Sandler et al. 2018] — Sandler 等（2018 年）：MobileNetV2 倒残差。
- [X. Zhang et al. 2018] — Zhang 等（2018 年）：ShuffleNet。
- [J. Hu, Shen, and G. Sun 2018] — Hu、Shen & Sun（2018 年）：SENet。
- [Woo et al. 2018] — Woo 等（2018 年）：CBAM 通道和空间注意力。
- [Tan and Q. Le 2019] — Tan & Le（2019 年）：EfficientNet 复合缩放。
- [Dosovitskiy et al. 2021] — Dosovitskiy 等（2020/2021 年）：Vision Transformer（ViT）。

- [Touvron, Cord, et al. 2021] — Touvron 等（2021 年）：数据高效图像 Transformer 的 DeiT。
- [Ze Liu et al. 2021] — Liu 等（2021 年）：Swin Transformer。
- [Zhuang Liu et al. 2022] — Liu 等（2022 年）：ConvNeXt。

8.19 练习题

1. 参数计数：计算 VGG-16 中可训练参数的总数。识别哪个组件（卷积层 vs. 全连接层）占主导。ResNet-50 如何用约 5 倍更少的参数达到相当的精度？
2. 感受野分析：证明两个连续的 3×3 卷积层在输入上具有 5×5 的有效感受野。推广：k 个堆叠的 3×3 卷积的有效感受野是多少？
3. 残差学习：假设一个块学习 $\mathbf{y} = \mathcal{F}(\mathbf{x}) + \mathbf{x}$。如果 $\mathcal{F}$ 可以表示任何函数，为什么这种公式比直接学习 $\mathbf{y} = \mathcal{G}(\mathbf{x})$ 更容易优化？讨论恒等映射在梯度流中的作用。
4. 深度可分离卷积：对于输入 $H \times W \times 32$ 和产生 $H \times W \times 64$ 输出的 3×3 卷积，计算（a）标准卷积的 FLOPs，（b）深度可分离卷积的 FLOPs。加速比是多少？
5. **Inception** 模块：在 PyTorch 中实现 Inception v1 模块。选择输入通道 $C_{in} = 256$，输出配置如原始论文（$64 + 128 + 32 + 32 = 256$ 输出通道）。比较使用和不使用 1×1 瓶颈卷积的 FLOPs。
6. **EfficientNet** 缩放：给定基线 EfficientNet-B0，参数 $(\alpha, \beta, \gamma) = (1.2, 1.1, 1.15)$，计算 $\phi = 3$（EfficientNet-B3）的深度、宽度和分辨率缩放因子。验证 $\alpha \cdot \beta^2 \cdot \gamma^2 \approx 2$。
7. **ViT** 令牌：ViT-B/16 通过将 224×224 图像分割为 16×16 块来处理。（a）有多少个块令牌？（b）包括 [CLS] 令牌，总序列长度是多少？（c）序列长度 N 的二次注意力复杂度是多少，与图像分辨率相比如何？

Chapter 9

参数与计算分析

9.1 模型效率的重要性

理解模型效率对于以下方面至关重要：

- 在资源受限设备上部署
- 训练成本估算
- 模型选择与架构设计
- 基准测试与比较

关键指标：

- 参数：模型大小与内存占用
- **FLOPs**：计算复杂度
- 延迟：推理时间
- 吞吐量：每秒处理的样本数

9.2 参数计数

参数是网络中可学习的权重和偏置。

9.2.1 全连接层

对于输入维度 d_{in} 和输出维度 d_{out}：

$$\text{权重} = d_{in} \times d_{out} \tag{9.1}$$

$$\text{偏置} = d_{out} \tag{9.2}$$

$$\text{总计} = d_{in} \times d_{out} + d_{out} \tag{9.3}$$

示例：从 784 到 256 的全连接层：

$$784 \times 256 + 256 = 200,704 + 256 = 200,960 \text{ 个参数} \tag{9.4}$$

9.2.2 卷积层

对于输入通道 C_{in}、输出通道 C_{out}、卷积核大小 K：

$$\text{权重} = K \times K \times C_{in} \times C_{out} \tag{9.5}$$
$$\text{偏置} = C_{out} \quad \text{（通常）} \tag{9.6}$$
$$\text{总计} = K^2 \times C_{in} \times C_{out} + C_{out} \tag{9.7}$$

示例：3×3 卷积，64 输入，128 输出：

$$3 \times 3 \times 64 \times 128 + 128 = 73,728 + 128 = 73,856 \text{ 个参数} \tag{9.8}$$

9.2.3 批量归一化

对于 C 个通道：

$$\gamma\text{（缩放）} = C \tag{9.9}$$
$$\beta\text{（平移）} = C \tag{9.10}$$
$$\mu\text{（运行均值）} = C \quad \text{（仅推理）} \tag{9.11}$$
$$\sigma\text{（运行方差）} = C \quad \text{（仅推理）} \tag{9.12}$$
$$\text{可训练参数总计} = 2C \tag{9.13}$$

注意：在训练期间，每个通道的 BN 有 2 个可训练和 2 个不可训练参数。

9.3 FLOPs（浮点运算）

FLOPs 衡量前向传播的计算成本。

9.3.1 全连接层的 FLOPs

对于矩阵乘法 $\mathbf{Y} = \mathbf{WX} + \mathbf{b}$：

$$\text{乘法} = d_{out} \times d_{in} \quad \text{（每样本）} \tag{9.14}$$
$$\text{加法} = d_{out} \times (d_{in} - 1) \quad \text{（近似于} d_{out} \times d_{in}\text{）} \tag{9.15}$$
$$\text{FLOPs} \approx 2 \times d_{out} \times d_{in} \quad \text{（将乘加计为 2 FLOPs）} \tag{9.16}$$

对于批次大小 N：$\text{FLOPs} = 2 \times N \times d_{out} \times d_{in}$

9.3.2 卷积层的 FLOPs

对于输出特征图 $H_{out} \times W_{out}$：

$$\text{每输出像素 FLOPs} = K^2 \times C_{in} \times C_{out} \quad \text{（乘-加对）} \times 2 \tag{9.17}$$

$$\text{总 FLOPs} = H_{out} \times W_{out} \times K^2 \times C_{in} \times C_{out} \times 2 \tag{9.18}$$

示例：224×224 输入，3×3 卷积，64 输入，128 输出：

$$H_{out} \times W_{out} = 224 \times 224 = 50,176 \tag{9.19}$$

$$\text{FLOPs} = 50,176 \times 3 \times 3 \times 64 \times 128 \times 2 \approx 1.17 \text{ GFLOPs} \tag{9.20}$$

MACs vs FLOPs

有时 FLOPs 被报告为乘累加（MAC）操作：

$$\text{MAC} = \sum_i a_i \cdot b_i \tag{9.21}$$

每个 MAC 通常计为 1 次操作（如果分别计算乘法和加法，则为 2 FLOPs）。1 MAC ≈ 2 FLOPs（对于基本算术）。

9.4 实际示例

9.4.1 LeNet-5 参数计数

层	输入	权重	**FLOPs**
卷积 1	$1 \times 32 \times 32$	150	$2.7M$
卷积 2	$6 \times 14 \times 14$	$2,400$	$4.2M$
全连接 1	400	$240K$	$480K$
全连接 2	120	$48K$	$48K$
总计	-	~60K	~7M

9.4.2 AlexNet 参数计数

9.5 迁移学习与效率

迁移学习可以显著减少所需的数据和训练成本。

层	输出	参数	**FLOPs**
卷积 1	$96 \times 55 \times 55$	35K	105M
卷积 2	$256 \times 27 \times 27$	614K	448M
卷积 3	$384 \times 13 \times 13$	885K	149M
卷积 4	$384 \times 13 \times 13$	1.3M	224M
卷积 5	$256 \times 13 \times 13$	884K	149M
全连接 6	4096	37.7M	37.7M
全连接 7	4096	16.8M	16.8M
全连接 8	1000	4.1M	4.1M
总计	-	~60M	~1.1B

迁移学习的优势

- 更低的训练成本（无需从头训练）
- 可以使用更小的数据集
- 收敛更快
- 通常最终性能更好

常见策略：

1. 冻结：保持预训练权重固定
2. 微调：用小学习率调整所有/部分层
3. 特征提取：使用预训练模型作为特征提取器

9.6 详细模型分解

9.6.1 ResNet-50 参数分析

ResNet-50 [He, X. Zhang, et al. 2016a] 由 4 个阶段组成，使用瓶颈块（1×1, 3×3, 1×1 卷积）。

关键观察：

- 阶段 3 和 4 包含 46% 的参数量，但占 55% 的 FLOPs（低分辨率下高通道数）
- 仅最终全连接层就有 2M 参数（占总数的 8%）——在现代变体中常被 GAP 取代
- ResNet-50-D（在下采样中使用平均池化）以零额外参数提高精度

Table 9.1 – ResNet-50 逐层参数计数

层	输出形状	参数	**FLOPs**
卷积 1 (7 × 7, 64, s2)	$112^2 \times 64$	9,408	118M
最大池化 3 × 3, s2	$56^2 \times 64$	0	–
阶段 1 (3 个块, 64)	$56^2 \times 256$	215,808	560M
阶段 2 (4 个块, 128)	$28^2 \times 512$	1,167,616	1,180M
阶段 3 (6 个块, 256)	$14^2 \times 1024$	4,715,776	2,360M
阶段 4 (3 个块, 512)	$7^2 \times 2048$	7,095,552	2,360M
全局平均池化	2048	0	–
FC（1000 类）	1000	2,049,000	4M
总计		**25,557,032**	**≈4.1B**

9.6.2 Transformer 模型缩放

对于具有 L 层、隐藏维度 d_{model}、h 个注意力头和词表大小 V 的 Transformer 模型：

$$\text{嵌入参数} = V \times d_{model} \tag{9.22}$$

$$\text{每层参数} \approx 12 \cdot d_{model}^2 \quad \text{（注意力：} 4d_{model}^2\text{，FFN：} 8d_{model}^2\text{）} \tag{9.23}$$

$$\text{总参数} \approx V \cdot d_{model} + 12L \cdot d_{model}^2 \tag{9.24}$$

Table 9.2 – Transformer 模型规模

模型	层数	d_{model}	参数	**FLOPs/token**
BERT-Base	12	768	110M	22.5B
BERT-Large	24	1024	340M	84B
GPT-2 Small	12	768	117M	–
GPT-2 Medium	24	1024	345M	–
GPT-2 XL	24	1600	774M	–
GPT-3	96	12288	175B	–
LLaMA-7B	32	4096	6.7B	–

缩放定律

[Kaplan et al. 2020] 中的缩放定律分析表明，Transformer 性能可预测地取决于三个因素：

$$\mathscr{L}(N, D, C) \approx \left(\frac{N_c}{N}\right)^{\alpha_N} + \left(\frac{D_c}{D}\right)^{\alpha_D} \tag{9.25}$$

其中 N = 参数，D = 数据量，C = 计算量，$\alpha_N \approx 0.076$，$\alpha_D \approx 0.095$。这意味着，为了保持效率，参数、数据和计算应该成比例缩放（每个" 数量级" 改进约需要 ×10）。

9.7 激活内存分析

在训练期间，内存消耗主要由激活（为反向传播存储的中间特征图）决定，而非参数。

9.7.1 激活内存公式

对于形状为 (N, C, H, W) 的卷积层输出（FP32）：

$$\text{激活内存} = N \times C \times H \times W \times 4 \text{ 字节（FP32）} \tag{9.26}$$

示例 **9.1** (ResNet-50 激活内存). 对于批次大小 $N = 32$，*FP32*：

- 阶段 *1* 输出（$56^2 \times 256$）：$32 \times 256 \times 56^2 \times 4 = 103\ MB$
- 阶段 *2* 输出（$28^2 \times 512$）：$32 \times 512 \times 28^2 \times 4 = 51\ MB$
- 阶段 *3* 输出（$14^2 \times 1024$）：$32 \times 1024 \times 14^2 \times 4 = 26\ MB$
- 所有层（包括中间层）的激活总计：≈ 3–$4\ GB$

使用混合精度（*FP16*），激活内存减半（$\approx 2\ GB$）。

减少激活内存的方法

- 梯度检查点：在反向传播期间重新计算激活。以计算换内存（~33% 开销）换取内存（~60% 减少）。
- 混合精度（**AMP**）：以 FP16 存储激活，以 FP32 存储主权重。以最小精度损失减半激活内存。
- 梯度累积：模拟大批量而不成比例增加内存。
- **DeepSpeed ZeRO**：跨 GPU 划分优化器状态、梯度和参数。

9.8 模型压缩与量化

9.8.1 量化

量化降低权重和激活的数值精度：

Table 9.3 – 量化级别

精度	字节/参数	范围	大小（**100M**）	加速比
FP32	4	$\pm 3.4 \times 10^{38}$	400 MB	1×
FP16	2	±65504	200 MB	2×
BF16	2	$\pm 3.4 \times 10^{38}$	200 MB	2×
INT8	1	$[-128, 127]$	100 MB	2–4×
INT4	0.5	$[-8, 7]$	50 MB	4–8×

训练后量化（**PTQ**）在不重新训练的情况下量化已训练模型。量化感知训练（**QAT**）在训练期间模拟量化以获得更高精度。

9.8.2 其他压缩技术

- 剪枝：移除不重要的权重（基于幅值或结构化）。可以在精度损失 <1% 的情况下减少 2–3 倍 FLOPs。
- 知识蒸馏：训练较小的" 学生" 模型来模仿较大的" 教师" 模型。学生通常超越直接训练的同等大小模型。
- 低秩分解：将权重矩阵分解为 $\mathbf{W} = \mathbf{U}\mathbf{V}^\top$，其中 $\text{rank}(\mathbf{U}) = \text{rank}(\mathbf{V}) \ll \text{rank}(\mathbf{W})$。

9.9 章节小结

1. 参数计数决定模型大小和内存需求
2. FLOPs 衡量计算复杂度（1 MAC ≈ 2 FLOPs）
3. ResNet-50：25.6M 参数，4.1B FLOPs；Transformer 参数按 $12Ld^2$ 缩放
4. 训练期间激活内存常超过参数内存
5. 梯度检查点和混合精度以计算换内存
6. 量化（INT8、INT4）实现边缘设备高效部署
7. 迁移学习高效利用预训练模型
8. 缩放定律指导大型基础模型的设计

9.10 延伸阅读

- [LeCun, Bottou, et al. 1998] — 基于梯度的文档识别学习（LeNet-5）
- [Krizhevsky, Sutskever, and G. E. Hinton 2012] — 使用深度 CNN 的 ImageNet 分类（AlexNet）
- [G. Hinton, Vinyals, and Jeff Dean 2015] — 蒸馏神经网络中的知识
- [Tianqi Chen et al. 2016] — 以亚线性内存成本训练深度网络
- [Han, Mao, and Dally 2016] — 深度压缩：剪枝、量化和霍夫曼编码
- [He, X. Zhang, et al. 2016a] — 用于图像识别的深度残差学习
- [Howard et al. 2017] — 移动视觉应用的高效 CNN
- [Micikevicius et al. 2018] — 混合精度训练
- [Kaplan et al. 2020] — 神经语言模型的缩放定律

9.11 练习题

1. 全连接层参数：计算输入维度 1024、输出维度 256、批次大小 32 的全连接层的参数和 FLOPs。
2. 卷积 **FLOPs**：推导输入通道 64、输出通道 128、输出特征图大小 28×28 的 3×3 卷积的 FLOPs。
3. **ResNet-50** 分解：使用本章中的阶段总计，估计 ResNet-50 参数中有多少比例位于最终分类层之外。
4. **Transformer** 缩放：对于 $L = 24$、$d_{model} = 1024$、词表大小 $V = 50{,}000$ 的 Transformer，使用本章中的近似公式估算总参数数。
5. 激活内存：对于批次大小 16、激活张量形状 $(16, 512, 28, 28)$ 以 FP16 存储，计算激活内存（MB）。
6. 压缩策略：您需要在内存紧张但计算能力适中的边缘设备上部署模型。您会首先优先考虑剪枝、量化、蒸馏还是低秩分解？证明您的选择。

Chapter 10

训练过程与框架

10.1 引言

训练神经网络是找到使给定损失函数在数据集上最小化的参数集的过程。虽然优化的数学基础（梯度下降、反向传播）已在前面章节介绍，但本章聚焦于训练过程本身的实践和理论方面。我们探讨为什么某些技术有效、如何诊断训练失败，以及现代框架如何使大规模训练变得可行。

训练过程涵盖多个相互关联的组成部分：优化算法、正则化策略、超参数配置以及高效计算的基础设施。深入理解这些组成部分如何相互作用对于成功训练现代神经网络至关重要。

10.2 训练循环

神经网络训练包括使用小批量随机梯度下降（SGD）或其变体对数据进行迭代优化。

10.2.1 形式化描述

给定数据集 $\mathscr{D} = \{(\mathbf{x}_i, \mathbf{y}_i)\}_{i=1}^N$、模型 $f(\mathbf{x};\theta)$（参数为 θ）和损失函数 ℓ，训练目标是：

$$\theta^* = \arg\min_\theta \frac{1}{N} \sum_{i=1}^N \ell\big(f(\mathbf{x}_i;\theta), \mathbf{y}_i\big) \tag{10.1}$$

实际上，我们对大小为 B 的小批量 $\mathscr{B}_t \subset \mathscr{D}$ 计算梯度：

$$\mathbf{g}_t = \nabla_\theta \frac{1}{B} \sum_{i \in \mathscr{B}_t} \ell\big(f(\mathbf{x}_i;\theta_t), \mathbf{y}_i\big) \tag{10.2}$$

Algorithm 2 训练循环

1. 初始化模型权重 θ_0
2. 对于每个轮次 $e = 1, 2, \ldots, E$：
   (a) 打乱训练数据
   (b) 对于每个小批量 $\mathscr{B}_t$：
      i. 前向传播：计算预测 $\hat{\mathbf{y}} = f(\mathbf{x}; \theta_t)$
      ii. 计算损失 $\mathscr{L}_t = \frac{1}{B} \sum_{i \in \mathscr{B}_t} \ell(\hat{\mathbf{y}}_i, \mathbf{y}_i)$
      iii. 反向传播：计算梯度 $\mathbf{g}_t = \nabla_\theta \mathscr{L}_t$
      iv. 更新权重：$\theta_{t+1} = \theta_t - \alpha \cdot \text{optimizer_step}(\mathbf{g}_t)$
   (c) 在验证集上评估
   (d) 如果验证指标改进则保存检查点
3. 返回最佳模型

10.2.2 为什么小批量 SGD 有效

一个自然的问题是：为什么要使用小批量而不是在整个数据集上计算精确梯度？答案在于三个因素之间的有益相互作用：

1. 噪声作为隐式正则化。小批量的梯度噪声防止优化器陷入泛化能力差的尖锐最小值。Keskar 等人 [Keskar et al. 2017] 表明，大批量训练倾向于收敛到" 尖锐" 最小值，泛化能力较差，而小批量 SGD 找到" 平坦" 最小值。
2. 计算效率。现代硬件（GPU/TPU）针对并行矩阵运算进行了优化。批次大小 32-256 可以充分利用 GPU 并行性，同时仍提供有意义的梯度估计。
3. 定期进度监控。每个小批量步骤都提供一个信号，允许在浪费数小时计算之前早期检测训练问题（发散、振荡）。

定理 10.1 (SGD 在递减学习率下的收敛性)**.** 在 $\mathscr{L}$ 的凸性和平滑性假设下，学习率调度 $\alpha_t = \mathscr{O}(1/\sqrt{t})$ 的 *SGD* 满足：

$$\mathbb{E}[\mathscr{L}(\theta_T)] - \mathscr{L}(\theta^*) \leq \mathscr{O}\left(\frac{1}{\sqrt{T}}\right) \tag{10.3}$$

相比之下，完整批次梯度下降为 $\mathscr{O}(1/T)$。较慢的速率是为每步更便宜计算付出的代价。

10.3 超参数

影响训练动态和最终模型质量的关键超参数。

10.3.1 学习率

学习率 α 可以说是最重要的超参数。它直接控制参数空间中的步长，并决定训练是收敛、振荡还是发散。

10.3.1.1 学习率的影响

- 太高：损失发散或剧烈振荡。优化器" 越过" 最小值。
- 太低：训练收敛极慢，可能被困在较差的局部最小值或鞍点。
- 适中：损失稳定下降，有轻微振荡提供正则化。
- 典型范围： 10^{-5} 到 0.1
- **Adam** 默认值： 10^{-3}
- **SGD** 默认值： 10^{-2}

10.3.1.2 学习率范围测试

Smith [L. N. Smith 2017] 提出了一种简单但有效的寻找良好学习率的方法。思路是在单次训练运行中从非常小的学习率指数增长到很大的值，绘制损失与学习率的关系图：

1. 从 $\alpha_{\min} = 10^{-6}$（或更小）开始
2. 每个批次将 α 增加 e 因子（或常数因子）
3. 当损失开始增加时停止
4. 选择最陡下降点的 α（通常为最大值的 10^{-1} 到 10^{-1}）

10.3.1.3 学习率预热

对于使用 Adam 训练的大型模型，标准做法是使用预热阶段，在前几百或几千步内将学习率从零逐渐增加到目标值。这可以防止在 Adam 动量估计不可靠的早期阶段（初始化为零时）出现不稳定。

预热调度通常是：

$$\alpha_t = \alpha_{\max} \cdot \min\left(1, \frac{t}{T_{\text{warmup}}}\right) \tag{10.4}$$

其中 T_{warmup} 是预热步数（例如，Transformer 模型 [Vaswani et al. 2017] 为 1000）。

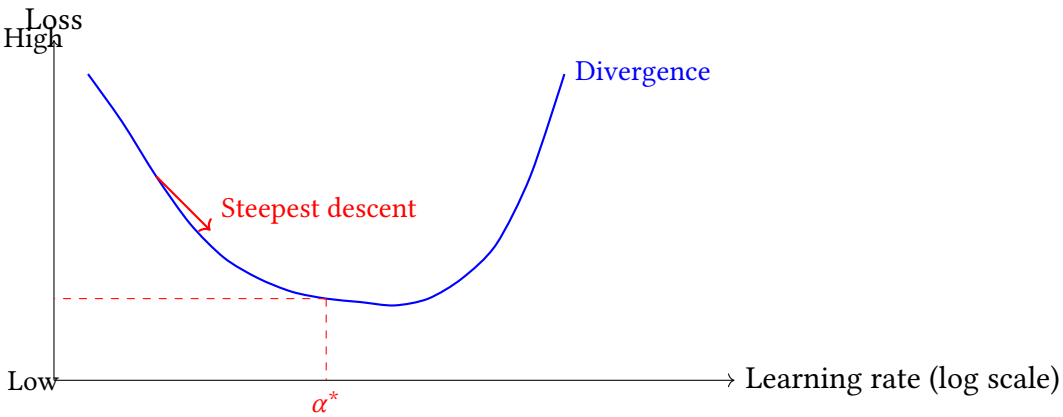


Figure 10.1 – 学习率范围测试。最陡下降点 α^* 提供了良好的初始学习率。

10.3.2 批次大小

批次大小 B 控制每次参数更新前处理的样本数。它对训练速度和模型质量都有深远影响。

10.3.2.1 批次大小的影响

- 小批次（**16-64**）：噪声更大的梯度提供隐式正则化，通常泛化能力更好。但 GPU 并行性利用不足。
- 中等批次（**64-256**）：泛化和硬件利用之间的良好权衡。大多数从业者的默认选择。
- 大批次（**256-8192+**）：更好的硬件利用率，更快的时钟训练。但通常需要学习率缩放，可能损害泛化能力。

10.3.2.2 线性缩放规则

Goyal 等人 [Goyal et al. 2017] 提出了大批次训练的线性缩放规则：当批次大小乘以 k 时，学习率也应乘以 k：

$$\alpha_{\text{large}} = \alpha_{\text{small}} \times \frac{B_{\text{large}}}{B_{\text{small}}} \tag{10.5}$$

批次大小与泛化

Keskar 等人 [Keskar et al. 2017] 和 Hoffer 等人 [Hoffer, Hubara, and Soudry 2017] 的研究表明，较小的批次通常获得更好的泛化能力，因为梯度噪声起到隐式正则化的作用。然而，效果取决于学习率、网络架构和训练时长等其他因素。

Table 10.1 – 超参数范围和默认值

超参数	典型范围	默认值	备注
学习率	10^{-5} 到 0.1	Adam: 10^{-3}；SGD: 10^{-2}	最重要
批次大小	16-8192	32-256	取决于 GPU 内存
训练轮次	10-1000	任务相关	推荐使用早停
权重衰减	10^{-6} 到 10^{-2}	10^{-2}（AdamW）	见第 10.4.2节
Dropout 率	0.1-0.5	0.1-0.3	见第 10.4.1节
动量	0.9-0.999	0.9（SGD）	Adam 为 0.9/0.999
预热步数	0-10K	1000（Transformers）	大模型受益更多

10.4 正则化

正则化包括所有通过约束模型容量或向训练过程添加噪声来减少过拟合的技术。我们为最重要的方法提供理论基础。

10.4.1 Dropout

Dropout [Srivastava et al. 2014] 是一种正则化技术，在训练期间以概率 p（dropout 率）随机将神经元激活置零。

10.4.1.1 带 Dropout 的前向传播

对于输入 $\mathbf{x}$ 和权重矩阵 $\mathbf{W}$ 的层，在训练期间：

$$\mathbf{y} = \mathbf{W} \cdot (\mathbf{x} \odot \mathbf{m}), \quad \mathbf{m} \sim \text{Bernoulli}(1-p) \tag{10.6}$$

其中 $\mathbf{m}$ 是二进制掩码，$\odot$ 表示逐元素乘法。

10.4.1.2 反转 Dropout

标准实现使用反转 **Dropout**，在训练期间缩放激活，以便在推理时不需要缩放：

$$\mathbf{y} = \frac{\mathbf{W} \cdot (\mathbf{x} \odot \mathbf{m})}{1-p} \tag{10.7}$$

这确保了 $\mathbb{E}[\mathbf{y}_{\text{train}}] = \mathbf{W}\mathbf{x} = \mathbf{y}_{\text{inference}}$，意味着有无 dropout 的期望输出相同。

证明：对于输入的任意分量 x_j：

$$\mathbb{E}\left[\frac{x_j \cdot m_j}{1-p}\right] = \frac{1}{1-p} \cdot \mathbb{E}[x_j \cdot m_j] \tag{10.8}$$

$$= \frac{1}{1-p} \cdot x_j \cdot \mathbb{E}[m_j] \tag{10.9}$$

$$= \frac{1}{1-p} \cdot x_j \cdot (1-p) = x_j \tag{10.10}$$

10.4.1.3 集成解释

Dropout 可以理解为训练 2^n 个共享参数的子网络集成（其中 n 是 dropout 单元数）。每次前向传播使用不同的掩码有效地对不同的子网络进行采样。在推理时，所有子网络被平均（根据缩放的线性性质，这等价于使用完整网络）。

更准确地说，具有 n 个 dropout 单元的网络有 2^n 个可能的精简网络。Dropout 训练目标最小化：

$$\mathscr{L}_{\text{dropout}} = \mathbb{E}_{\mathbf{m}\sim\text{Bernoulli}(1-p)}\left[\ell\big(f(\mathbf{x};\theta \odot \mathbf{m}), \mathbf{y}\big)\right] \tag{10.11}$$

这等价于最小化装袋集成目标，解释了 Dropout 通过方差减少的正则化效果。

10.4.1.4 Dropout 作为近似贝叶斯推理

Gal 和 Ghahramani [Gal and Ghahramani 2016] 表明，神经网络中的 Dropout 在数学上等价于深度高斯过程中的近似变分推理。在这个解释下：

- 每个 dropout 掩码定义了来自近似后验 $q(\theta)$ 的样本
- 隐式地最小化近似后验和真实后验之间的 KL 散度
- 推理时多次随机前向传播提供不确定性估计

这为 Dropout 不仅提供正则化而且提供模型不确定性估计的原则性解释——这是安全关键应用的关键特性。

10.4.1.5 Dropout 变体

10.4.2 权重衰减

10.4.2.1 L2 正则化作为 MAP 估计

权重衰减（L2 正则化）对模型参数的大小添加惩罚：

$$\mathscr{L}_{\text{reg}} = \mathscr{L} + \frac{\lambda}{2}\|\mathbf{W}\|_F^2 \tag{10.12}$$

Table 10.2 – Dropout 变体及其特性

变体	机制	使用场景
标准 Dropout	随机将激活置零	全连接层
空间 Dropout	将整个特征图置零	卷积层
DropConnect	随机将权重置零	Dropout 的替代
DropBlock	将连续区域置零	卷积层
注意力 Dropout	将注意力权重置零	Transformer 模型
特征 Dropout	将输入特征置零	输入正则化

这作为最大后验（MAP）估计有着优雅的解释。假设权重 $\mathbf{W} \sim \mathcal{N}(0, \sigma_w^2\mathbf{I})$ 的高斯先验。MAP 目标是：

$$\theta_{\text{MAP}} = \arg\max_{\theta} \log p(\mathcal{D}|\theta) + \log p(\theta) \tag{10.13}$$

$$= \arg\min_{\theta} \underbrace{-\sum_{i=1}^{N} \log p(\mathbf{y}_i|\mathbf{x}_i, \theta)}_{\mathcal{L}} + \underbrace{\frac{1}{2\sigma_w^2}\|\mathbf{W}\|_F^2}_{\lambda/2\cdot\|\mathbf{W}\|_F^2} \tag{10.14}$$

因此，系数为 λ 的 L2 正则化等价于方差为 $\sigma_w^2 = 1/\lambda$ 的高斯权重先验的 MAP 估计。

10.4.2.2 权重衰减 vs. L2 正则化：AdamW

对于 Adam 等自适应优化器，L2 正则化和解耦权重衰减之间存在微妙但重要的区别。在带 L2 正则化的标准 Adam 中，权重更新变为：

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot (\nabla\mathcal{L}_t + \lambda\mathbf{W}_t) \tag{10.15}$$

问题是权重衰减项 $\lambda\mathbf{W}_t$ 被自适应学习率 $\frac{1}{\sqrt{\hat{\mathbf{v}}_t}+\epsilon}$ 缩放，意味着具有大历史梯度的参数接收较少的正则化——这与我们想要的相反。

Loshchilov 和 Hutter [Loshchilov and Hutter 2019a] 提出了 **AdamW**，将权

重衰减与基于梯度的更新解耦：

$$\mathbf{W}_{t+1} = \mathbf{W}_t - \alpha \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \nabla \mathscr{L}_t - \alpha\lambda \mathbf{W}_t \tag{10.16}$$

$$= (1 - \alpha\lambda)\mathbf{W}_t - \alpha \cdot \frac{\hat{\mathbf{m}}_t}{\sqrt{\hat{\mathbf{v}}_t} + \epsilon} \odot \nabla \mathscr{L}_t \tag{10.17}$$

这确保了权重衰减均匀应用，无论自适应学习率如何，在实践中导致明显更好的泛化能力。

10.4.3 其他正则化技术

10.4.3.1 梯度裁剪

对于训练非常深的网络或循环网络，梯度可能变得非常大，导致使训练不稳定的参数更新。梯度裁剪限制梯度范数：

$$\text{若} \|\mathbf{g}\| > c: \quad \mathbf{g} \leftarrow \frac{c}{\|\mathbf{g}\|}\mathbf{g} \tag{10.18}$$

其中 c 是阈值（Transformer 模型通常为 1.0 或 5.0）。Pascanu 等人 [Pascanu, Mikolov, and Bengio 2013] 表明，梯度裁剪对于在长序列上训练 RNN 至关重要。

10.4.3.2 标签平滑

标签平滑 [Szegedy, Vanhoucke, et al. 2016] 用更软的分布替换硬独热标签。不是目标 $\mathbf{y} = [0, \ldots, 0, 1, 0, \ldots, 0]$，而是使用：

$$y_k^{\text{smooth}} = (1 - \epsilon) \cdot y_k + \frac{\epsilon}{K} \tag{10.19}$$

其中 ϵ 是平滑系数（通常为 0.1），K 是类别数。

标签平滑有效的原因：它防止模型变得过于自信。在训练数据上输出 $p(\text{correct}) = 0.999$ 的模型可能是在记忆而非学习。标签平滑通过要求模型将一些概率质量分配给不正确的类别来惩罚这一点，这：

- 改善校准（预测概率更好地反映真实准确性）
- 起到隐式正则化的作用；[R. Müller, Kornblith, and G. E. Hinton 2019] 将其与输出分布和均匀先验之间的 KL 散度添加联系起来
- 在机器翻译中将 BLEU 分数提高 0.5-1.0

10.4.3.3 随机深度

Huang 等人 [Huang, Y. Sun, et al. 2016] 提出在训练期间随机丢弃整个残差块（类似于 dropout 但应用于块级别）。这允许通过减少训练期间的有效深度来训练非常深的网络（例如 1200 层）：

$$\mathbf{y} = \begin{cases} \mathcal{F}_l(\mathbf{x}) + \mathbf{x} & \text{概率为} p_l \\ \mathbf{x} & \text{概率为} 1 - p_l \end{cases} \tag{10.20}$$

其中 p_l 是层 l 的存活概率，通常遵循从第一层到最后一层的线性衰减调度。

10.5 早停

当验证性能停止改进时，早停会停止训练。尽管简单，它是世界上最有效和广泛使用的正则化技术之一。

10.5.1 算法

1. 在每个轮次结束时监控验证损失（或合适的指标）
2. 每当验证指标改进时保存模型检查点
3. 如果连续 N 个轮次没有观察到改进（耐心值），停止训练
4. 返回最佳检查点（不是最终的）

典型耐心值：小模型 10-20 个轮次，大模型 5-10 个轮次。

10.5.2 早停作为正则化

Yao 等人表明，在某些条件下，早停提供的正则化效果等价于权重衰减。直觉很清晰：

1. 轮次 t 的模型具有沿优化轨迹的 参数 θ_t
2. 随着训练进展，模型拟合训练数据中越来越细粒度的模式
3. 超过某一点后，这些模式是噪声（过拟合）
4. 早停选择参数处于” 最佳点” 的模型，此时模型已学习信号但尚未学习噪声

形式上，对于具有二次损失的线性模型上的梯度下降，早停等价于 Tikhonov 正则化（L2 权重衰减），其中梯度步数扮演正则化强度的角色。

10.6 反向传播

10.6.1 计算图视角

反向传播是通过以反向拓扑顺序应用链式法则，高效计算通过计算图的梯度的算法。虽然数学原理很简单——链式法则——但计算效率来自于重用中间结果。

10.6.1.1 前向传播

考虑一个简单计算：$z = f(g(x))$。前向传播计算：

$$a = g(x) \tag{10.21}$$

$$z = f(a) \tag{10.22}$$

10.6.1.2 反向传播

反向传播以反向顺序计算梯度：

$$\frac{\partial \mathscr{L}}{\partial a} = \frac{\partial \mathscr{L}}{\partial z} \cdot \frac{\partial z}{\partial a} = \frac{\partial \mathscr{L}}{\partial z} \cdot f'(a) \tag{10.23}$$

$$\frac{\partial \mathscr{L}}{\partial x} = \frac{\partial \mathscr{L}}{\partial a} \cdot \frac{\partial a}{\partial x} = \frac{\partial \mathscr{L}}{\partial a} \cdot g'(x) \tag{10.24}$$

10.6.2 示例：双层网络

对于具有损失 $\mathscr{L}$ 的双层网络：

$$\mathbf{h} = \sigma(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1) \tag{10.25}$$

$$\hat{\mathbf{y}} = \mathbf{W}_2\mathbf{h} + \mathbf{b}_2 \tag{10.26}$$

$$\mathscr{L} = \ell(\hat{\mathbf{y}}, \mathbf{y}) \tag{10.27}$$

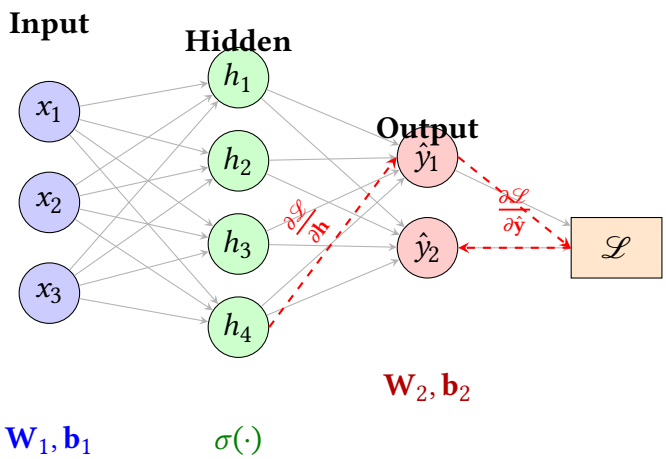


Figure 10.2 – 双层网络的计算图。实线箭头表示前向计算；红色虚线箭头表示反向梯度流。

反向传播计算：

$$\frac{\partial \mathscr{L}}{\partial \hat{\mathbf{y}}} = \nabla_{\hat{\mathbf{y}}} \ell \tag{10.28}$$

$$\frac{\partial \mathscr{L}}{\partial \mathbf{W}_2} = \frac{\partial \mathscr{L}}{\partial \hat{\mathbf{y}}} \mathbf{h}^\top, \qquad \frac{\partial \mathscr{L}}{\partial \mathbf{b}_2} = \frac{\partial \mathscr{L}}{\partial \hat{\mathbf{y}}} \tag{10.29}$$

$$\frac{\partial \mathscr{L}}{\partial \mathbf{h}} = \mathbf{W}_2^\top \frac{\partial \mathscr{L}}{\partial \hat{\mathbf{y}}} \tag{10.30}$$

$$\frac{\partial \mathscr{L}}{\partial \mathbf{a}} = \frac{\partial \mathscr{L}}{\partial \mathbf{h}} \odot \sigma'(\mathbf{a}) \tag{10.31}$$

$$\frac{\partial \mathscr{L}}{\partial \mathbf{W}_1} = \frac{\partial \mathscr{L}}{\partial \mathbf{a}} \mathbf{x}^\top, \qquad \frac{\partial \mathscr{L}}{\partial \mathbf{b}_1} = \frac{\partial \mathscr{L}}{\partial \mathbf{a}} \tag{10.32}$$

其中 $\mathbf{a} = \mathbf{W}_1 \mathbf{x} + \mathbf{b}_1$ 是预激活。

10.6.3 自动微分

现代深度学习框架实现反向模式自动微分（autodiff），它不同于数值微分和符号微分：

反向模式 autodiff 的关键洞察是，计算标量值函数相对于 n 个输入的所有偏导数仅花费前向传播成本的 $\mathcal{O}(1)$ 倍，与 n 无关。这使其成为神经网络的理想选择，其中损失是标量但有数百万个参数。

Table 10.3 – 微分方法比较

方法	精度	速度	用途
数值微分	近似	慢（n 次评估）	仅调试
符号微分	精确	快但内存重	CAS（Mathematica）
反向 autodiff	精确（至机器 ϵ）	$\mathcal{O}(1)\times$ 前向传播	PyTorch, JAX

Table 10.4 – 诊断训练失败

症状	可能原因	解决方案
损失不下降	学习率太低	增加 LR 或检查数据
损失为 NaN	学习率太高	降低 LR，添加梯度裁剪
训练损失下降，验证损失上升	过拟合	添加正则化，早停
损失振荡	批次大小太小	增加批次大小或降低 LR
收敛很慢	学习率太低	使用 LR 查找器
梯度消失	深度网络，初始化差	使用残差连接，ReLU

10.6.4 常见训练陷阱

10.7 模型检查点保存

在训练期间保存和恢复模型状态对于长时间运行的实验和生产部署至关重要。

- 模型架构：网络定义（层大小、类型）
- 权重：学到的参数 θ
- 优化器状态：动量项 $\mathbf{m}$、自适应学习率 $\mathbf{v}$（需要精确恢复训练）
- 轮次和步数计数器：训练进度
- 最佳指标：用于模型选择的验证指标
- 随机状态：用于可重复性的 RNG 种子

10.8 混合精度训练

10.8.1 动机

标准训练对所有计算使用 32 位浮点（FP32）。混合精度训练对大多数操作使用 16 位浮点（FP16），同时维护 32 位权重主副本，实现：

- **2** 倍内存减少：以 FP16 存储激活和梯度
- **2-3** 倍加速：现代 GPU 的 FP16 吞吐量是 FP32 的 2 倍（NVIDIA GPU 上的 Tensor Cores 为 FP16 矩阵乘法提供高达 8 倍的加速）
- 无精度损失：正确实现时

10.8.2 损失缩放问题

FP16 的动态范围有限（$\approx 6 \times 10^{-5}$ 到 65504）。小于 2^{-24} 的梯度下溢为零。由于深层网络中的梯度可能跨越多个数量级，小梯度可能完全丢失。

解决方案：在反向传播前将损失缩放因子 S：

$$\hat{\mathscr{L}} = S \cdot \mathscr{L} \tag{10.33}$$

$$\mathbf{g}_{\text{FP32}} = \frac{1}{S} \cdot \nabla_{\theta_{\text{FP16}}} \hat{\mathscr{L}} \tag{10.34}$$

动态损失缩放 [Micikevicius et al. 2018] 自动调整 S：每 N 个成功步骤将 S 增加 2 倍；当梯度溢出（包含 NaN/Inf）时将 S 减半。

10.8.3 BF16：现代替代方案

Bfloat16（BF16）使用与 FP32 相同的 8 个指数位，但只有 7 个尾数位（而 FP16 为 10 个）。这使 BF16 具有与 FP32 相同的动态范围（$\approx 10^{-38}$ 到 10^{38}），完全消除了对损失缩放的需要。BF16 在 NVIDIA A100/H100 GPU 上支持，正在成为训练大型语言模型的默认选择。

Table 10.5 – 浮点格式比较

属性	**FP32**	**FP16**	**BF16**	**TF32**
总位数	32	16	16	19
指数位	8	5	8	8
尾数位	23	10	7	10
动态范围	$\pm 3.4 \times 10^{38}$	± 65504	$\pm 3.4 \times 10^{38}$	$\pm 3.4 \times 10^{38}$
需要损失缩放	否	是	否	否
硬件支持	通用	Tensor Cores	A100+	A100+

10.9 PyTorch 框架

PyTorch 由于其即时执行模型和 Pythonic API，是深度学习研究中最流行的框架。

代码 **10.1** – PyTorch 中的完整训练循环

```
import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import DataLoader

# Model definition
model = nn.Sequential(
    nn.Linear(784, 512),
    nn.BatchNorm1d(512),
    nn.GELU(),
    nn.Dropout(0.2),
    nn.Linear(512, 256),
    nn.BatchNorm1d(256),
    nn.GELU(),
    nn.Dropout(0.1),
    nn.Linear(256, 10)
)

# Loss, optimizer, and scheduler
criterion = nn.CrossEntropyLoss(label_smoothing=0.1)
optimizer = optim.AdamW(model.parameters(), lr=1e-3,
                        weight_decay=0.01)
scheduler = optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100, eta_min=1e-5)

# Training loop with mixed precision
scaler = torch.cuda.amp.GradScaler()
device = torch.device('cuda' if torch.cuda.is_available()
                      else 'cpu')
model = model.to(device)

best_val_loss = float('inf')
patience = 10
patience_counter = 0

for epoch in range(num_epochs):
    # --- Training ---
    model.train()
    train_loss = 0.0
    for data, target in train_loader:
```

```
        data, target = data.to(device), target.to(device)

        optimizer.zero_grad(set_to_none=True)

        # Mixed precision forward pass
        with torch.cuda.amp.autocast():
            output = model(data.view(data.size(0), -1))
            loss = criterion(output, target)

        scaler.scale(loss).backward()
        scaler.unscale_(optimizer)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)
        scaler.step(optimizer)
        scaler.update()

        train_loss += loss.item()

    scheduler.step()

    # --- Validation ---
    model.eval()
    val_loss = 0.0
    correct = 0
    with torch.no_grad():
        for data, target in val_loader:
            data, target = data.to(device), target.to(device)
            with torch.cuda.amp.autocast():
                output = model(data.view(data.size(0), -1))
                val_loss += criterion(output, target).item()
            pred = output.argmax(dim=1)
            correct += pred.eq(target).sum().item()

    val_acc = 100. * correct / len(val_loader.dataset)
    avg_val_loss = val_loss / len(val_loader)

    # --- Checkpointing with early stopping ---
    if avg_val_loss < best_val_loss:
        best_val_loss = avg_val_loss
        patience_counter = 0
        torch.save({
            'epoch': epoch,
            'model_state_dict': model.state_dict(),
            'optimizer_state_dict': optimizer.state_dict(),
            'best_val_loss': best_val_loss,
        }, 'best_model.pt')
    else:
        patience_counter += 1
```

```
        if patience_counter >= patience:
            print(f'Early stopping at epoch {epoch}')
            break
```

10.10 CUDA 与 GPU 训练

10.10.1 GPU 编程模型

GPU 通过组织成流多处理器（SM）的数千个轻量级核心擅长数据并行操作。高效 GPU 利用率的关键考虑因素：

- 数据传输：CPU-GPU 传输（PCIe）比 GPU 计算慢几个数量级。通过将数据保存在 GPU 上最小化传输。
- 内核启动开销：许多小操作比少数大操作慢。在可能时使用融合内核。
- 内存合并：相邻线程应访问相邻内存地址以获得最佳带宽。
- 占用率：活动线程与最大线程的比率影响利用率，但不是唯一因素。

10.10.2 分布式训练

对于单个 GPU 无法容纳的模型，分布式训练策略至关重要：

Table 10.6 – 分布式训练策略

策略	机制	内存	速度
数据并行	复制模型，分割数据	与模型成线性关系	近线性扩展
模型并行	跨 GPU 分割模型	减少	通信受限
流水线并行	跨 GPU 分割层	减少	气泡开销
张量并行	在层内分割张量	减少	全归约成本
FSDP	分片参数 + 梯度	减少	通信成本

10.10.3 梯度累积

当所需有效批次大小超过 GPU 内存时，梯度累积通过在每次优化器步骤前累积多个微批次的梯度来模拟更大的批次：

$$\mathbf{g} = \frac{1}{K}\sum_{k=1}^{K} \nabla_\theta \mathscr{L}(\mathscr{B}_k) \tag{10.35}$$

其中 K 是累积步数，有效批次大小为 $B_{\text{eff}} = B \times K$。

代码 10.2 – PyTorch 中的梯度累积

```
accumulation_steps = 4  # Effective batch = 32 * 4 = 128
optimizer.zero_grad()

for i, (data, target) in enumerate(train_loader):
    output = model(data)
    loss = criterion(output, target) / accumulation_steps
    loss.backward()  # Accumulate gradients

    if (i + 1) % accumulation_steps == 0:
        optimizer.step()
        optimizer.zero_grad()
```

10.11 训练诊断

理解训练期间发生的情况需要系统地监控多个指标。

10.11.1 损失曲线

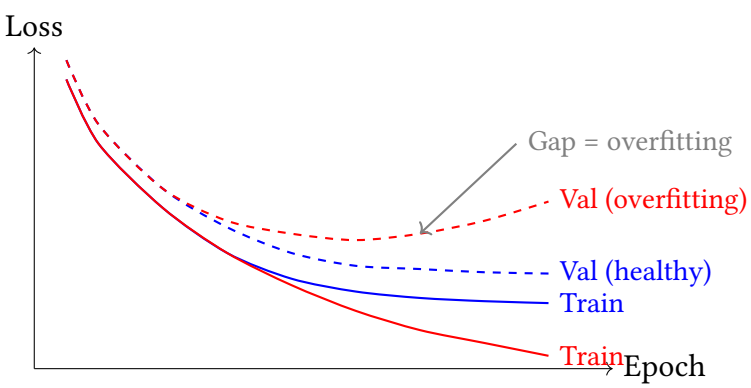


Figure 10.3 – 训练损失曲线：健康训练（蓝色）vs. 过拟合（红色）。实线：训练损失。虚线：验证损失。

10.11.2 梯度统计

监控梯度统计有助于诊断训练问题：

- 梯度范数：应该稳定。尖峰表示不稳定；范数消失表示死神经元或饱和激活。
- 梯度分布：应该近似高斯分布。重尾可能表示异常值样本。

- 更新-权重比：比率 $\|\alpha \mathbf{g}\|/\|\mathbf{W}\|$ 应该在 10^{-3} 左右。大得多表示不稳定；小得多表示停滞。
- 激活统计：层激活的均值和方差应在整个训练过程中保持稳定。批量归一化有助于此。

10.12 章节小结

1. 训练涉及对数据的迭代小批量优化，其中梯度噪声提供隐式正则化
2. 学习率是最关键的超参数；LR 范围测试提供了选择它的原则性方法
3. 预热对于具有自适应优化器的大型模型稳定早期训练至关重要
4. Dropout 通过集成平均提供正则化，可以解释为近似贝叶斯推理
5. AdamW 将权重衰减与自适应学习率解耦，比 Adam 的 L2 正则化导致更好的泛化能力
6. 早停防止过拟合，在某些条件下等价于正则化
7. 混合精度训练（FP16/BF16）在正确实现时提供 2-3 倍加速且无精度损失
8. 梯度累积使有效批次大小大于 GPU 内存的训练成为可能
9. 系统监控损失曲线、梯度统计和验证指标对于诊断训练问题至关重要

10.13 延伸阅读

- [Pascanu, Mikolov, and Bengio 2013] — 训练循环神经网络的困难（梯度裁剪）
- [Srivastava et al. 2014] — Dropout：防止神经网络过拟合的简单方法
- [I. Goodfellow, Bengio, and Courville 2016] — 第 7 章（正则化）和第 8 章（训练深度模型的优化）
- [Gal and Ghahramani 2016] — Dropout 作为贝叶斯近似：深度学习中模型不确定性的表示
- [Huang, Y. Sun, et al. 2016] — 带随机深度的深度网络（随机深度）
- [Szegedy, Vanhoucke, et al. 2016] — 重新思考用于计算机视觉的 Inception 架构（标签平滑）
- [Goyal et al. 2017] — 准确的大批次 SGD：一小时内训练 ImageNet
- [Keskar et al. 2017] — 关于大批次训练深度学习：泛化差距和尖锐最小值
- [L. N. Smith 2017] — 用于训练神经网络的循环学习率（LR 范围测试）

- [Micikevicius et al. 2018] — 混合精度训练
- [Loshchilov and Hutter 2019a] — 解耦权重衰减正则化（AdamW）
- [Rajbhandari et al. 2020] — ZeRO：面向万亿参数模型训练的内存优化

10.14 练习题

1. 学习率范围测试：描述您将如何运行 LR 范围测试并解释损失曲线以选择初始学习率。
2. 预热原理：为什么预热对于大批次训练和 Transformer 优化特别重要？它防止什么失败模式？
3. 权重衰减：在自适应优化器背景下比较 L2 正则化和 AdamW。为什么两者可能表现不同？
4. 早停：解释为什么早停可以被视为一种正则化形式。什么验证模式表明应该停止训练？
5. 混合精度：如果 FP16 使激活内存减半，通常添加什么次要机制来保持梯度数值稳定？
6. 梯度累积：假设您可以在一块 GPU 上容纳批次大小 16，但需要有效批次大小 128。需要多少累积步数，调用 backward 前应该对损失做什么？

Chapter 11

数据集处理

11.1 引言

机器学习模型的质量从根本上受限于其训练数据的质量俗话说："垃圾进，垃圾出。"本章涵盖神经网络训练数据准备的端到端流水线：收集、标注、划分、增强、加载、归一化，以及日益重要的数据质量和版本管理考量。

虽然讨论的技术广泛适用，但我们特别关注不同数据模态——图像、文本和音频——的独特处理要求。

11.2 数据收集

任何机器学习系统的基础是高质量数据。数据集决定了模型所能达到的性能上限。

11.2.1 数据来源

数据可以从多个来源获取，每个来源都有其质量和成本权衡：

- 公共数据集：策划的基准数据集（ImageNet、COCO、GLUE、SQuAD）提供标准化评估，但可能与目标分布不匹配。
- 网页抓取：从网站自动收集数据以低成本提供大量数据，但需要仔细去重和质量过滤。
- 传感器：摄像头、麦克风、医学成像设备、加速度计产生丰富数据，但需要仔细校准和预处理。
- 人工标注：专家标注提供高质量的真实标签，但成本高且速度慢。众包平台（Amazon Mechanical Turk、Scale AI）以规模换速度，牺牲标注质量。
- 合成数据：从模拟或生成模型生成的数据可以扩充有限的真实数据，在医学成像和机器人技术中特别有价值。

11.2.2 数据质量标准

高质量数据集共享几个特性：

1. 正确性：标签准确反映真实标签。错误引入偏差并限制模型性能上限。

2. 一致性：标注指南明确无歧义，标注者统一应用。
3. 代表性：数据集分布与目标部署领域匹配。在日光图像上训练的模型在夜间场景会失败。
4. 平衡性：类别频率和难度级别适合任务。严重不平衡需要特殊处理。
5. 去重：近似重复的样本膨胀了有效数据集大小并导致训练-测试泄露。

11.3 数据标注

标签实现监督学习。标注策略取决于任务类型。

11.3.1 标注类型

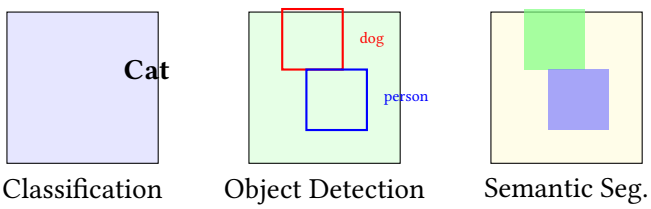


Figure 11.1 – 标注粒度：从图像级标签到像素级分割掩码。

- 分类：每张图像一个标签（如猫 vs 狗）。最简单的标注任务，容易出现模糊情况。
- 多标签分类：每张图像多个标签（如" 海滩、日落、人"）。每个标签是独立的二元决策。
- 目标检测：带类别标签的边界框。每个框指定一个区域及其类别。
- 语义分割：像素级类别标签。每个像素被分配一个类别（或" 背景"）。
- 实例分割：区分同类的不同个体（如每个人是一个单独的实例）。
- 关键点检测：感兴趣点（面部标志、人体关节、车辆检测点）。

11.3.2 标注工具

- **LabelImg**：用于边界框标注的开源工具。快速且轻量级。
- **CVAT**：计算机视觉标注工具，支持视频、图像和文本标注，具有协作功能。
- **Supervisely**：功能齐全的平台，带 AI 辅助标注（用现有模型预标注）。

- **Amazon Mechanical Turk**：大规模众包人工标注。通过共识/黄金问题进行质量控制。
- **Label Studio**：高度可配置的开源标注工具，支持多种数据类型。

11.4 数据划分

正确的数据划分对于获得模型性能的无偏估计至关重要。

11.4.1 基本划分

- 训练集：用于更新模型参数的样本。通常占数据的 70-90%。
- 验证集：用于超参数调整和模型选择。不用于训练。
- 测试集：用于最终评估。不得影响任何训练决策。

典型比例：

- 小数据集（<10K）：70/10/20 或 80/10/10（训练/验证/测试）
- 中等数据集（10K-1M）：80/10/10 或 85/5/10
- 大数据集（>1M）：98/1/1 可能足够，但确保测试集足够大以获得可靠估计

测试集泄露

永远不要将测试集用于训练期间的任何决策。这包括选择超参数、选择架构或早停阈值。这样做会导致过度拟合测试集和不可靠的性能估计。测试集应该”锁起来”直到最终评估。

11.4.2 分层划分

当类别分布不平衡时，随机划分可能产生不具代表性的划分。分层划分保持每个划分中的类别比例：

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, stratify=y
)
```

对于多标签分类，分层更复杂。一种方法是将每个标签组合视为一个单独的类别，尽管这可能导致罕见的组合。或者，迭代标签并平衡每个。

11.4.3 交叉验证

对于有限数据的稳健评估，K 折交叉验证提供更可靠的估计：

Algorithm 3 K 折交叉验证

1. 打乱数据并分成 K 个相等的折
2. 对于 $i = 1, 2, \ldots, K$：
   (a) 在折 $\{1, 2, \ldots, K\} \setminus \{i\}$ 上训练
   (b) 在折 i 上验证
3. 报告验证指标的平均值和标准差

常见值：$K = 5$（对于中等数据）或 $K = 10$（对于小数据，较大方差减少）。分层 K 折保持每个折中的类别比例。

11.5 数据增强

数据增强通过对现有样本应用随机变换人为增加数据集大小，在不需要额外数据收集的情况下提高泛化能力。

11.5.1 图像增强

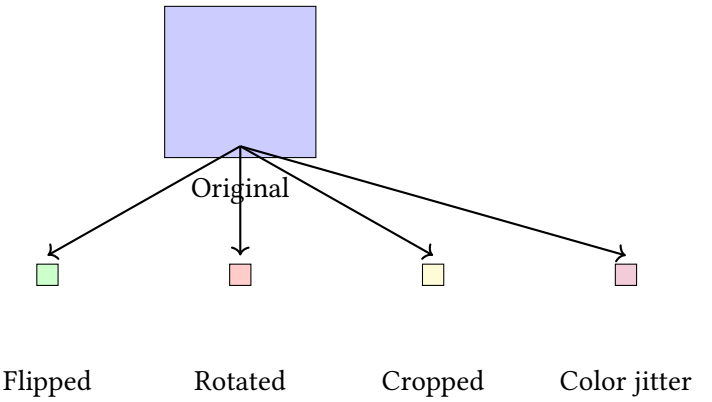


Figure 11.2 – 图像增强示例：水平翻转、旋转、随机裁剪和颜色抖动。

常见几何增强：

- 旋转：随机角度旋转（如 $\pm 15^\circ$）。应在保持标签的范围内。

- 翻转：水平翻转（左右镜像）。对于大多数分类任务是安全的，但不适用于文本或不对称物体。
- 随机裁剪：裁剪到较小区域，然后调整大小到原始大小。强制尺度不变性。
- 缩放：随机缩放（放大/缩小）。与不同裁剪比例的随机裁剪效果相同。
- 平移：在 x 和 y 方向上的随机平移。等价于非对称裁剪。

颜色增强：

- 颜色抖动：对亮度、对比度、饱和度和色调的随机调整。
- 高斯噪声：注入像素级高斯噪声以提高鲁棒性。
- **Cutout**/随机擦除：随机遮挡矩形区域。迫使模型使用分布式特征。
- 灰度/**RGB** 排列：对于多通道图像。

Mixup 和 CutMix 增强将样本混合在一起：

11.5.2 Mixup：样本的凸组合

Mixup [H. Zhang et al. 2018] 通过在样本对之间线性插值创建虚拟训练样本：

$$\tilde{\mathbf{x}} = \lambda \mathbf{x}_i + (1-\lambda)\mathbf{x}_j \tag{11.1}$$

$$\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1-\lambda)\mathbf{y}_j \tag{11.2}$$

其中 $\lambda \sim \mathrm{Beta}(\alpha, \alpha)$，$\alpha \in (0, \infty)$。混合系数 λ 控制每个样本的相对权重。

Mixup 为什么有效：

1. 正则化：模型看到平滑的决策边界，防止过度自信
2. 数据清理：将噪声标签与干净标签混合稀释标签噪声
3. 更好的校准：对混合输入的预测鼓励更软的目标
4. 标签平滑效果：对于分类，即使 y_i, y_j 是独热的，$\tilde{y}$ 也是类上的概率分布

对于具有独热标签 $\mathbf{y}_i, \mathbf{y}_j \in \{0,1\}^K$ 的 K 类分类问题，混合标签 $\tilde{\mathbf{y}}$ 是类上的概率分布。Mixup 损失变为：

$$\mathscr{L}_{\text{mixup}} = -\sum_{k=1}^{K} \tilde{y}_k \log \hat{y}_k = -\lambda \sum_k y_{i,k} \log \hat{y}_k - (1-\lambda) \sum_k y_{j,k} \log \hat{y}_k \tag{11.3}$$

这等价于最小化预测与两个独热标签混合之间的交叉熵。

11.5.3 CutMix：粘贴补丁

CutMix [Yun et al. 2019] 将一个图像的矩形区域粘贴到另一个图像中，而不是混合：

$$\tilde{\mathbf{x}} = \mathbf{x}_i \odot \mathbf{M} + \mathbf{x}_j \odot (1 - \mathbf{M}) \tag{11.4}$$

$$\tilde{\mathbf{y}} = \lambda \mathbf{y}_i + (1 - \lambda)\mathbf{y}_j \tag{11.5}$$

其中 $\mathbf{M} \in \{0, 1\}^{H \times W}$ 是二进制掩码，$\lambda = \frac{\text{area}(\mathbf{M})}{HW}$。

CutMix vs. Mixup：CutMix 保留图像的像素级特性（边界处无模糊），同时仍提供 Mixup 的正则化优势。矩形掩码在区域之间提供清晰对比，这可以帮助模型学习局部特征。

代码 **11.1** – PyTorch 中的 Mixup 和 CutMix

```python
import torch
import numpy as np

def mixup_data(x, y, alpha=1.0):
    """Returns mixed inputs, pairs of targets, and lambda."""
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1

    batch_size = x.size(0)
    index = torch.randperm(batch_size).to(x.device)

    mixed_x = lam * x + (1 - lam) * x[index, :]
    y_a, y_b = y, y[index]
    return mixed_x, y_a, y_b, lam

def mixup_criterion(criterion, pred, y_a, y_b, lam):
    return lam * criterion(pred, y_a) + (1 - lam) * criterion(pred, y_b)

def cutmix_data(x, y, alpha=1.0):
    """CutMix: Cuts and pastes patches between images."""
    if alpha > 0:
        lam = np.random.beta(alpha, alpha)
    else:
        lam = 1

    batch_size = x.size(0)
    index = torch.randperm(batch_size).to(x.device)

```

```
    # Get bounding box
    W = x.size(3); H = x.size(2)
    cut_rat = np.sqrt(1. - lam)
    cut_w = int(W * cut_rat); cut_h = int(H * cut_rat)

    cx = np.random.randint(W)
    cy = np.random.randint(H)
    bbx1 = np.clip(cx - cut_w // 2, 0, W)
    bby1 = np.clip(cy - cut_h // 2, 0, H)
    bbx2 = np.clip(cx + cut_w // 2, 0, W)
    bby2 = np.clip(cy + cut_h // 2, 0, H)

    x[:, :, bby1:bby2, bbx1:bbx2] = x[index, :, bby1:bby2, bbx1:bbx2]
    lam = 1 - ((bbx2 - bbx1) * (bby2 - bby1) / (W * H))

    y_a, y_b = y, y[index]
    return x, y_a, y_b, lam
```

11.5.4 高级增强策略

而不是手动设计增强，自动化方法搜索最优增强策略：

- **AutoAugment** [Cubuk, Zoph, Mane, et al. 2019]：从 CIFAR-10/CIFAR-100 学习的策略。发现的策略可以很好地迁移到其他数据集，但计算成本高。
- **RandAugment** [Cubuk, Zoph, Shlens, et al. 2020]：简化的搜索空间，只有两个参数（幅度、操作数量）。以少得多的计算几乎匹配 AutoAugment。
- **TrivialAugment Wide** [S. G. Müller and Hutter 2023]：无需搜索。以随机幅度应用随机选择的增强。出人意料地有效且简单。
- 测试时增强（**TTA**）：在推理时应用增强并平均预测。以 $K\times$ 推理时间为代价提供免费的精度提升（通常 1-2%）。

11.6 文本数据处理

文本数据需要与图像根本不同的预处理，围绕分词概念展开。

11.6.1 分词

分词将原始文本字符串转换为模型可以处理的离散令牌序列（词 ID 或子词 ID）。

Table 11.1 – 分词策略

策略	示例	优点/缺点
词级	“hello world” → [12, 45]	简单；大词表，OOV 问题
字符级	“hello” → [104, 101, 108, 108, 111]	小词表；长序列，无语义
BPE	“unhappiness” → [“un”, “happy”, “ness”]	良好的平衡；从数据学习
WordPiece	BERT 分词器	类似 BPE；掩码 LM 训练
SentencePiece	统一分词	语言无关；处理任意文本

11.6.2 字节对编码（BPE）

BPE [Sennrich, Haddow, and Birch 2016] 是现代 LLM 中最广泛使用的子词分词算法。它通过迭代合并语料库中最频繁的相邻字节/字符对来构建词表：

1. 从所有单个字符加上词尾令牌开始词表
2. 计算语料库中所有相邻对
3. 将最频繁的对合并为单个令牌
4. 对 N 次合并操作重复（其中 N 是超参数）

词表大小是一个关键超参数：更大的词表将更多单词表示为单个令牌（推理更快）但需要更多嵌入和更高的内存占用。

对于大小为 V 的词表，BPE 需要 $\mathcal{O}(V)$ 空间。现代 LLM 通常使用 $V = 30{,}000$–$100{,}000$ 个令牌。

11.7 音频数据处理

音频数据通常作为频谱图或梅尔频谱图处理。

11.7.1 频谱图计算

短时傅里叶变换（STFT）将时域信号 $x[t]$ 转换为时频表示：

$$X[m,k] = \sum_{n=0}^{N-1} x[n]w[n]e^{-j2\pi kn/N} \tag{11.6}$$

其中 $w[n]$ 是窗口函数（Hann 或 Hamming），N 是 FFT 大小。

梅尔频谱图将频率映射到梅尔刻度，近似人类听觉感知：

$$f_{\text{mel}} = 2595 \log_{10}\left(1 + \frac{f_{\text{Hz}}}{700}\right) \tag{11.7}$$

11.8 数据归一化

归一化标准化输入特征，通过确保所有输入在相似尺度上来稳定训练。

11.8.1 图像归一化

对于 RGB 图像，像素值 $x \in [0, 255]$ 通常归一化到 $[0, 1]$ 或标准化到零均值和单位方差：

$$x_{\text{norm}} = \frac{x - \mu}{\sigma} \tag{11.8}$$

ImageNet 归一化参数：

- 均值：[0.485, 0.456, 0.406]
- 标准差：[0.229, 0.224, 0.225]

这些值是从 ImageNet 训练集计算的。对于自定义数据集，计算特定于数据集的统计量：

```
loader = DataLoader(dataset, batch_size=32)
mean = torch.zeros(3); std = torch.zeros(3)
for images, _ in loader:
    mean += images.mean(dim=[0, 2, 3])
    std += images.std(dim=[0, 2, 3])
mean /= len(loader.dataset)
std /= len(loader.dataset)
```

注意：使用预训练模型时，始终使用模型训练时使用的统计量，即使它们对您的数据不是最佳的。

11.9 数据加载

高效的数据加载防止 GPU 饥饿并最大化训练吞吐量。

代码 11.2 – PyTorch 中的高效数据加载

```
from torch.utils.data import DataLoader, Dataset

```

```python
class ImageDataset(Dataset):
    def __init__(self, data, labels, transform=None):
        self.data = data
        self.labels = labels
        self.transform = transform

    def __len__(self):
        return len(self.data)

    def __getitem__(self, idx):
        image = self.data[idx]
        label = self.labels[idx]
        if self.transform:
            image = self.transform(image)
        return image, label

train_loader = DataLoader(
    train_dataset,
    batch_size=32,
    shuffle=True,         # Important for SGD convergence
    num_workers=4,        # Parallel data loading
    pin_memory=True,      # Faster GPU transfer (CUDA pinning)
    drop_last=True,       # Consistent batch sizes
    persistent_workers=True  # Keep workers alive between epochs
)

# Prefetch next epoch while current epoch trains
val_loader = DataLoader(
    val_dataset,
    batch_size=64,
    shuffle=False,         # No need to shuffle validation
    num_workers=4,
    pin_memory=True
)
```

DataLoader 优化

- **num_workers**：更多 worker = 更快加载，但太多会导致 CPU-GPU 争用。好的起点是 4-8。
- **pin_memory**：固定 CPU 内存以便更快传输到 GPU（避免额外的内存复制）。
- **drop_last**：删除最后一个不完整批次以避免变化的批次大小（对于批量归一化尤其重要）。
- **persistent_workers**：在轮次之间保持 worker 进程存活，减少每轮启动开销。

11.10 处理数据不平衡

类别不平衡在现实世界数据集中普遍存在，如果不解决，会严重降低模型性能。

11.10.1 不平衡比例

常见不平衡比例：

- 轻度（1:10 到 1:100）：标准医学成像、异常检测
- 严重（1:100 到 1:1000）：欺诈检测、罕见疾病诊断
- 极端（<1:1000）：科学应用（粒子物理、天文学）

11.10.2 不平衡数据技术

1. 加权损失函数：为少数类分配更高的损失权重：

$$\mathscr{L}_{\text{weighted}} = -\sum_{k=1}^{K} w_k \cdot y_k \log \hat{y}_k \tag{11.9}$$

   其中 $w_k = N/(K \cdot N_k)$，N_k 是类 k 中的样本数。

2. 过采样少数类：复制少数样本或生成合成样本（表格数据的 SMOTE）。
3. 欠采样多数类：随机删除多数类样本以平衡数据集。
4. **Focal Loss** [T.-Y. Lin, Goyal, et al. 2017]：降低简单样本的权重，将训练重点放在难样本上：

$$\mathscr{L}_{\text{focal}} = -(1 - p_t)^{\gamma} \log p_t \tag{11.10}$$

其中 p_t 是模型对真实类别的预测概率，$\gamma \geq 0$ 控制降权强度。详见第 3 章的详细推导。

11.11 数据质量与版本管理

随着数据集在重要性和复杂性上的增长，系统的数据管理变得必不可少。

11.11.1 数据版本管理

数据版本管理跟踪数据集随时间的变化，实现：

- 可重复性：不同时间点使用完全相同的数据进行训练
- 溯源跟踪：理解哪个模型在哪个版本的数据上训练
- 回滚：如果发现问题则回滚到以前的数据版本
- 实验比较：测试关于数据质量的假设

像 **DVC**（数据版本控制）这样的工具扩展 Git 以处理大文件和 ML 流水线：

```
dvc init
dvc add data/
git add data.dvc
git commit -m "Add dataset v1"
# Later, update and version:
dvc add data/
git add data.dvc
git commit -m "Update to dataset v2"
```

DVC 将实际数据存储在远程存储（S3、GCS、Azure Blob）中，而 Git 跟踪元数据和哈希，实现高效协作。

11.11.2 数据溯源

理解数据溯源回答：这个样本从哪里来？谁标注的？什么时候添加的？

关键溯源元数据：

- 来源 URL 或收集方法
- 收集/标注时间戳
- 标注者 ID 及其资质
- 标注质量分数（如果使用共识标注）

- 处理流水线版本（应用的增强、过滤器）

此元数据对于调试模型失败、审计偏差以及在高风险领域（医疗保健、金融、法律）满足监管要求至关重要。

11.12 评估指标

虽然第 3 章涵盖了损失函数，但本节讨论特定任务的评估指标。

11.12.1 分类指标

- 准确率：正确预测的比例。简单但对不平衡数据有误导。
- 精确率、召回率、**F1**：对于二分类：

$$\text{精确率} = \frac{TP}{TP + FP} \quad \text{（预测为正的中有多少正确？）} \tag{11.11}$$

$$\text{召回率} = \frac{TP}{TP + FN} \quad \text{（实际为正的中找到多少？）} \tag{11.12}$$

$$F_1 = \frac{2 \cdot \text{精确率} \cdot \text{召回率}}{\text{精确率} + \text{召回率}} \tag{11.13}$$

- 宏/微 **F1**：宏平均先计算每类 F1 再平均；微平均先跨类计算 TP/FP/FN 再计算 F1。
- **AUC-ROC**：ROC 曲线下面积。与阈值无关；衡量排序质量。

11.12.2 分割指标

- 交并比（**IoU**）：$\text{IoU} = \frac{|A \cap B|}{|A \cup B|}$
- **Dice** 系数：$\text{Dice} = \frac{2|A \cap B|}{|A| + |B|} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}}$
- 像素准确率：正确分类像素的比例。

注意：IoU 和 Dice 相关：$\text{Dice} = \frac{2 \cdot \text{IoU}}{1 + \text{IoU}}$，所以优化一个近似优化另一个。

11.13 章节小结

1. 数据质量是模型性能的基础；即使是最好的架构也无法克服糟糕的数据
2. 正确的训练/验证/测试划分防止泄露并提供无偏的性能估计
3. 交叉验证（K 折）在数据有限时提供稳健估计

4. 数据增强通过随机变换人为扩展有效数据集
5. Mixup 和 CutMix 通过凸组合或补丁粘贴创建虚拟训练样本，提供强正则化
6. 文本数据需要在模型输入前进行分词（BPE、WordPiece）
7. 归一化通过标准化输入尺度稳定训练
8. 带多个 worker 和固定内存的高效数据加载最大化 GPU 利用率
9. 类别不平衡需要通过加权损失或 focal loss 进行特殊处理
10. 数据版本管理和溯源跟踪对于可重复的 ML 研究和生产系统必不可少
11. 评估指标应匹配下游任务要求；对于不平衡或多类问题，仅准确率是不够的

11.14 延伸阅读

- [I. Goodfellow, Bengio, and Courville 2016] — 深度学习章节，数据处理部分
- [Sennrich, Haddow, and Birch 2016] — 带有子词单元的神经机器翻译
- [T.-Y. Lin, Goyal, et al. 2017] — 密集目标检测的 Focal Loss
- [Chollet 2017] — Python 深度学习，第 4 章预处理流水线
- [H. Zhang et al. 2018] — mixup：超越经验风险最小化
- [Cubuk, Zoph, Mane, et al. 2019] — AutoAugment：从数据学习增强策略
- [Yun et al. 2019] — CutMix：训练强分类器与可定位特征的正则化策略
- [Cubuk, Zoph, Shlens, et al. 2020] — RandAugment：实用自动化数据增强与减少的搜索空间
- DVC 文档：`https://dvc.org/` — ML 项目的数据版本控制

11.15 练习题

1. 数据泄露：在图像分类或表格 ML 流水线中给出两个数据泄露的例子。为什么泄露可能使验证准确率误导性地高？
2. 划分策略：您有 12,000 个标注样本和 5 个类别，中等不平衡。设计一个训练/验证/测试划分策略，并解释是否需要分层。
3. **Mixup** 标签：如果两个具有独热标签 y_a 和 y_b 的图像以 $\lambda = 0.3$ 组合，写出用于训练的新目标标签。

4. 增强策略：比较 AutoAugment、RandAugment 和 TrivialAugment Wide。当策略搜索的计算受限时，哪个最容易部署？
5. 分词：解释为什么子词分词（BPE/WordPiece）通常比纯词级分词对开放词汇 NLP 效果更好。
6. 输入流水线效率：为什么多个 data-loader worker 和固定内存通常可以提高 GPU 利用率？什么时候它们可能无法帮助？

Chapter 12

Transformer 架构

12.1 引言

Transformer 架构，由 [Vaswani et al. 2017] 引入，从根本上改变了序列建模和表示学习的格局。通过用自注意力机制取代循环，Transformer 实现了跨整个序列的并行计算，显著提高了训练效率，并使得扩展到数十亿参数成为可能。

本章提供 Transformer 架构的全面论述，从基本的注意力机制到现代大型语言模型。

12.2 注意力机制

12.2.1 背景：RNN 的局限性

在 Transformer 出现之前，序列模型依赖于循环神经网络（RNN）及其变体（LSTM、GRU）。虽然有效，但 RNN 存在以下问题：

- 顺序计算：无法跨时间步并行化
- 长程依赖：通过许多步骤的梯度传播导致梯度消失/爆炸
- 计算效率低：每个时间步需要从内存加载隐藏状态

12.2.2 缩放点积注意力

基本构建块是缩放点积注意力：

$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V} \tag{12.1}$$

其中：

- $\mathbf{Q} \in \mathbb{R}^{n \times d_k}$：查询矩阵
- $\mathbf{K} \in \mathbb{R}^{m \times d_k}$：键矩阵
- $\mathbf{V} \in \mathbb{R}^{m \times d_v}$：值矩阵
- d_k：键的维度（也是查询维度）

为什么要缩放？

缩放因子 $\sqrt{d_k}$ 防止点积变得很大，否则会推动 softmax 进入梯度极小的区域。当 d_k 很大时，点积往往很大，导致 softmax 饱和（输出接近独热）。

12.2.3 自注意力

自注意力是将注意力应用于查询、键和值都来自同一源的序列：

$$\mathbf{X}_{\text{out}} = \text{Attention}(\mathbf{X}\mathbf{W}_Q, \mathbf{X}\mathbf{W}_K, \mathbf{X}\mathbf{W}_V) \tag{12.2}$$

其中 $\mathbf{W}_Q, \mathbf{W}_K, \mathbf{W}_V$ 是学习的投影矩阵。
关键特性：

- 全到全连接：每个位置可以关注所有其他位置
- 恒定路径长度：信息以 $O(1)$ 步流动，与距离无关
- 计算复杂度：对于序列长度 n 和维度 d 为 $O(n^2 \cdot d)$

12.3 多头注意力

不是执行单一的注意力函数，而是用学习的线性投影将查询、键和值投影 h 次：

$$\text{MultiHead}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Concat}\left(\text{head}_1, \ldots, \text{head}_h\right) \mathbf{W}^O \tag{12.3}$$

其中每个头是：

$$\text{head}_i = \text{Attention}\left(\mathbf{Q}\mathbf{W}_i^Q, \mathbf{K}\mathbf{W}_i^K, \mathbf{V}\mathbf{W}_i^V\right) \tag{12.4}$$

典型配置：$h = 8$ 个头，$d_k = d_v = d_{\text{model}}/h$。

多头注意力的优势

- 每个头可以关注表示的不同方面
- 不同的头可能专门处理：位置、句法、语义信息
- 模型可以联合关注来自不同表示子空间的信息

12.4 Transformer 架构

12.4.1 前馈网络

每层包括一个位置前馈网络：

$$\mathrm{FFN}(\mathbf{x}) = \mathbf{W}_2\,\sigma\left(\mathbf{W}_1\mathbf{x} + \mathbf{b}_1\right) + \mathbf{b}_2 \tag{12.5}$$

通常实现为两个带 ReLU 激活的线性变换。内层维度通常为 $4 \times d_{\mathrm{model}}$。

12.4.2 层归一化

层归一化通过归一化激活来稳定训练：

$$\mathbf{y} = \frac{\mathbf{x} - \mu}{\sqrt{\sigma^2 + \epsilon}} \odot \gamma + \beta \tag{12.6}$$

其中 $\mu = \frac{1}{d}\sum_{i=1}^{d} x_i$，$\sigma^2 = \frac{1}{d}\sum_{i=1}^{d}(x_i - \mu)^2$。

存在两种变体：

- 后 **LN**（原始）：残差连接后的 LayerNorm
- 前 **LN**：在注意力/FFN 之前的 LayerNorm（对深度网络更稳定）

12.4.3 残差连接

每个子层都有一个残差连接：

$$\mathbf{x}_{\mathrm{out}} = \mathbf{x} + \mathrm{SubLayer}(\mathbf{x}) \tag{12.7}$$

这使得梯度可以通过跳跃连接流动，并允许网络在有益时学习恒等映射。

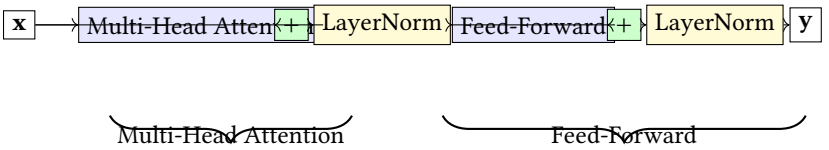


Figure 12.1 – 带残差连接和层归一化的 Transformer 层。

12.5 位置编码

由于自注意力是排列不变的（对输入顺序不变），我们必须注入位置信息：

$$\mathrm{PE}_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{12.8}$$

$$\mathrm{PE}_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{\text{model}}}}\right) \tag{12.9}$$

其中 pos 是位置，i 是维度。

位置编码特性

- 线性关系： $\mathrm{PE}(pos+k)$ 可以表示为 $\mathrm{PE}(pos)$ 的线性函数
- 泛化：可以编码超出训练序列长度的位置
- 相对位置：旋转位置嵌入（RoPE）在注意力计算中更直接地编码相对位置

替代方案：旋转位置嵌入（RoPE）[Su et al. 2021] 用于 LLaMA 和 GPT-NeoX，在注意力计算中更直接地编码相对位置。

12.6 编码器-解码器架构

原始 Transformer 使用编码器-解码器结构：

- 编码器： N 个处理输入序列的相同层
- 解码器： N 个带跨编码器输出的交叉注意力的相同层
- 交叉注意力：解码器的查询关注编码器的键/值

代码 12.1 – PyTorch 多头注意力实现

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import math

class MultiHeadAttention(nn.Module):
    def __init__(self, d_model, num_heads):
        super().__init__()
        assert d_model % num_heads == 0

        self.d_model = d_model
        self.num_heads = num_heads
        self.d_k = d_model // num_heads

        self.W_q = nn.Linear(d_model, d_model)
```

```python
        self.W_k = nn.Linear(d_model, d_model)
        self.W_v = nn.Linear(d_model, d_model)
        self.W_o = nn.Linear(d_model, d_model)

    def split_heads(self, x, batch_size):
        """Split the last dimension into num_heads."""
        x = x.view(batch_size, -1, self.num_heads, self.d_k)
        return x.permute(0, 2, 1, 3)  # (batch, heads, seq, d_k)

    def forward(self, query, key, value, mask=None):
        batch_size = query.size(0)

        # Linear projections
        Q = self.W_q(query)
        K = self.W_k(key)
        V = self.W_v(value)

        # Split heads
        Q = self.split_heads(Q, batch_size)
        K = self.split_heads(K, batch_size)
        V = self.split_heads(V, batch_size)

        # Scaled dot-product attention
        scores = torch.matmul(Q, K.transpose(-2, -1)) / math.sqrt(self.d_k)

        if mask is not None:
            scores = scores.masked_fill(mask == 0, -1e9)

        attention_weights = F.softmax(scores, dim=-1)
        attention_output = torch.matmul(attention_weights, V)

        # Concatenate heads
        attention_output = attention_output.permute(0, 2, 1, 3).contiguous()
        attention_output = attention_output.view(batch_size, -1, self.d_model)

        # Final linear projection
        output = self.W_o(attention_output)

        return output, attention_weights
```

12.7 Vision Transformer（ViT）

Vision Transformer [Dosovitskiy et al. 2021] 通过将图像块作为令牌来处理，将 Transformer 架构应用于图像分类：

1. 将图像分割为 16×16 块
2. 展平每个块并投影到 d_{model} 维度
3. 添加 [CLS] 令牌（用于分类）和位置嵌入
4. 通过标准 Transformer 编码器处理
5. 使用最终层的 [CLS] 令牌输出进行分类

计算复杂度：$O(n^2)$，其中 $n = \frac{H \times W}{P^2}$ 是块的数量。

12.8 大型语言模型

12.8.1 GPT 架构

GPT（生成式预训练 Transformer）[Radford et al. 2019; Brown et al. 2020] 使用仅解码器架构：

- 因果（掩码）自注意力防止关注未来令牌
- 语言建模目标：预测下一个令牌
- 在大型文本语料库上预训练，然后微调

GPT-3 扩展到 1750 亿参数，展示了涌现能力。

12.8.2 BERT 架构

BERT [Devlin et al. 2019] 使用带双向注意力的仅编码器架构：

- 掩码语言建模（MLM）：预测掩码令牌
- 下一句预测（NSP）：预测一个句子是否跟随另一个句子
- 实现深度双向表示

12.8.3 现代 LLM

现代大型语言模型结合了几项创新：

关键架构差异：

- **RoPE**（旋转位置嵌入）：在旋转矩阵中编码位置，对相对位置更好
- 分组查询注意力（**GQA**）：多个查询头共享键/值头，减少内存
- **SwiGLU** 激活：用于 LLaMA [Touvron, Lavril, et al. 2023]，比 GeGLU 更好
- **Flash Attention**：IO 感知的精确注意力算法 [Dao 2022]，将内存从 $O(n^2)$ 减少到 $O(n)$

Table 12.1 – 现代 LLM 架构比较

模型	架构	位置编码	关键特性
GPT-2	仅解码器	学习到的	Unigram 分词
GPT-3	仅解码器	学习到的	上下文学习
LLaMA	仅解码器	RoPE	高效架构
ChatGLM	Prefix-LM	RoPE	双语预训练
Mistral	仅解码器	RoPE	分组查询注意力

12.9 效率考量

12.9.1 注意力复杂度

标准注意力在序列长度上有 $O(n^2)$ 复杂度，对于长序列变得难以承受：

- **Flash Attention**：近似注意力与 IO 感知，2-4 倍加速
- 稀疏注意力：仅关注位置的子集（Longformer、BigBird）
- 线性注意力：基于核的近似，复杂度 $O(n)$（Performer）
- 低秩注意力：分解注意力矩阵（Linformer）

12.9.2 内存优化

对于大型模型，内存变得至关重要：

- 梯度检查点：以计算换内存
- 混合精度：FP16/BF16 训练减少 2 倍内存
- 量化：INT8/INT4 用于推理
- 分页注意力：KV 缓存的内存分页（vLLM）

12.10 章节小结

1. 自注意力实现并行计算，以 $O(1)$ 路径长度捕获长程依赖
2. 带 $\sqrt{d_k}$ 缩放的缩放点积注意力防止梯度饱和
3. 多头注意力允许关注不同的表示子空间
4. 残差连接和层归一化实现深度网络的稳定训练
5. 由于注意力是排列不变的，Transformer 必须注入位置信息
6. Vision Transformer（ViT）通过块嵌入将 Transformer 架构应用于图像

7. 现代 LLM（GPT、BERT、LLaMA）将 Transformer 扩展到数十亿参数
8. 效率技术（Flash Attention、稀疏注意力、量化）实现大型模型的实用部署

12.11 延伸阅读

- [Vaswani et al. 2017] — Attention Is All You Need（原始 Transformer 论文）
- [Devlin et al. 2019] — BERT: 深度双向 Transformer 的语言理解预训练
- [Radford et al. 2019] — 语言模型是无监督多任务学习器（GPT-2）
- [Brown et al. 2020] — 语言模型是少样本学习器（GPT-3）
- [Dosovitskiy et al. 2021] — 一张图值 16x16 个词：大规模图像识别的 Transformer（ViT）
- [Su et al. 2021] — RoFormer: 带旋转位置嵌入的增强 Transformer
- [Dao 2022] — FlashAttention: 带 IO 感知的快速内存高效精确注意力
- [Hoffmann et al. 2022] — 训练计算最优大型语言模型（Chinchilla 缩放定律）
- [J. Wei et al. 2022] — 大型语言模型的涌现能力
- [Touvron, Lavril, et al. 2023] — LLaMA: 开放高效的基础语言模型

12.12 练习题

1. 注意力矩阵形状：对于序列长度 $n = 128$、模型维度 $d_{model} = 512$ 和 8 个头，**Q**、**K** 和 **V** 每个头的形状是什么？
2. 为什么要按 $\sqrt{d_k}$ 缩放？当点积变得非常大时，解释 softmax 分布会发生什么。
3. 位置编码：为什么 Transformer 必须注入位置信息，而 RNN 不需要单独的位置编码机制？
4. **ViT** 令牌： 224×224 图像被分割为 14×14 块。在添加 [CLS] 令牌后，Transformer 处理多少个令牌？
5. 编码器 **vs.** 解码器：从掩码模式、架构类型和预训练目标比较 BERT 和 GPT。
6. 效率权衡：为什么 Flash Attention 可以加速精确注意力而不改变注意力操作的数学结果？

Chapter 13

现代训练技术

13.1 引言

训练现代深度学习系统涉及的远不止梯度下降。本章涵盖使在大规模数据集上训练、在任务间传递知识、实现最先进性能的高级训练技术。这些技术对于任何从事当代神经网络工作的人都是必不可少的。

13.2 预训练与微调

构建强大模型的主导范式是预训练后接微调。

13.2.1 预训练目标

预训练从大型无标签数据学习通用表示：

- 语言建模（**CLM**）：给定前面的令牌预测下一个令牌
- 掩码语言建模（**MLM**）：预测随机掩码的令牌（BERT 风格）[Devlin et al. 2019]
- 去噪自编码器：重构损坏的输入
- 下一句预测（**NSP**）：预测句子顺序

对于视觉：

- 对比学习：最大化正样本对之间的相似度 [Ting Chen et al. 2020]
- 掩码自编码（**MAE**）：重构掩码块 [He, X. Chen, et al. 2022a]
- 自蒸馏：学生从教师学习

13.2.2 微调策略

预训练后，模型适应特定任务：

1. 全量微调：在目标任务数据上更新所有参数
2. 线性探测：在冻结特征上仅训练线性分类器
3. 适配器方法：在层之间添加小型可训练模块

4. **LoRA**（低秩适应）：优化权重矩阵的低秩分解

何时使用每种方法

- 丰富的任务数据：全量微调通常最好
- 有限的任务数据：LoRA/适配器防止过拟合
- 多任务：适配器方法实现参数高效的多任务学习

13.3 LoRA：低秩适应

LoRA [E. J. Hu et al. 2022] 通过分解权重更新来减少可训练参数：

$$\mathbf{W} \leftarrow \mathbf{W} + \Delta\mathbf{W}, \quad \Delta\mathbf{W} = \mathbf{B}\mathbf{A} \tag{13.1}$$

其中 $\mathbf{B} \in \mathbb{R}^{d\times r}$，$\mathbf{A} \in \mathbb{R}^{r\times k}$，$r \ll \min(d,k)$。
前向传播时（输入 $\mathbf{x}$）：

$$\mathbf{h} = \mathbf{W}\mathbf{x} + \frac{\alpha}{r}\mathbf{B}\mathbf{A}\mathbf{x} \tag{13.2}$$

其中 α 是缩放因子（通常等于秩 r）。

代码 **13.1** – PyTorch 中的 LoRA 实现

```python
import torch
import torch.nn as nn

class LoRALinear(nn.Module):
    def __init__(self, in_features, out_features, rank=4, alpha=4):
        super().__init__()
        self.rank = rank
        self.alpha = alpha
        self.scaling = alpha / rank

        # Original frozen weights
        self.weight = nn.Parameter(
            torch.randn(out_features, in_features),
            requires_grad=False
        )

        # Trainable low-rank matrices
        self.lora_A = nn.Parameter(torch.randn(rank, in_features))
        self.lora_B = nn.Parameter(torch.zeros(out_features, rank))

        # Initialize A with random Gaussian, B with zeros
        nn.init.normal_(self.lora_A, std=1.0 / rank)

```

```
    def forward(self, x):
        # Original forward pass
        h = F.linear(x, self.weight)
        # LoRA update
        h += self.scaling * (x @ self.lora_A.T @ self.lora_B.T)
        return h

# Usage in a model
class LoRALinearLayer(nn.Module):
    def __init__(self, linear_layer, rank=4, alpha=4):
        super().__init__()
        self.linear = linear_layer
        self.lora = LoRALinear(
            linear_layer.in_features,
            linear_layer.out_features,
            rank=rank,
            alpha=alpha
        )

    def forward(self, x):
        return self.lora(x)
```

可训练参数从 $d \times k$ 减少到 $r \times (d + k)$。

13.4 迁移学习

迁移学习利用相关任务的知识。

13.4.1 特征迁移

在一个领域学习的特征通常可以很好地迁移：

- ImageNet 特征迁移到医学成像、卫星图像
- BERT 特征跨 NLP 任务迁移
- 关键洞察：较低层捕获通用特征；较高层捕获任务特定模式

13.4.2 渐进式解冻

微调期间逐渐解冻层：

1. 仅训练分类头（冻结骨干）
2. 解冻最后 n 层，微调
3. 可选：解冻剩余层

这防止预训练特征的灾难性遗忘。

13.5 多任务学习

同时在多个任务上训练单个模型。

13.5.1 硬参数共享

跨任务共享编码器权重：

$$\mathscr{L}_{\text{total}} = \sum_{t=1}^{T} \lambda_t \mathscr{L}_t(\theta_{\text{shared}}, \theta_t) \tag{13.3}$$

其中 λ_t 是任务权重，θ_t 是任务特定头。

13.5.2 软参数共享

每个任务有自己的参数，用正则化鼓励相似性：

$$\mathscr{L}_{\text{total}} = \sum_{t=1}^{T} \mathscr{L}_t(\theta_t) + \lambda \sum_{i,j} \|\theta_i^{(l)} - \theta_j^{(l)}\|^2 \tag{13.4}$$

13.5.3 任务平衡

多任务学习需要仔细的任务加权：

- 等权重：简单但可能不是最优的
- 不确定性加权：按逆任务不确定性加权 [Kendall, Gal, and Cipolla 2018]
- 梯度平衡：跨任务平衡梯度大小
- 动态加权：训练期间调整权重

13.6 持续学习

在新任务上顺序训练而不遗忘之前的任务。

13.6.1 灾难性遗忘

神经网络在训练新任务时倾向于遗忘之前的任务。缓解策略：

1. 经验回放：存储之前任务的样本并在训练新任务时重放

2. 弹性权重巩固（**EWC**）：用正则化保护重要权重 [Kirkpatrick et al. 2017]
3. 渐进网络：为新任务添加新列同时冻结旧的
4. 架构增长：为新任务扩展网络容量

13.6.2 EWC 正则化

$$\mathscr{L}_{\text{EWC}} = \mathscr{L}_B(\theta) + \frac{\lambda}{2} \sum_i \mathscr{F}_i(\theta_i - \theta^*_{A,i})^2 \tag{13.5}$$

其中 $\mathscr{F}$ 是 Fisher 信息矩阵，捕获参数重要性。

13.7 自监督学习

使用前置任务从无标签数据学习表示。

13.7.1 对比学习

最大化同一图像的不同增强视图之间的一致性：

$$\mathscr{L} = -\log \frac{\exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_j)/\tau)}{\sum_{k=1}^{2N} \mathbf{1}_{[k \neq i]} \exp(\text{sim}(\mathbf{z}_i, \mathbf{z}_k)/\tau)} \tag{13.6}$$

其中 τ 是温度，$\mathbf{z}$ 是归一化嵌入。
流行的对比学习框架：

- **SimCLR**：带强增强的简单框架
- **MoCo**：动量对比以获得大型负样本
- **SwAV**：用聚类分配代替成对比较

13.7.2 掩码图像建模

图像的 BERT 风格掩码（MAE、BEiT）：

- 掩码随机块（MAE 为 75%）
- 解码器从可见块重构掩码块
- 编码器仅在可见块上操作（高度高效）

13.8 蒸馏与压缩

从大型模型向小型模型传递知识。

13.8.1 知识蒸馏

较小的学生模型从较大的教师学习：

$$\mathscr{L}_{\text{distill}} = \alpha \cdot \text{CE}(y, \text{softmax}(z_s/T)) + (1-\alpha) \cdot \text{CE}(y, \text{softmax}(z_t/T)) \tag{13.7}$$

其中 z_s 和 z_t 是学生和教师 logits，T 是温度。
关键洞察：来自教师的软目标包含比硬标签更多信息。

13.8.2 自蒸馏

模型将知识蒸馏到自己：

- 训练模型 A，然后用 A 的预测训练同大小的模型 B
- BYOL、SimSiam：带预测网络的两个视图
- 抖出噪声并产生更锐利的预测

13.9 大批量训练

扩展到数千个 GPU 需要仔细考虑批次大小。

13.9.1 线性缩放规则

当增加批次大小 B 时，按比例缩放学习率：

$$\alpha_{\text{new}} = \alpha_{\text{old}} \times \frac{B_{\text{new}}}{B_{\text{old}}} \tag{13.8}$$

Goyal 等人 [Goyal et al. 2017] 在 8K 批次大小下实现了 ImageNet 上 256 路数据并行。

13.9.2 预热与缩放

对于非常大的批次（>16K）：

- 渐进式预热以防止早期不稳定
- 分层学习率衰减（LLRD）：较低层用较低 LR
- 更长训练（相应调整调度）

13.10 高效训练技术

13.10.1 梯度检查点

通过在反向传播期间重新计算激活以计算换内存：

- 仅在选定的检查点存储激活
- 在反向期间重新计算前向传播段
- 内存减少：$O(\sqrt{n})$，计算开销 $O(n)$

13.10.2 卸载

在 CPU 和 GPU 内存之间移动张量：

- CPU 卸载：参数保存在 CPU 上，每层加载到 GPU
- ZeRO（零冗余优化器）：跨 GPU 划分优化器状态
- 激活卸载：流水线重新计算

13.11 调试训练

13.11.1 常见问题与解决方案

Table 13.1 – 常见训练问题与解决方案

症状	可能原因	解决方案
损失 NaN	大梯度	降低 LR，梯度裁剪
损失不下降	学习率错误	LR 范围测试，降低 LR
过拟合	数据太少，参数太多	正则化，增强
欠拟合	模型太简单	更大模型，更多特征
灾难性遗忘	单任务训练	EWC，回放，渐进式
初始化时不稳定	初始化差	Kaiming/He 初始化，正确归一化

13.11.2 训练配方

经过验证的常见场景配方：

Algorithm 4 视觉模型训练配方

1. 从小学习率开始（Adam 为 1e-4，SGD 为 0.1）
2. 使用 LR 范围测试找到最优 LR
3. 应用带预热的余弦退火调度
4. 使用强增强（RandAugment、Mixup、CutMix）
5. 标签平滑（分类为 0.1）
6. 权重衰减（AdamW 为 0.01）
7. 权重的指数移动平均（EMA，衰减 =0.9999）
8. 在验证损失上早停

Algorithm 5 LLM 训练配方

1. 在大型语料库上预训练（GPT-3 规模为万亿 + 令牌）
2. 使用混合精度（BF16）和梯度检查点
3. 应用梯度裁剪（1.0）
4. 使用学习率预热（2000-10000 步）
5. 余弦衰减到小 LR（通常为峰值的 10%）
6. 权重衰减（LLaMA-2 为 0.1）
7. 微调期间使用注意力 dropout
8. LoRA 用于下游任务的高效微调

13.12 章节小结

1. 在大型数据集上预训练后接微调是主导范式
2. LoRA 通过分解权重更新实现参数高效微调
3. 迁移学习允许跨领域利用表示
4. 多任务学习需要仔细的任务平衡策略
5. 持续学习技术防止灾难性遗忘
6. 自监督学习（对比、掩码）从无标签数据学习

7. 知识蒸馏将能力从大型模型传递到小型模型
8. 大批量训练受益于线性学习率缩放和预热
9. 系统调试和经过验证的配方加速开发

13.13 延伸阅读

- [G. Hinton, Vinyals, and Jeff Dean 2015] — 蒸馏神经网络中的知识
- [Goyal et al. 2017] — 准确的大批次 SGD：一小时内训练 ImageNet
- [Kirkpatrick et al. 2017] — 克服神经网络中的灾难性遗忘
- [Devlin et al. 2019] — BERT：深度双向 Transformer 的预训练
- [Kendall, Gal, and Cipolla 2018] — 使用不确定性权衡多任务学习的损失
- [Ting Chen et al. 2020] — 对比学习的一个简单框架（SimCLR）
- [E. J. Hu et al. 2022] — LoRA：大型语言模型的低秩适应
- [He, X. Chen, et al. 2022a] — 掩码自编码器是可扩展的视觉学习器（MAE）
- [Rombach et al. 2022] — 带潜在扩散模型的高分辨率图像合成
- [P. Wei and Mckeown 2022] — 链式思考提示引出大型语言模型的推理

13.14 练习题

1. 微调策略：对于下游任务数据有限且 GPU 内存有限的情况，比较全量微调、线性探测和 LoRA。
2. **LoRA** 参数计数：线性层形状为 4096×4096，LoRA 秩 $r = 8$。比较全量微调与 LoRA 中可训练参数的数量。
3. 多任务加权：为什么等权重在多任务学习中可能失败？不确定性加权如何解决这个问题？
4. 持续学习：解释弹性权重巩固背后的直觉。Fisher 信息矩阵扮演什么角色？
5. 自监督学习：对比对比学习与掩码图像建模。每种方法从无标签数据创建什么监督信号？
6. 大批量训练：如果您将批次大小增加 8 倍，线性缩放规则建议学习率怎么做，为什么仍然需要预热？

Bibliography

[1] Carl Eckart and Gale Young. "The approximation of one matrix by another of lower rank". In: *Psychometrika* 1.3 [1936], pp. 211–218. DOI: 10.1007/BF02288367.

[2] Claude E Shannon. "A mathematical theory of communication". In: *The Bell System Technical Journal* 27.3 [1948], pp. 379–423. DOI: 10.1002/j.1538-7305.1948.tb01338.x.

[3] Herbert Robbins and Sutton Monro. "A stochastic approximation method". In: *The Annals of Mathematical Statistics* 22.3 [1951], pp. 400–407. DOI: 10.1214/aoms/1177729586.

[4] Frank Rosenblatt. "The perceptron: A probabilistic model for information storage and organization in the brain". In: *Psychological Review* 65.6 [1958], pp. 386–408. DOI: 10.1037/h0042519.

[5] Bernard Widrow and Marcian E Hoff. "Adaptive "ADALINE" neuron using chemical "memistors"". In: 3 [1960]. Reprinted in [Widrow and Stearns 1985], pp. 1554–1556.

[6] Peter J Huber. "Robust estimation of a location parameter". In: *The Annals of Mathematical Statistics* 35.1 [1964], pp. 73–101.

[7] Marvin Minsky and Seymour Papert. *Perceptrons: An Introduction to Computational Geometry*. MIT Press, 1969.

[8] Bernard Widrow and Samuel D Stearns. *Adaptive Signal Processing*. Prentice-Hall, Englewood Cliffs, NJ, 1985.

[9] David E Rumelhart, Geoffrey E Hinton, and Ronald J Williams. "Learning representations by back-propagating errors". In: *Nature* 323.6088 [1986], pp. 533–536. DOI: 10.1038/323533a0.

[10] Yann LeCun, Bernhard Boser, et al. "Backpropagation applied to handwritten zip code recognition". In: *Neural Computation* 1.4 [1989], pp. 541–551. DOI: 10.1162/neco.1989.1.4.541.

[11] Kurt Hornik. "Approximation capabilities of multilayer feedforward networks". In: *Neural Networks* 4.2 [1991], pp. 251–257. DOI: 10.1016/0893-6080(91)90009-T.

[12] Yoshua Bengio, Patrice Simard, and Paolo Frasconi. "Learning long-term dependencies with gradient descent is difficult". In: *IEEE Transactions on Neural Networks* 5.2 [1994], pp. 157–166. DOI: 10.1109/72.279181.

[13] Sepp Hochreiter and Jürgen Schmidhuber. "Long short-term memory". In: *Neural Computation* 9.8 [1997], pp. 1735–1780. DOI: 10.1162/neco.1997.9.8.1735.

[14] Yann LeCun, Léon Bottou, et al. "Gradient-based learning applied to document recognition". In: *Proceedings of the IEEE* 86.11 [1998], pp. 2278–2324. DOI: 10.1109/5.726791.

[15] Christopher M Bishop. *Pattern Recognition and Machine Learning*. Information Science and Statistics. Springer, 2006. ISBN: 978-0387310732.

[16] Thomas M Cover and Joy A Thomas. *Elements of Information Theory*. 2nd ed. Hoboken, NJ: Wiley-Interscience, 2006. ISBN: 978-0471241959.

[17] Geoffrey E Hinton and Ruslan R Salakhutdinov. "Reducing the dimensionality of data with neural networks". In: *Science* 313.5786 [2006], pp. 504–507. DOI: 10.1126/science.1127647.

[18] Yoshua Bengio and Yann LeCun. "Scaling learning algorithms towards AI". In: *Large-Scale Kernel Machines*. Ed. by Léon Bottou et al. Cambridge, MA: MIT Press, 2007, pp. 321–359.

[19] Simon O Haykin. *Neural Networks and Learning Machines*. 3rd ed. Pearson, 2009. ISBN: 978-0131471399.

[20] Gilbert Strang. *Introduction to Linear Algebra*. 4th ed. Wellesley, MA: Wellesley-Cambridge Press, 2009. ISBN: 978-0980232714.

[21] Xavier Glorot and Yoshua Bengio. "Understanding the difficulty of training deep feedforward neural networks". In: *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*. Sardinia, Italy: PMLR, 2010, pp. 249–256.

[22] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E Hinton. "ImageNet classification with deep convolutional neural networks". In: *Advances in Neural Information Processing Systems 25*. Red Hook, NY: Curran Associates, Inc., 2012, pp. 1097–1105.

[23] Kevin P Murphy. *Machine Learning: A Probabilistic Perspective*. Adaptive Computation and Machine Learning. MIT Press, 2012. ISBN: 978-0262018029.

[24] Tijmen Tieleman and Geoffrey Hinton. *Lecture 6.5-rmsprop: Divide the gradient by a running average of its recent magnitude*. COURSERA: Neural Networks for Machine Learning. 2012.

[25] Matthew D Zeiler. "ADADELTA: An adaptive learning rate method". In: *arXiv preprint arXiv:1212.5701* [2012]. arXiv: 1212.5701 [cs.LG].

[26] Razvan Pascanu, Tomas Mikolov, and Yoshua Bengio. "On the Difficulty of Training Recurrent Neural Networks". In: *International Conference on Machine Learning (ICML)* [2013], pp. 1310–1318.

[27] Dzmitry Bahdanau, Kyunghyun Cho, and Yoshua Bengio. "Neural machine translation by jointly learning to align and translate". In: [2014]. arXiv: 1409.0473 [cs.CL].

[28] Kyunghyun Cho et al. "On the properties of neural machine translation: Encoder-decoder approaches". In: *Proceedings of SSST-8, 8th Workshop on Syntax, Semantics and Structure in Statistical Translation*. Stroudsburg, PA: Association for Computational Linguistics, 2014, pp. 103–111. DOI: 10.3115/v1/W14-4012.

[29] Junyoung Chung et al. "Empirical evaluation of gated recurrent neural networks on sequence modeling". In: *NIPS 2014 Workshop on Recurrent Neural Networks*. arXiv:1412.3555. 2014.

[30] Ross Girshick et al. "Rich feature hierarchies for accurate object detection and semantic segmentation". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2014, pp. 580–587. DOI: 10.1109/CVPR.2014.81.

[31] Ian J Goodfellow et al. "Generative adversarial networks". In: *Advances in Neural Information Processing Systems 27*. Curran Associates, Inc., 2014, pp. 2672–2680.

[32] Diederik P Kingma and Max Welling. "Auto-encoding variational Bayes". In: [2014]. Originally presented at the NIPS 2013 Workshop on Deep Learning. arXiv: 1312.6114 [stat.ML].

[33] Guido F Montúfar et al. "On the Number of Linear Regions of Deep Neural Networks". In: *Advances in Neural Information Processing Systems 27 (NeurIPS)*. 2014, pp. 2924–2932.

[34] Danilo Jimenez Rezende, Shakir Mohamed, and Daan Wierstra. "Stochastic backpropagation and approximate inference in deep generative models". In: *Proceedings of the 31st International Conference on Machine Learning*. Beijing, China: PMLR, 2014, pp. 1278–1286. DOI: 10.5555/3044805.3044940.

[35] Nitish Srivastava et al. "Dropout: A simple way to prevent neural networks from overfitting". In: *Journal of Machine Learning Research* 15.56 [2014], pp. 1929–1958.

[36] Anna Choromanska et al. "The loss surfaces of multilayer networks". In: *Proceedings of the 18th International Conference on Artificial Intelligence and Statistics* [2015], pp. 192–204.

[37] Djork-Arné Clevert, Thomas Unterthiner, and Sepp Hochreiter. "Fast and Accurate Deep Network Learning by Exponential Linear Units (ELUs)". In: *arXiv preprint arXiv:1511.07289* [2015]. arXiv: 1511.07289 [cs.NE].

[38] Kaiming He, Xiangyu Zhang, et al. "Delving deep into rectifiers: Surpassing human-level performance on ImageNet classification". In: *Proceedings of the IEEE International Conference on Computer Vision*. IEEE, 2015, pp. 1026–1034. DOI: 10.1109/ICCV.2015.123.

[39] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. "Distilling the knowledge in a neural network". In: *arXiv preprint arXiv:1503.02531* [2015]. arXiv: 1503.02531 [stat.ML].

[40] Sergey Ioffe and Christian Szegedy. "Batch normalization: Accelerating deep network training by reducing internal covariate shift". In: *Proceedings of the 32nd International Conference on Machine Learning*. Lille, France: PMLR, 2015, pp. 448–456. DOI: 10.5555/3045118.3045167.

[41] Diederik P Kingma and Jimmy Ba. "Adam: A method for stochastic optimization". In: [2015]. Originally published as arXiv:1412.6980 (2014). arXiv: 1412.6980 [cs.LG].

[42] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep learning". In: *Nature* 521.7553 [2015], pp. 436–444. DOI: 10.1038/nature14539.

[43] Jonathan Long, Evan Shelhamer, and Trevor Darrell. "Fully convolutional networks for semantic segmentation". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2015, pp. 3431–3440. DOI: 10.1109/TPAMI.2016.2572683.

[44] Shaoqing Ren et al. "Faster R-CNN: Towards real-time object detection with region proposal networks". In: *Advances in Neural Information Processing Systems 28*. Curran Associates, Inc., 2015, pp. 91–99.

[45] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. "U-net: Convolutional networks for biomedical image segmentation". In: *Medical Image Computing and Computer-Assisted Intervention (MICCAI 2015)*. Cham: Springer, 2015, pp. 234–241. DOI: 10.1007/978-3-319-24574-4_28.

[46] Karen Simonyan and Andrew Zisserman. "Very deep convolutional networks for large-scale image recognition". In: *3rd International Conference on Learning Representations (ICLR 2015)*. arXiv:1409.1556. 2015.

[47] Christian Szegedy, Wei Liu, et al. "Going deeper with convolutions". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2015, pp. 1–9. DOI: 10.1109/CVPR.2015.7298594.

[48] Naftali Tishby and Noga Zaslavsky. "Deep learning and the information bottleneck principle". In: [2015]. IEEE Information Theory Workshop (ITW) 2015. arXiv: 1503.02406 [cs.LG].

[49] Kelvin Xu et al. "Show, attend and tell: Neural image caption generation with visual attention". In: *Proceedings of the 32nd International Conference on Machine Learning*. Lille, France: PMLR, 2015, pp. 2048–2057. DOI: 10.5555/3045118.3045259.

[50] Martín Abadi et al. "TensorFlow: A system for large-scale machine learning". In: *12th USENIX Symposium on Operating Systems Design and Implementation*. Savannah, GA: USENIX Association, 2016, pp. 265–283.

[51] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. "Layer normalization". In: [2016]. arXiv: 1607.06450 [cs.NE].

[52] Tianqi Chen et al. "Training deep nets with sublinear memory cost". In: *arXiv preprint arXiv:1604.06174* [2016]. arXiv: 1604.06174 [cs.LG].

[53] Yarin Gal and Zoubin Ghahramani. "Dropout as a Bayesian approximation: Representing model uncertainty in deep learning". In: *Proceedings of the 33rd International Conference on Machine Learning*. PMLR, 2016, pp. 1050–1059.

[54] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. Adaptive Computation and Machine Learning. MIT Press, 2016. ISBN: 978-0262035613.

[55] Song Han, Huizi Mao, and William J Dally. "Deep compression: Compressing deep neural networks with pruning, trained quantization and Huffman coding". In: *3rd International Conference on Learning Representations (ICLR 2016)*. 2016. arXiv: 1510.00149 [cs.CV].

[56] Kaiming He, Xiangyu Zhang, et al. "Deep residual learning for image recognition". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90.

[57] Kaiming He, Xiangyu Zhang, et al. "Identity mappings in deep residual networks". In: *Computer Vision – ECCV 2016*. Cham: Springer, 2016, pp. 630–645. DOI: 10.1007/978-3-319-46493-0_38.

[58] Dan Hendrycks and Kevin Gimpel. "Gaussian Error Linear Units (GELUs)". In: *arXiv preprint arXiv:1606.08415* [2016]. arXiv: 1606.08415 [cs.LG].

[59] Gao Huang, Yu Sun, et al. "Deep networks with stochastic depth". In: *Proceedings of the European Conference on Computer Vision*. Springer, 2016, pp. 646–661. DOI: 10.1007/978-3-319-46493-0_39.

[60] Wenjie Luo et al. "Understanding the effective receptive field in deep convolutional neural networks". In: *Advances in Neural Information Processing Systems 29*. 2016. arXiv: 1701.04128 [cs.CV].

[61] Fausto Milletari, Nassir Navab, and Seyed-Ahmad Ahmadi. "V-Net: Fully convolutional neural networks for volumetric medical image segmentation". In: [2016], pp. 565–571. DOI: 10.1109/3DV.2016.79.

[62] Joseph Redmon et al. "You only look once: Unified, real-time object detection". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2016, pp. 779–788. DOI: 10.1109/CVPR.2016.91.

[63] Rico Sennrich, Barry Haddow, and Alexandra Birch. "Neural Machine Translation of Rare Words with Subword Units". In: *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (ACL)* [2016], pp. 1715–1725. arXiv: 1508.07909 [cs.CL].

[64] Christian Szegedy, Vincent Vanhoucke, et al. "Rethinking the inception architecture for computer vision". In: [2016], pp. 2818–2826. DOI: 10.1109/CVPR.2016.308.

[65] Dmitry Ulyanov, Andrea Vedaldi, and Victor Lempitsky. "Instance normalization: The missing ingredient for fast stylization". In: [2016]. arXiv: 1607.08022 [cs.NE].

[66] Liang-Chieh Chen et al. "Rethinking atrous convolution for semantic image segmentation". In: *arXiv preprint arXiv:1706.05587* [2017]. arXiv: 1706.05587 [cs.CV].

[67] Francois Chollet. *Deep Learning with Python*. Manning Publications, 2017.

[68] Priya Goyal et al. "Accurate, Large Minibatch SGD: Training ImageNet in 1 Hour". In: *arXiv preprint arXiv:1706.02677* [2017]. arXiv: 1706.02677 [cs.CV].

[69] Kaiming He, Georgia Gkioxari, et al. "Mask R-CNN". In: *Proceedings of the IEEE International Conference on Computer Vision*. IEEE, 2017, pp. 2961–2969. DOI: 10.1109/ICCV.2017.322.

[70] Elad Hoffer, Itay Hubara, and Daniel Soudry. "Train Longer, Generalize Better: Closing the Generalization Gap in Large Batch Training of Neural Networks". In: *Advances in Neural Information Processing Systems (NeurIPS)* 30 [2017].

[71] Andrew G Howard et al. "MobileNets: Efficient convolutional neural networks for mobile vision applications". In: *arXiv preprint arXiv:1704.04861* [2017]. arXiv: 1704.04861 [cs.CV].

[72] Gao Huang, Zhuang Liu, et al. "Densely connected convolutional networks". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2017, pp. 4700–4708. DOI: 10.1109/CVPR.2017.243.

[73] Nitish Shirish Keskar et al. "On Large-Batch Training for Deep Learning: Generalization Gap and Sharp Minima". In: *International Conference on Learning Representations (ICLR)* [2017]. arXiv: 1609.04836 [cs.LG].

[74] James Kirkpatrick et al. "Overcoming Catastrophic Forgetting in Neural Networks". In: *Proceedings of the National Academy of Sciences* 114.13 [2017], pp. 3521–3526. DOI: 10.1073/pnas.1611835114.

[75] Tsung-Yi Lin, Piotr Dollár, et al. "Feature pyramid networks for object detection". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2017, pp. 2117–2125. DOI: 10.1109/CVPR.2017.106.

[76] Tsung-Yi Lin, Priya Goyal, et al. "Focal loss for dense object detection". In: *Proceedings of the IEEE International Conference on Computer Vision*. IEEE, 2017, pp. 2980–2988. DOI: 10.1109/ICCV.2017.324.

[77] Ilya Loshchilov and Frank Hutter. "SGDR: Stochastic gradient descent with warm restarts". In: [2017]. Originally published as ICLR 2017 (workshop paper from ICLR 2016). arXiv: 1608.03983 [cs.LG].

[78] Prajit Ramachandran, Barret Zoph, and Quoc V Le. "Searching for Activation Functions". In: *arXiv preprint arXiv:1710.05941* [2017]. arXiv: 1710.05941 [cs.NE].

[79] Leslie N Smith. "Cyclical learning rates for training neural networks". In: *Proceedings of the IEEE Winter Conference on Applications of Computer Vision*. IEEE, 2017, pp. 464–472. DOI: 10.1109/WACV.2017.58.

[80] Ashish Vaswani et al. "Attention is all you need". In: [2017]. NeurIPS 2017. arXiv: 1706.03762 [cs.CL].

[81] Saining Xie et al. "Aggregated residual transformations for deep neural networks". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2017, pp. 1492–1500. DOI: 10.1109/CVPR.2017.156.

[82] Jie Hu, Li Shen, and Gang Sun. "Squeeze-and-excitation networks". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2018, pp. 7132–7141. DOI: 10.1109/CVPR.2018.00745.

[83] Alex Kendall, Yarin Gal, and Roberto Cipolla. "Multi-Task Learning Using Uncertainty to Weigh Losses for Scene Geometry and Semantics". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* [2018], pp. 7482–7491.

[84] Paul Micikevicius et al. "Mixed precision training". In: *6th International Conference on Learning Representations (ICLR 2018)*. 2018. arXiv: 1710.03740 [cs.LG].

[85] Aaron van den Oord, Yazhe Li, and Oriol Vinyals. "Representation learning with contrastive predictive coding". In: [2018]. arXiv: 1807.03748 [cs.LG].

[86] Sashank J Reddi, Satyen Kale, and Sanjiv Kumar. "On the convergence of Adam and beyond". In: *6th International Conference on Learning Representations (ICLR 2018)*. 2018. arXiv: 1904.09237 [cs.LG].

[87] Mark Sandler et al. "MobileNetV2: Inverted residuals and linear bottlenecks". In: [2018], pp. 4510–4520. DOI: 10.1109/CVPR.2018.00474.

[88] Leslie N Smith. "A disciplined approach to neural network hyper-parameters: Part 1 – learning rate, batch size, momentum, and weight decay". In: *NeurIPS Workshop*. 2018. arXiv: 1803.09820 [cs.LG].

[89] Xiaolong Wang et al. "Non-local neural networks". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2018, pp. 7794–7803. DOI: 10.1109/CVPR.2018.00813.

[90] Sanghyun Woo et al. "CBAM: Convolutional block attention module". In: *Proceedings of the European Conference on Computer Vision*. Springer, 2018, pp. 3–19. DOI: 10.1007/978-3-030-01234-2_1.

[91] Yuxin Wu and Kaiming He. "Group normalization". In: *Proceedings of the European Conference on Computer Vision*. Springer, 2018, pp. 3–19. DOI: 10.1007/978-3-030-01234-2_1.

[92] Hongyi Zhang et al. "mixup: Beyond Empirical Risk Minimization". In: *International Conference on Learning Representations (ICLR)* [2018]. arXiv: 1710.09412 [cs.LG].

[93] Xiangyu Zhang et al. "ShuffleNet: An extremely efficient convolutional neural network for mobile devices". In: *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*. IEEE, 2018, pp. 6848–6856. DOI: 10.1109/CVPR.2018.00716.

[94] Ekin D Cubuk, Barret Zoph, Dandelion Mane, et al. "AutoAugment: Learning Augmentation Policies from Data". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* [2019], pp. 113–123. arXiv: 1805.09501 [cs.CV].

[95] Jacob Devlin et al. "BERT: Pre-training of deep bidirectional transformers for language understanding". In: [2019], pp. 4171–4186. DOI: 10.18653/v1/N19-1423.

[96] Ilya Loshchilov and Frank Hutter. "Decoupled Weight Decay Regularization". In: *International Conference on Learning Representations (ICLR)* [2019]. arXiv: 1711.05101 [cs.LG].

[97] Ilya Loshchilov and Frank Hutter. "Decoupled weight decay regularization". In: [2019]. Originally published as arXiv:1711.05101 (2017). arXiv: 1711.05101 [cs.LG].

[98] Diganta Misra. "Mish: A self regularized non-monotonic neural activation function". In: [2019]. arXiv preprint. arXiv: 1908.08681 [cs.LG].

[99] Rafael Müller, Simon Kornblith, and Geoffrey E Hinton. "When Does Label Smoothing Help?" In: *Advances in Neural Information Processing Systems 32 (NeurIPS)*. 2019.

[100] Adam Paszke et al. "PyTorch: An imperative style, high-performance deep learning library". In: [2019].

[101] Alec Radford et al. "Language models are unsupervised multitask learners". In: [2019]. OpenAI Technical Report GPT-2.

[102] Connor Shorten and Taghi M Khoshgoftaar. "A survey on deep learning: Concepts, architectures, and algorithms". In: [2019]. arXiv: 1908.08994 [cs.LG].

[103] Mingxing Tan and Quoc Le. "EfficientNet: Rethinking model scaling for convolutional neural networks". In: *Proceedings of the 36th International Conference on Machine Learning*. PMLR, 2019, pp. 6105–6114.

[104] Sangdoo Yun et al. "CutMix: Regularization Strategy to Train Strong Classifiers with Localizable Features". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)* [2019], pp. 6023–6032. arXiv: 1905.04899 [cs.CV].

[105] Tom B Brown et al. "Language models are few-shot learners". In: [2020], pp. 1877–1901. arXiv: 2005.14165 [cs.CL].

[106] Ting Chen et al. "A simple framework for contrastive learning of visual representations". In: *Proceedings of the 37th International Conference on Machine Learning*. PMLR, 2020, pp. 1597–1607.

[107] Ekin D Cubuk, Barret Zoph, Jon Shlens, et al. "RandAugment: Practical Automated Data Augmentation with a Reduced Search Space". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* [2020], pp. 3008–3017. arXiv: 1909.13719 [cs.CV].

[108] Xiaocong Cui et al. *Cutting catalog impact: A comparison of neural network approaches on medium-size datasets*. arXiv preprint. 2020.

[109] Jean Deotte et al. "GridMask data augmentation". In: 2020. arXiv: 2001.04086 [cs.CV].

[110] Jared Kaplan et al. "Scaling laws for neural language models". In: *arXiv preprint arXiv:2001.08361* [2020]. arXiv: 2001.08361 [cs.LG].

[111] Samyam Rajbhandari et al. "ZeRO: Memory Optimizations Toward Training Trillion Parameter Models". In: *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis*. 2020.

[112] Noam Shazeer. "GLU variants improve transformer". In: *arXiv preprint arXiv:2002.05202*. 2020. arXiv: 2002.05202 [cs.CL].

[113] Alexey Dosovitskiy et al. "An image is worth 16x16 words: Transformers for image recognition at scale". In: *9th International Conference on Learning Representations (ICLR 2021)*. 2021. arXiv: 2010.11929 [cs.CV].

[114] Ze Liu et al. "Swin transformer: Hierarchical vision transformer using shifted windows". In: *Proceedings of the IEEE/CVF International Conference on Computer Vision*. IEEE, 2021, pp. 10012–10022. DOI: 10.1109/ICCV48922.2021.00971.

[115] Jianlin Su et al. "RoFormer: Enhanced transformer with rotary position embedding". In: *arXiv preprint arXiv:2104.09864* [2021]. arXiv: 2104.09864 [cs.CL].

[116] Hugo Touvron, Matthieu Cord, et al. "Training data-efficient image transformers and distillation through attention". In: *Proceedings of the 38th International Conference on Machine Learning*. Virtual: PMLR, 2021, pp. 10347–10357. DOI: 10.5555/3454284.3455222.

[117] Kevin Clark. "A unified view of GELU activation and dropout". In: *arXiv preprint*. 2022. arXiv: 2209.03448 [cs.LG].

[118] Tri Dao. "FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness". In: *Advances in Neural Information Processing Systems (NeurIPS)* 35 [2022].

[119] Kaiming He, Xinlei Chen, et al. "Masked Autoencoders Are Scalable Vision Learners". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* [2022], pp. 16000–16009. arXiv: 2111.06377 [cs.CV].

[120] Kaiming He, Xinlei Chen, et al. "Masked autoencoders are scalable vision learners". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 2022, pp. 16000–16009. DOI: 10.1109/CVPR52688.2022.01553.

[121] Jordan Hoffmann et al. "Training Compute-Optimal Large Language Models". In: *Advances in Neural Information Processing Systems (NeurIPS)* 35 [2022].

[122] Edward J Hu et al. "LoRA: Low-rank adaptation of large language models". In: [2022]. arXiv: 2106.09685 [cs.CL].

[123] Zhuang Liu et al. "A convnet for the 2020s". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 2022, pp. 11976–11986. DOI: 10.1109/CVPR52688.2022.01167.

[124] Ofir Press, Noah A Smith, and Mike Lewis. "Train short, test long: Attention with linear biases enables input length extrapolation". In: *Proceedings of the 10th International Conference on Learning Representations*. 2022. arXiv: 2108.12409 [cs.LG].

[125] Robin Rombach et al. "High-Resolution Image Synthesis with Latent Diffusion Models". In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* [2022], pp. 10684–10695. arXiv: 2112.10752 [cs.CV].

[126] Jason Wei et al. "Emergent Abilities of Large Language Models". In: *arXiv preprint arXiv:2206.07682* [2022]. arXiv: 2206.07682 [cs.CL].

[127] Peter Wei and Krzysztof Mckeown. "Symbolic Knowledge Distillation: from General Language Models to Commonsense Models". In: *arXiv preprint arXiv:2210.04678* [2022]. arXiv: 2210.04678 [cs.CL].

[128] Samuel G Müller and Frank Hutter. "Torchvision's Mobile Vision Application: The Effect of Trivial Augmentations on Image Classification". In: *arXiv preprint arXiv:2305.10813* [2023]. arXiv: 2305.10813 [cs.CV].

[129] Hugo Touvron, Thibaut Lavril, et al. "LLaMA: Open and Efficient Foundation Language Models". In: *arXiv preprint arXiv:2302.13971* [2023]. arXiv: 2302.13971 [cs.CL].

[130] Ujjwal Maulik et al. "StarReLU: Are self-gated activations better than their sibling ReLU for transformers?" In: *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*. IEEE, 2024. DOI: 10.1109/CVPR52700.2024.01549.

Chapter A

数学参考

A.1 线性代数

A.1.1 矩阵运算

- $\mathbf{AB} \neq \mathbf{BA}$（一般来说不可交换）
- $(\mathbf{AB})^\top = \mathbf{B}^\top \mathbf{A}^\top$
- $(\mathbf{AB})^{-1} = \mathbf{B}^{-1}\mathbf{A}^{-1}$
- $\det(\mathbf{AB}) = \det(\mathbf{A})\det(\mathbf{B})$
- $\mathrm{tr}(\mathbf{AB}) = \mathrm{tr}(\mathbf{BA})$

A.1.2 范数

对于向量 $\mathbf{x} \in \mathbb{R}^n$：

$$\|\mathbf{x}\|_1 = \sum_{i=1}^{n} |x_i| \quad \text{（L1 范数）} \tag{A.1}$$

$$\|\mathbf{x}\|_2 = \sqrt{\sum_{i=1}^{n} x_i^2} \quad \text{（L2 / 欧几里得范数）} \tag{A.2}$$

$$\|\mathbf{x}\|_p = \left(\sum_{i=1}^{n} |x_i|^p\right)^{1/p} \quad \text{（L}p\text{ 范数）} \tag{A.3}$$

$$\|\mathbf{x}\|_\infty = \max_i |x_i| \quad \text{（L}\infty\text{ 范数）} \tag{A.4}$$

对于矩阵 $\mathbf{A}$：

$$\|\mathbf{A}\|_F = \sqrt{\sum_{i,j} A_{ij}^2} \quad \text{（Frobenius 范数）} \tag{A.5}$$

$$\|\mathbf{A}\|_2 = \sigma_{\max}(\mathbf{A}) \quad \text{（谱范数 = 最大奇异值）} \tag{A.6}$$

A.1.3 特征值分解

对于方阵 $\mathbf{A} \in \mathbb{R}^{n\times n}$，如果可对角化，可以分解为：

$$\mathbf{A} = \mathbf{Q}\boldsymbol{\Lambda}\mathbf{Q}^{-1} \tag{A.7}$$

其中 $\boldsymbol{\Lambda} = \mathrm{diag}(\lambda_1, \dots, \lambda_n)$ 是特征值，$\mathbf{Q}$ 包含特征向量。

性质：

- $\det(\mathbf{A}) = \prod_{i=1}^{n} \lambda_i$
- $\mathrm{tr}(\mathbf{A}) = \sum_{i=1}^{n} \lambda_i$
- 当且仅当所有 $\lambda_i > 0$ 时，$\mathbf{A}$ 是正定的
- $\mathbf{A}^k = \mathbf{Q}\boldsymbol{\Lambda}^k\mathbf{Q}^{-1}$

A.1.4 奇异值分解（SVD）

任意矩阵 $\mathbf{A} \in \mathbb{R}^{m\times n}$ 可以分解为：

$$\mathbf{A} = \mathbf{U}\boldsymbol{\Sigma}\mathbf{V}^\top \tag{A.8}$$

其中 $\mathbf{U} \in \mathbb{R}^{m\times m}$（左奇异向量），$\boldsymbol{\Sigma} \in \mathbb{R}^{m\times n}$（对角奇异值 $\sigma_1 \geq \sigma_2 \geq \cdots \geq 0$），$\mathbf{V} \in \mathbb{R}^{n\times n}$（右奇异向量）。

深度学习中的应用：

- 低秩近似：$\mathbf{A}_k = \mathbf{U}_k\boldsymbol{\Sigma}_k\mathbf{V}_k^\top$（保留前 k 个奇异值）
- 主成分分析（PCA）
- 理解权重矩阵：$\sigma_{\max}(\mathbf{W})$ 控制通过一层的梯度流动

A.2 微积分

A.2.1 常用导数

$$\frac{d}{dx}x^n = nx^{n-1} \tag{A.9}$$

$$\frac{d}{dx}e^x = e^x \tag{A.10}$$

$$\frac{d}{dx}\log(x) = \frac{1}{x} \tag{A.11}$$

$$\frac{d}{dx}\sin(x) = \cos(x) \tag{A.12}$$

$$\frac{d}{dx}\cos(x) = -\sin(x) \tag{A.13}$$

$$\frac{d}{dx}\sigma(x) = \sigma(x)(1-\sigma(x)) \quad \text{（sigmoid）} \tag{A.14}$$

$$\frac{d}{dx}\mathrm{ReLU}(x) = \begin{cases} 1 & x > 0 \\ 0 & x \leq 0 \end{cases} \tag{A.15}$$

$$\frac{d}{dx}\tanh(x) = 1 - \tanh^2(x) \tag{A.16}$$

A.2.2 链式法则

对于 $y = f(g(x))$：

$$\frac{dy}{dx} = f'(g(x)) \cdot g'(x) \tag{A.17}$$

对于多变量函数，链式法则变为：

$$\frac{\partial \mathscr{L}}{\partial \mathbf{x}} = \frac{\partial \mathscr{L}}{\partial \mathbf{y}} \cdot \frac{\partial \mathbf{y}}{\partial \mathbf{x}} \tag{A.18}$$

其中 $\frac{\partial \mathbf{y}}{\partial \mathbf{x}}$ 是 **Jacobian** 矩阵，其元素为 $J_{ij} = \frac{\partial y_i}{\partial x_j}$。

A.2.3 Jacobian 和 Hessian

$\mathbf{f}: \mathbb{R}^n \to \mathbb{R}^m$ 在 $\mathbf{x}$ 处的 **Jacobian**：

$$\mathbf{J}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial f_1}{\partial x_1} & \cdots & \frac{\partial f_1}{\partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial f_m}{\partial x_1} & \cdots & \frac{\partial f_m}{\partial x_n} \end{bmatrix} \in \mathbb{R}^{m \times n} \tag{A.19}$$

$f: \mathbb{R}^n \to \mathbb{R}$ 在 $\mathbf{x}$ 处的 **Hessian**：

$$\mathbf{H}_f(\mathbf{x}) = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix} \in \mathbb{R}^{n \times n} \tag{A.20}$$

深度学习中的应用：

- Jacobian 描述梯度在层之间的流动（对于 RNN 梯度消失分析至关重要）
- Hessian 的特征值决定了损失景观的曲率（尖锐 vs 平坦最小值）
- 二阶方法（牛顿法）使用 $\mathbf{H}^{-1}$ 进行参数更新

A.2.4 泰勒展开

对于 $f(\mathbf{x})$ 在 $\mathbf{x}_0$ 附近的光滑函数：

$$f(\mathbf{x}) = f(\mathbf{x}_0) + \nabla f(\mathbf{x}_0)^\top (\mathbf{x} - \mathbf{x}_0) + \frac{1}{2}(\mathbf{x} - \mathbf{x}_0)^\top \mathbf{H}(\mathbf{x}_0)(\mathbf{x} - \mathbf{x}_0) + \cdots \tag{A.21}$$

A.2.5 拉格朗日乘数法

对于约束优化问题 $\min_{\mathbf{x}} f(\mathbf{x})$，约束条件为 $g(\mathbf{x}) = 0$：

$$\mathscr{L}(\mathbf{x}, \lambda) = f(\mathbf{x}) + \lambda \cdot g(\mathbf{x}) \tag{A.22}$$

解为：$\nabla_{\mathbf{x}} \mathscr{L} = \mathbf{0}$，$\nabla_{\lambda} \mathscr{L} = 0$。

A.3 概率与统计

A.3.1 贝叶斯定理

$$P(A|B) = \frac{P(B|A)P(A)}{P(B)} \tag{A.23}$$

A.3.2 概率分布

$$\text{高斯分布：} \quad p(x) = \frac{1}{\sigma\sqrt{2\pi}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right) \tag{A.24}$$

$$\text{伯努利分布：} \quad P(X=k) = p^k(1-p)^{1-k}, \quad k \in \{0,1\} \tag{A.25}$$

$$\text{类别分布：} \quad P(X=k) = p_k, \quad \sum_k p_k = 1 \tag{A.26}$$

A.3.3 期望与方差

$$\mathbb{E}[X] = \sum_x x \cdot P(X=x) \quad \text{（离散）} \tag{A.27}$$

$$\mathbb{E}[X] = \int x \cdot p(x)\,dx \quad \text{（连续）} \tag{A.28}$$

$$\mathrm{Var}(X) = \mathbb{E}[X^2] - (\mathbb{E}[X])^2 \tag{A.29}$$

$$\mathrm{Var}(aX+b) = a^2\mathrm{Var}(X) \tag{A.30}$$

$$\mathbb{E}[XY] = \mathbb{E}[X]\mathbb{E}[Y] \quad \text{若} X \perp Y \tag{A.31}$$

A.3.4 KL 散度与交叉熵

$$\mathrm{KL}(p\|q) = \sum_x p(x) \log \frac{p(x)}{q(x)} \quad \text{（KL 散度，} \geq 0\text{）} \tag{A.32}$$

$$H(p,q) = -\sum_x p(x) \log q(x) \quad \text{（交叉熵）} \tag{A.33}$$

$$H(p,q) = H(p) + \mathrm{KL}(p\|q) \tag{A.34}$$

A.3.5 信息论

$$H(X) = -\sum_{x} P(X = x) \log P(X = x) \quad \text{（熵）} \tag{A.35}$$

$$I(X;Y) = H(X) - H(X|Y) \quad \text{（互信息）} \tag{A.36}$$

Chapter B

快速参考表

B.1 激活函数

Table B.1 – 激活函数参考

名称	公式	值域	应用场景
Sigmoid	$\sigma(x) = \frac{1}{1+e^{-x}}$	$(0, 1)$	二分类输出
Tanh	$\tanh(x)$	$(-1, 1)$	RNN 隐藏层
ReLU	$\max(0, x)$	$[0, \infty)$	默认（隐藏层）
Leaky ReLU	$\max(\alpha x, x)$	$(-\infty, \infty)$	避免神经元死亡
GELU	$x \cdot \Phi(x)$	$(-\infty, \infty)$	Transformer
Swish	$x \cdot \sigma(\beta x)$	$(-\infty, \infty)$	现代 CNN
Softmax	$\frac{e^{x_i}}{\sum_j e^{x_j}}$	$(0, 1)$	分类输出

B.2 损失函数

Table B.2 – 损失函数参考

损失函数	公式	应用场景
MSE	$\frac{1}{N}\sum_i (y_i - \hat{y}_i)^2$	回归
MAE	$\frac{1}{N}\sum_i \lvert y_i - \hat{y}_i \rvert$	鲁棒回归
交叉熵	$-\sum_c y_c \log \hat{y}_c$	分类
二分类交叉熵	$-y \log \hat{y} - (1-y)\log(1-\hat{y})$	二分类
Huber	$\begin{cases} \frac{1}{2}(y-\hat{y})^2 & \lvert y-\hat{y}\rvert \le \delta \\ \delta(\lvert y-\hat{y}\rvert - \frac{\delta}{2}) & \text{其他} \end{cases}$	鲁棒回归
Focal	$-(1-p_t)^\gamma \log p_t$	数据不平衡
KL 散度	$\sum_c p_c \log \frac{p_c}{q_c}$	分布匹配
Triplet	$\max(0, d^+ - d^- + m)$	度量学习

B.3 常用数据集

Table B.3 – 基准数据集

数据集	任务	规模	类别数	年份
MNIST	分类	70K	10	1998
CIFAR-10	分类	60K	10	2009
CIFAR-100	分类	60K	100	2009
ImageNet-1K	分类	1.2M	1000	2009
COCO	检测/分割	330K	80	2014
LibriSpeech	语音识别	281H	–	2015
SQuAD	问答	150K	–	2016
GLUE	NLP 任务	多样	多样	2018

Chapter C

PyTorch 快速参考

C.1 常用操作

```
import torch
import torch.nn as nn
import torch.nn.functional as F

# 创建张量
x = torch.randn(3, 4)  # 随机正态分布
x = torch.zeros(3, 4)  # 零矩阵
x = torch.ones(3, 4)   # 全一矩阵
x = torch.tensor([1, 2, 3])  # 从数据创建
x = torch.arange(0, 10)  # 0, 1, ..., 9

# 形状操作
y = x.view(1, 12)     # 重塑
y = x.reshape(2, 6)   # 重塑（可能复制）
y = x.permute(1, 0, 2) # 转置维度
y = x.flatten()        # 展平为一维
y = x.squeeze(0)       # 移除大小为 1 的维度
y = x.unsqueeze(0)     # 在位置 0 添加维度

# 数学操作
y = x.sum(dim=0)
y = x.mean(dim=1)
y = x.max(dim=1)       # 返回（值，索引）
y = x.softmax(dim=-1)
y = torch.argmax(x, dim=-1)

# GPU 操作
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
x = x.to(device)
x = x.cuda()
x = x.cpu()
```

Listing C.1 – PyTorch 张量常用操作

C.2 模型定义

```
class ConvBlock(nn.Module):
    def __init__(self, in_ch, out_ch, stride=1):
```

```
        super().__init__()
        self.conv = nn.Conv2d(in_ch, out_ch, 3, stride, 1, bias=False)
        self.bn = nn.BatchNorm2d(out_ch)
        self.relu = nn.ReLU(inplace=True)

    def forward(self, x):
        return self.relu(self.bn(self.conv(x)))

class MyModel(nn.Module):
    def __init__(self, num_classes=10):
        super().__init__()
        self.features = nn.Sequential(
            ConvBlock(3, 64),
            nn.MaxPool2d(2),
            ConvBlock(64, 128),
            nn.AdaptiveAvgPool2d(1)
        )
        self.classifier = nn.Linear(128, num_classes)

    def forward(self, x):
        x = self.features(x)
        x = x.flatten(1)
        return self.classifier(x)
```

Listing C.2 – PyTorch 模型定义模式

C.3 训练工具

```
# 梯度裁剪
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

# 学习率调度器
scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
    optimizer, T_max=100, eta_min=1e-6)
scheduler = torch.optim.lr_scheduler.OneCycleLR(
    optimizer, max_lr=0.01, total_steps=10000)

# 混合精度训练
scaler = torch.amp.GradScaler('cuda')
with torch.amp.autocast('cuda'):
    output = model(data)
    loss = criterion(output, target)
scaler.scale(loss).backward()
scaler.step(optimizer)
scaler.update()

```

```python
# 模型摘要
from torchsummary import summary
summary(model, (3, 224, 224))

# 保存和加载
torch.save(model.state_dict(), 'model.pth')
model.load_state_dict(torch.load('model.pth'))
```

Listing C.3 – 常用训练工具

Chapter D

术语表

D.1 架构术语

ANN 人工神经网络 — 最广泛的神经网络类别

CNN 卷积神经网络 — 专门处理网格状数据（图像、音频频谱图）

RNN 循环神经网络 — 通过隐藏状态循环处理序列数据

LSTM 长短期记忆网络 — 解决梯度消失问题的门控 RNN

GRU 门控循环单元 — 简化的 LSTM，包含两个门（重置门、更新门）

Transformer 基于注意力机制的架构，并行处理序列

ViT 视觉 Transformer — 将 Transformer 架构应用于图像块

GAN 生成对抗网络 — 用于生成建模的生成器-判别器框架

VAE 变分自编码器 — 具有编码器-解码器结构的概率生成模型

自编码器 自监督模型，学习压缩和重建数据

U-Net 具有跳跃连接的编码器-解码器架构，用于图像分割

D.2 训练术语

Epoch（轮次） 对整个训练数据集进行一次完整遍历

Batch（批次） 在一次前向/反向传播中一起处理的一组样本

Batch Size（批量大小） 每个批次的样本数量（如 32、64、256）

Backpropagation（反向传播） 通过反向模式自动微分计算梯度的算法

Learning Rate（学习率） 优化过程中参数更新的步长

Loss Function（损失函数） 模型在训练期间最小化的目标函数

Gradient（梯度） 损失相对于模型参数的偏导数

收敛 损失停止下降的状态（模型已学习数据）

过拟合 模型在训练数据上表现良好，但在未见数据上表现差

泛化 模型在未见数据上表现良好的能力

正则化 防止过拟合的技术（dropout、权重衰减等）

早停 当验证性能下降时停止训练

微调 将预训练模型适配到新任务/数据集

预训练 在任务特定适配之前，在大数据集上训练模型

迁移学习 使用从一个任务获得的知识来改进另一个任务

D.3 层与操作术语

卷积 滑动卷积核提取局部特征的操作

池化 空间下采样操作（最大池化、平均池化）

步长 卷积/池化中滑动窗口的步长

填充 在输入边界周围添加零（或其他值）

卷积核/滤波器 卷积中应用的可学习权重

特征图 卷积层的输出

激活函数 在线形变换之后应用的非线性函数

Dropout 训练期间随机将激活置零以进行正则化

BatchNorm 批归一化 – 在批次维度上归一化特征

LayerNorm 层归一化 – 在特征维度上归一化

Softmax 将 logits 转换为概率分布（和为 1）

Embedding（嵌入） 离散标记（词、块）的密集向量表示

注意力 基于查询-键相似性计算值加权和的机制

自注意力 查询、键、值来自同一序列的注意力

交叉注意力 查询与键/值来自不同序列的注意力

位置编码 序列模型的位置信息编码

残差连接 将输入加到输出的跳跃连接（$\mathbf{y} = \mathbf{x} + \mathscr{F}(\mathbf{x})$）

D.4 性能指标

FLOPs 浮点运算次数 – 计算成本的度量

MACs 乘加运算次数（≈ FLOPs 的一半）

GFLOPs 十亿次 FLOPs（10^9 FLOPs）

Top-1 准确率 最高置信度预测正确的样本比例

Top-5 准确率 正确类别在前 5 个预测中的样本比例

mAP 平均精度均值 – 目标检测的标准指标

IoU 交并比 – 分割/检测的重叠度量

BLEU 双语评估辅助 – 文本生成质量指标

困惑度 交叉熵的指数；语言模型越低越好

吞吐量 每秒处理的样本数（samples/sec）

延迟 处理单个样本的时间（ms）

D.5 其他常用缩写

SGD 随机梯度下降

Adam 自适应矩估计优化器

AdamW 带解耦权重衰减的 Adam

ReLU 线性整流单元激活函数

GELU 高斯误差线性单元激活函数

MSE 均方误差损失

MAE 平均绝对误差损失

KL Kullback-Leibler 散度

PCA 主成分分析

SVD 奇异值分解

AMP 自动混合精度

NAS 神经架构搜索

TTA 测试时增强

QA 问答

NLP 自然语言处理

CV 计算机视觉

致谢

“在科学中重要的不是获得新的事实，而是发现新的思考方式。”

— 威廉 · 劳伦斯 · 布拉格

我们希望这本教材能够为您在神经网络和深度学习方面打下坚实的基础。这个领域不断发展，我们鼓励您保持好奇心，继续探索。

祝您学习愉快！